哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期：2025 春季

课程名称：计算机网络

实验名称：协议栈设计与实现

学生班级：1 班

学生学号：220110120

学生姓名：文梓锋

评阅教师：

报告成绩：

实验与创新实践教育中心制

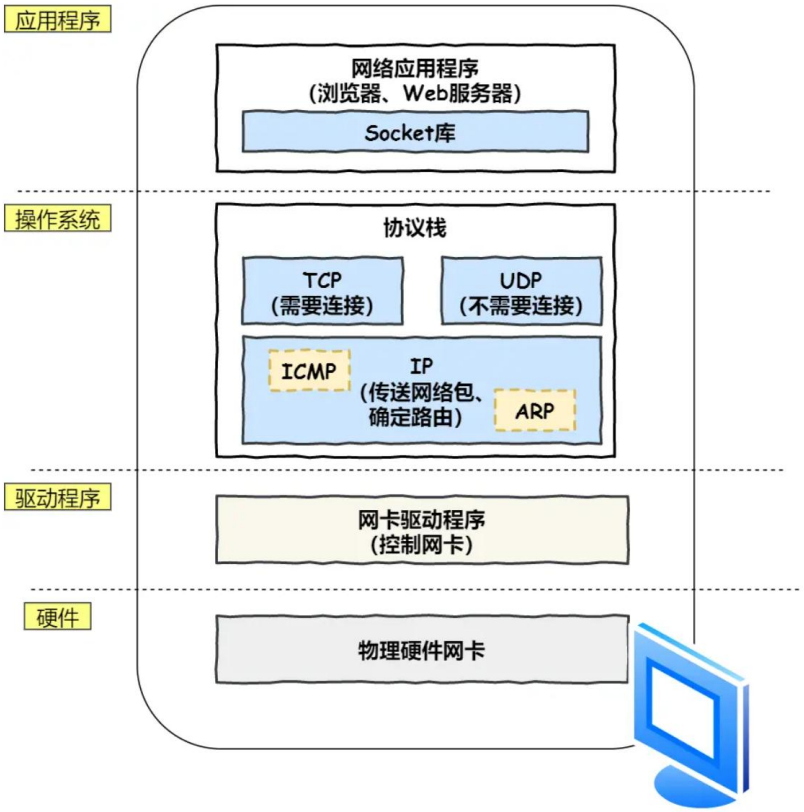
2025 年 3 月

一、 协议实现详述

（注意不要完全照搬实验指导书上的内容，请根据你自己的设计方案来填写图文并茂地描述实验实现的所有功能和详细的设计方案及实验过程中的特色部分。）

1. 请给出协议栈实验的整体流程图

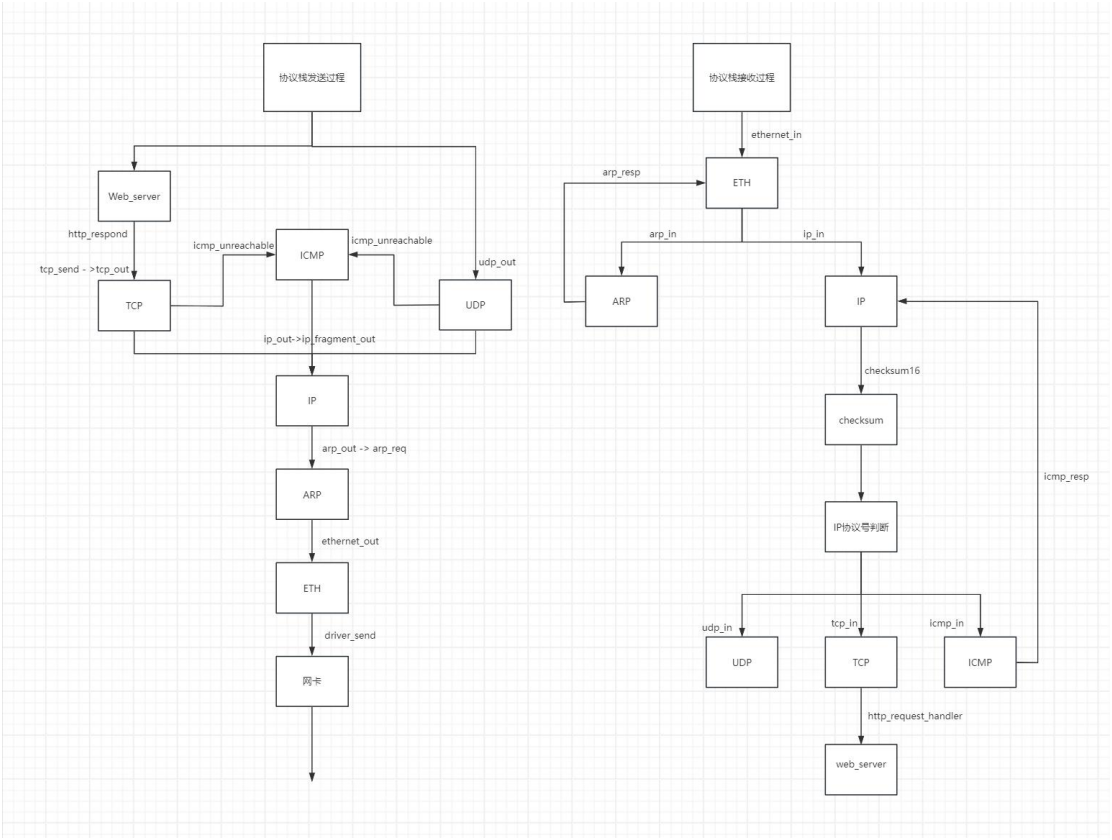
（绘制协议栈实验的整体流程图，涵盖协议栈接收和发送主要步骤，包括 Eth 接收 / 发送、ARP 处理、IP 接收 / 发送、ICMP 处理、UDP 接收 / 发送、TCP 接收 / 发送以及 web 服务器请求处理等步骤，并标注主要函数调用关系。）



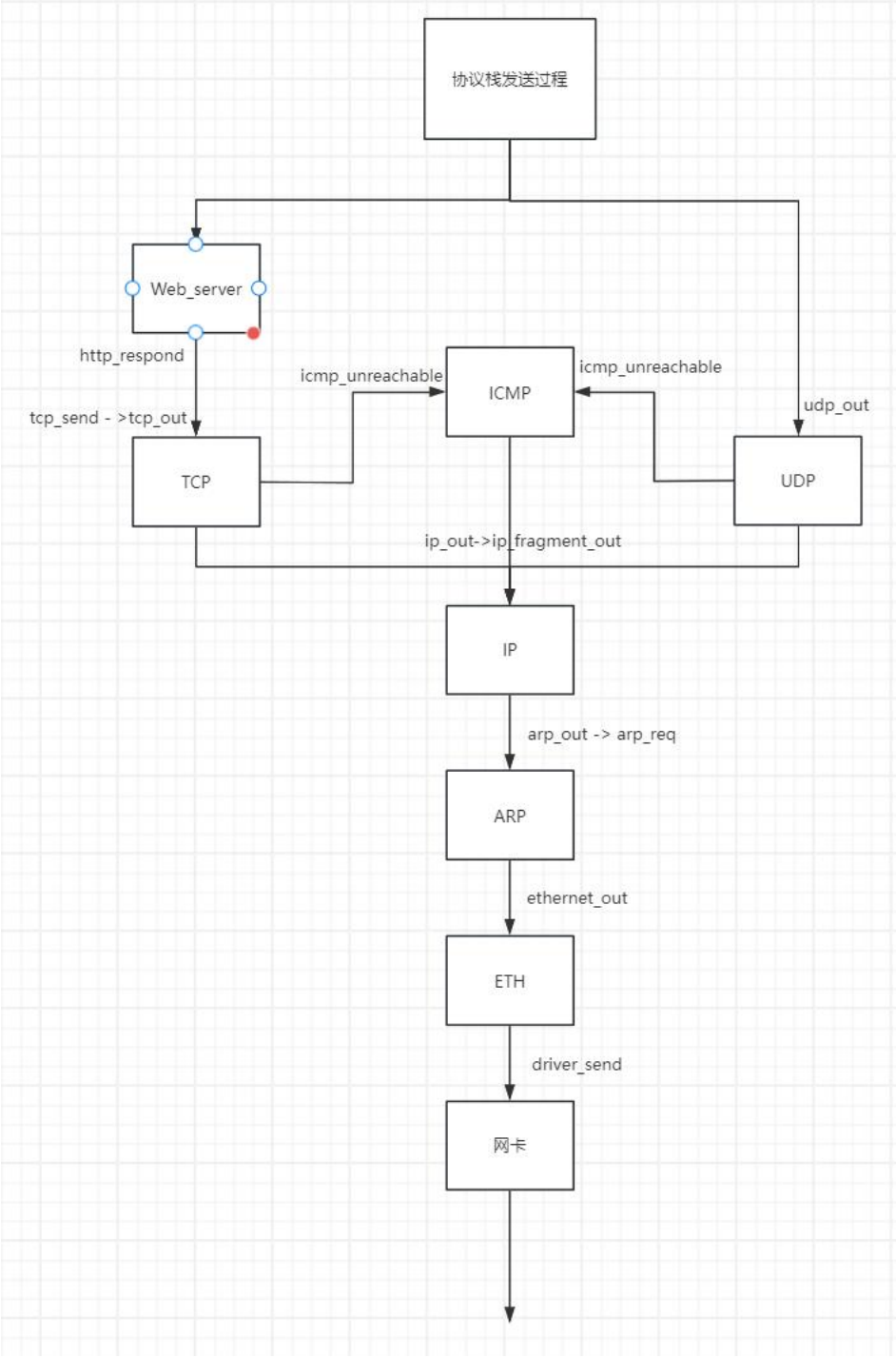
层级	作用	主要函数
Ethernet 层	接收/发送帧	`ethernet_in`, `ethernet_out`
ARP 协议	地址解析	`arp_in`, `arp_req`, `arp_resp`, `arp_out`
IP 层	分发协议、分片、校验	`ip_in`, `ip_out`, `checksum16`, `ip_fragment_out`
ICMP 协议	Ping、不可达等	`icmp_in`, `icmp_resp`, `icmp_unreachable`
UDP 协议	轻量级通信	`udp_in`, `udp_out`, `transport_checksum`
TCP 协议	连接通信	`tcp_in`, `tcp_out`
Web服务器	应用层响应	`http_respond`

绘制的图像如下图所示：

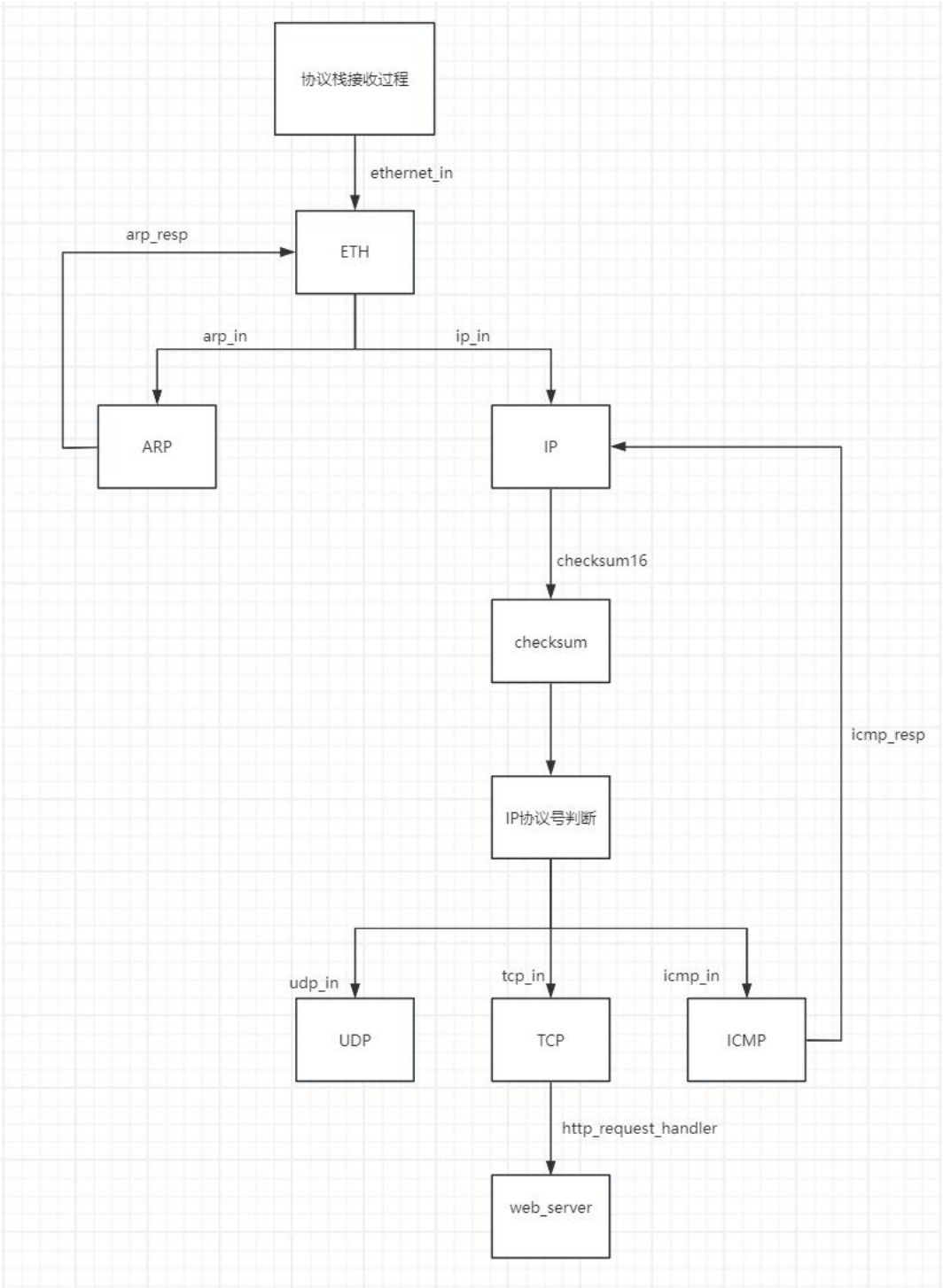
绘制协议栈实验的整体流程图，涵盖协议栈接收和发送主要步骤



协议栈发送过程


协议栈发送过程
Web_server
http_respond
tcp_send - >tcp_out
TCP
icmp_unreachable
ICMP
icmp_unreachable
udp_out
UDP
ip_out->ip_fragment_out
IP
arp_out -> arp_req
ARP
ethernet_out
ETH
driver_send
网卡

协议栈接收过程


协议栈接收过程
ethernet_in
ETH
arp_resp
arp_in
ip_in
ARP
IP
checksum16
checksum
icmp_resp
IP协议号判断
udp_in
tcp_in
icmp_in
UDP
TCP
ICMP
http_request_handler
web_server

Eth 协议详细设计

（描述以太网（Eth）协议的数据封装与解封装过程等。）

【总体设计思路】

以太网（Ethernet，简称 Eth）协议工作在数据链路层，主要负责在主机发送数据时，将上层协议传下来的数据包进行封装，添加以太网帧头信息（包括目的 MAC 地址、源 MAC 地址以及长度/类型字段等）；在接收数据时，则需对从驱动层传来的数据帧进行解封装，解析出帧头中的关键信息（如源 MAC 地址、协议类型等），并去除帧头后将有效负载交给上层处理。

需要特别注意的是，由于网络通信采用大端字节序，而 x86 架构的主机通常使用小端字节序存储数据，因此在封装与解析的过程中，需要对涉及多字节的字段（如协议类型）进行字节序转换，以保证数据在不同平台间正确识别。

以太网数据帧格式

MAC 目标地址	MAC 源地址	长度/类型
6 字节	6 字节	2 字节

- 目的 MAC 地址：表示数据帧接收方网络适配器的物理地址（即 MAC 地址），占用 6 个字节。当网卡收到一个数据帧时，会首先检查该地址字段。若目的地址与本机适配器的 MAC 地址一致，或是广播地址（FF-FF-FF-FF-FF-FF），则会继续处理该帧；否则，该帧将被忽略并丢弃。
- 源 MAC 地址：表示发送方网络适配器的物理地址，占用 6 个字节。
- 长度/类型字段：该字段长度为 2 个字节，用于指示数据帧的长度或上层协议类型。当该值小于 0x05DC（即十进制的 1500）时，表示该字段标识的是数据部分的长度；当该值大于等于 0x0600 时，则表示该字段用于指定上层协议的类型。例如，值为 0x0800 表示帧内数据应交由 IP 协议处理，值为 0x0806 则表示该帧是一个 ARP 请求或应答。

2.1 以太网数据帧发送处理流程（即 ethernet_out 函数）的设计与实现

void ethernet_out(buf_t *buf, const uint8_t *mac, net_protocol_t protocol)

设计流程如下：

1. 数据长度检查与填充：对数据长度进行判断，若数据长度不足 46 字节，需显式填充 0，调用 buf_add_padding()函数
2. 添加以太网包头：调用已经提供的 buf_add_header 函数在 buf 头部添加一段长度，并用指针 hdr 指向这段长度的起始地址，便于后续处理。
3. 填写目的 MAC 地址：根据传入的 mac 参数，使用 memcpy 函数为以太网头部填写目的 MAC 地址。
4. 填写源 MAC 地址，即本机的 MAC 地址：使用 memcpy 函数可以为以太网头部填写源 MAC 地址。目的 MAC 地址和源 MAC 地址都是由 uint8_t 类型数组表示的，每个数组元素占一个字节，因此在复制时无需考虑大小端格式的转换问题。
5. 填写协议类型 protocol：填写以太网头部中的协议类型字段时，由于该字段占两个字节，因此在赋值前需要通过 swap16 函数进行字节序的转换，以确保大小端格式正确。
6. 发送数据帧：将添加完以太网包的数据帧发送到驱动层

代码实现如下：

```
29  /**
30   * @brief 处理一个要发送的数据包
31   *
32   * @param buf 要处理的数据包
33   * @param mac 目标MAC地址
34   * @param protocol 上层协议
35   */
     解释代码 | 注释代码 | 生成单测 | ×
36  void ethernet_out(buf_t *buf, const uint8_t *mac, net_protocol_t protocol) {
37      // TO-DO
38      // Step1: 数据长度检查和填充
39      if (buf->len < 46) {
40          size_t len_padding = 46 - buf->len;
41          buf_add_padding(buf, len_padding);
42      }
43
44      // Step2: 添加以太网包头
45      buf_add_header(buf, sizeof(ether_hdr_t));
46      ether_hdr_t *hdr = (ether_hdr_t *) buf->data;
47
48      // Step3: 填写目的MAC地址
49      memcpy(hdr->dst, mac, NET_MAC_LEN);
50
51      // Step4: 填写源MAC地址，即本机的MAC地址
52      memcpy(hdr->src, net_if_mac, NET_MAC_LEN);
53
54      // Step5: 填写协议类型protocol
55      hdr->protocol16 = swap16(protocol);
56
57      // Step6: 发送数据帧
58      driver_send(buf);
59  }
```

2.2 以太网数据帧接收处理流程（即 ethernet_in 函数）的设计与实现

void ethernet_in(buf_t *buf)

设计流程如下：

1. 数据长度检查：首先判断接收到的数据长度，若数据长度小于以太网头部长度，则说明该数据不完整，属于无效数据，此时应立即丢弃，不再做进一步处理

2. 移除以太网包头：对以太网帧头进行解析，提取协议类型、源 MAC 地址等关键信息。具体操作为：从数据缓冲区 buf 中获取数据起始地址，并将其转换为以太网头部结构体指针 hdr。通过该指针访问以太网头中的 protocol 16 和 src 字段以获取相应信息。其中，protocol 16 为 2 字节字段，因此在读取时需要进行字节序转换（小端转大端），而 src 是一个 uint8_t 数组，每个元素占用 1 字节，因而无需做字节序处理。在成功解析完以太网帧头后，调用 buf_remove_header()函数移除加以太网包头，保留纯净的负载数据。

3. 向上层传递数据包：通过 net_in 函数将去除以太网头部的净荷数据向上传递，交由上层协议进行进一步处理。

代码实现如下：

```
 8   /**
 9    * @brief 处理一个收到的数据包
10    *
11    * @param buf 要处理的数据包
12    */
      解释代码 | 注释代码 | 生成单测 | ×
13   void ethernet_in(buf_t *buf) {
14       // TO-DO
15       // Step1: 数据长度检查
16       if(buf->len < sizeof(ether_hdr_t)) {
17           return;
18       }
19
20       // Step2: 移除以太网包头
21       ether_hdr_t *hdr = (ether_hdr_t *) buf->data;
22       net_protocol_t protocol = swap16(hdr->protocol16);
23       uint8_t *src = hdr->src;
24       buf_remove_header(buf, sizeof(ether_hdr_t));
25
26       // Step3: 向上层传递数据包
27       net_in(buf, protocol, src);
28   }
```

2. ARP 协议详细设计

（描述 ARP 请求/响应处理逻辑、ARP 表项的更新机制等。）

【总体设计思路】

在 TCP/IP 网络体系中，进行通信时并不需要提前获知目标主机的 MAC 地址，只需知道其 IP 地址即可发送 IP 数据报。然而，由于数据链路层的通信依赖于物理地址（即 MAC 地址），而 IP 地址在物理网络中并不能被直接识别，因此需要一种机制将 IP 地址转换为对应的 MAC 地址。这一过程便是通过 ARP（地址解析协议）来完成的。

ARP 协议通过两类消息——ARP 请求与 ARP 响应，实现 IP 地址到 MAC 地址的映射。在启用了 ARP 协议的主机中，通常会维护一张缓存表，用于存储近期接收到的 IP 与 MAC 地址的对应关系。当主机收到一个 ARP 响应包后，会将其中包含的映射关系保存至缓存中。之后在发送数据报时，主机会先查询 ARP 缓存，若已存在目标 IP 对应的 MAC 地址，则直接进行数据传输；若未命中缓存，则触发 ARP 请求，并将当前因地址缺失无法立即发送的报文临时缓存起来，待解析完成后再发送。

此外，当某主机接收到一个 ARP 请求报文，若其中请求的 IP 地址正好是本机的地址，则应当立即生成并发送相应的 ARP 响应包。

因此，本实验的核心任务是实现以下功能模块：

- arp_req：用于发起 ARP 请求；
- arp_out：处理并发送 ARP 协议数据包；
- arp_in：处理接收到的 ARP 数据包；
- arp_resp：构造并发送 ARP 响应报文。

【ARP 表项的更新机制】

无论是主机、交换机还是路由器，都会有一个用于缓存同一网段设备 IP 地址和 MAC 地址的 ARP 映射表，该表用于数据帧的转发。设备通过 ARP 解析到目的 MAC 地址后，会在自己的 ARP 映射表中添加 IP 地址到 MAC 地址的映射项，以便后续向同一目的地转发数据帧。ARP 表项分为动态 ARP 表项和静态 ARP 表项。在本实验中，我们需要实现 动态 ARP 表项。

动态 ARP 表项由 ARP 协议通过 ARP 报文自动生成和维护，这些表项可以老化、被新的 ARP 报文更新，也可能被静态 ARP 表项覆盖。当达到老化时间或接口关闭时，相应的动态 ARP 表项将被删除。

```c
/**
* @brief 内部函数，判断键值对是否有效
*
* @param map 要判断的map
* @param entry 键值对指针
* @return int 1为合法，0为不合法
*/
int map_entry_valid(map_t *map, const void *entry)
{
   time_t entry_time = *(time_t *)((uint8_t *)entry + map->key_len + map->value_len);
   return entry_time && (!map->timeout || entry_time + map->timeout >= time(NULL));
}

/**
* @brief 获取map中指定键的值
*
* @param map 要获取的map
* @param key 键指针
* @return void* 值指针，找不到为NULL
*/
void *map_get(map_t *map, const void *key)
{
   if (key == NULL)
      return NULL;
   for (size_t i = 0; i < map->max_size; i++)
   {
      uint8_t *entry = map_entry_get(map, i);
      if (map_entry_valid(map, entry) && !memcmp(key, entry, map->key_len))
            return entry + map->key_len;
   }
   return NULL;
}
```

3.1 ARP 请求函数（即 arp_req 函数）的设计与实现

设计流程如下：

1. 初始化缓冲区：调用 buf_init() 函数对 txbuf 进行初始化，传入的数据初始长度为 ARP 报文的长度（即 arp_pkt_t 的大小）
2. 填写 ARP 报头：按照 ARP 协议规范，准确填写 ARP 报头信息。用预先定义好的初始 ARP 包 arp_init_pkt 进行填充
3. 设置操作类型：将 ARP 操作类型设置为 ARP_REQUEST，并修改目的 IP 地址为 target_ip。同时要注意进行大小端转换。pkt->opcode16 的大小为两个字节，因此需要进行大小端转换；而 pkt->target_ip 是单个元素大小为一个字节的 uint8_t 数组，因此不需要进行大小端转换
4. 发送 ARP 报文：调用 ethernet_out 函数将 ARP 报文发送出去。需要注意的是，ARP announcement 或 ARP 请求报文均为广播报文，其目标 MAC 地址应设置为广播地址：FF - FF - FF - FF - FF - FF。

代码实现如下：

```
54  /**
55   * @brief 发送一个arp请求
56   *
57   * @param target_ip 想要知道的目标的ip地址
58   */
     解释代码 | 注释代码 | 生成单测 | ×
59  void arp_req(uint8_t *target_ip) {
60      // TO-DO
61      // Step1: 初始化缓冲区 buf_init() txbuf
62      buf_init(&txbuf, sizeof(arp_pkt_t));
63
64      // Step2: 填写ARP报头
65      arp_pkt_t *pkt = (arp_pkt_t *) txbuf.data;
66      memcpy(pkt, &arp_init_pkt, sizeof(arp_pkt_t));
67
68      // Step3: 设置操作类型 ARP_REQUEST
69      pkt->opcode16 = swap16(ARP_REQUEST);
70      memcpy(pkt->target_ip, target_ip, NET_IP_LEN);
71
72      // Step4: 发送ARP报文 ethernet_out
73      ethernet_out(&txbuf, ether_broadcast_mac, NET_PROTOCOL_ARP);
74  }
```

3.2 ARP 发送处理函数（即 arp_out 函数）的设计与实现

设计流程如下：

1. 查找 ARP 表：调用 map_get() 函数，根据 IP 地址在 ARP 表（arp_table）中进行查找。
2. 找到对应 MAC 地址：如果能找到该 IP 地址对应的 MAC 地址，则调用 ethernet_out 函数将数据包直接发送给以太网层。
3. 未找到对应 MAC 地址：若未找到对应的 MAC 地址，需进一步判断 arp_buf 中是否已经有包。若有包，说明正在等待该 IP 回应 ARP 请求，此时不能再发送 ARP 请求；若没有包，则调用 map_set() 函数将来自 IP 层的数据包缓存到 arp_buf 中，然后调用 arp_req() 函数，发送一个请求目标 IP 地址对应的 MAC 地址的 ARP request 报文。

代码实现如下：

```
158  /**
159   * @brief 处理一个要发送的数据包
160   *
161   * @param buf 要处理的数据包
162   * @param ip 目标ip地址
163   * @param protocol 上层协议
164   */
      解释代码 | 注释代码 | 生成单测 | ×
165  void arp_out(buf_t *buf, uint8_t *ip) {
166      // TO-DO
167      // Step1: 查找ARP表 map_get(), 在arp_table中查找
168      uint8_t *MAC = (uint8_t *) map_get(&arp_table, ip);
169
170      // Step2: 找到对应MAC地址
171      // 找到MAC地址，将数据包直接发送给以太网层
172      // 调用ethernet_out函数将数据包发出
173      if(MAC != NULL) {
174          ethernet_out(buf, MAC, NET_PROTOCOL_IP);
175      } else {
176          // Step3: 未找到对应MAC地址
177          // 若未找到对应的MAC地址
178          // 进一步判断arp_buf是否有包
179          buf_t *buf2 = (buf_t *) map_get(&arp_buf, ip);
180          if(buf2 != NULL) {
181              // 有包，正在等待IP回应ARP请求， 不再发送ARP请求
182              return;
183          } else {
184              // 没有包，调用map_set()函数将IP层的数据包缓存到arp_buf
185              // 调用arp_req()函数，发送一个请求目标IP地址对应的MAC地址的ARP request报文
186              map_set(&arp_buf, ip, buf);
187              arp_req(ip);
188          }
189      }
190  }
```

3.3 ARP 接收处理函数（即 arp_in 函数）的设计与实现

设计流程如下：

1. 检查数据长度：首先判断接收到的数据长度，若数据长度小于 ARP 头部长度，则说明该数据包不完整，属于无效数据，此时应立即丢弃，不再做进一步处理

2. 报头检查：对报头进行详细检查，查看报文是否完整。检测 ARP 报头的硬件类型、上层协议类型、MAC 硬件地址长度、IP 协议地址长度、操作类型等等，确保该报头符合协议规定。

3. 更新 ARP 表项：调用 map_set() 函数更新 ARP 表项，更新之前缓存的 IP 地址-MAC 地址映射。

4. 查看缓冲情况：调用 map_get() 函数查看该接收报文的 IP 地址是否有对应的 arp_buf 缓存。

   4.1 有缓存情况：若有缓存，说明 ARP 分组队列里面有待发送的数据包。即上一次调用 arp_out() 函数发送来自 IP 层的数据包时，由于没有找到对应的 MAC 地址而先发送了 ARP request 报文，此时收到了该 request 的应答报文。此时，将缓存的数据包 arp_buf 发送给以太网层，即调用 ethernet_out() 函数将其发出，接着调用 map_delete() 函数将这个缓存的数据包删除。

   4.2 无缓存情况：若该接收报文的 IP 地址没有对应的 arp_buf 缓存，还需要判断接收到的报文是否为 ARP_REQUEST 请求报文，并且该请求报文的 target_ip 是本机的 IP。若是，则认为是请求本主机 MAC 地址的 ARP 请求报文，调用 arp_resp() 函数回应一个响应报文。

代码实现如下：

```
 99  /**
100   * @brief 处理一个收到的数据包
101   *
102   * @param buf 要处理的数据包
103   * @param src_mac 源mac地址
104   */
      解释代码 | 注释代码 | 生成单测 | ×
105  void arp_in(buf_t *buf, uint8_t *src_mac) {
106      // TO-DO
107      // Step1: 检查数据长度
108      // 判断数据长度，若数据长度小于 ARP 头部长度，则认为数据包不完整，将其丢弃，不予处理。
109      if(buf->len < sizeof(arp_pkt_t)) {
110          return;
111      }
112      // Step2: 报头检查,查看报文是否完整
113      arp_pkt_t *arp_pkt = (arp_pkt_t *) buf->data;
114      // 硬件类型
115      if(arp_pkt->hw_type16 != swap16(ARP_HW_ETHER)) {
116          return;
117      }
118      // 上层协议类型
119      if(arp_pkt->pro_type16 != swap16(NET_PROTOCOL_IP)) {
120          return;
121      }
122      // MAC硬件地址长度
123      if(arp_pkt->hw_len != NET_MAC_LEN) {
124          return;
125      }
126      // IP协议地址长度
127      if(arp_pkt->pro_len != NET_IP_LEN) {
128          return;
129      }
```

```
    // 操作类型
    if(arp_pkt->opcode16 != swap16(ARP_REQUEST) && arp_pkt->opcode16 != swap16(ARP_REPLY)) {
        return;
    }

    // Step3: 更新 ARP 表项
    // 调用 map_set() 函数更新 ARP 表项
    map_set(&arp_table, arp_pkt->sender_ip, arp_pkt->sender_mac);

    // Step4: 查看缓存情况
    // 调用 map_get() 函数查看该接收报文的 IP 地址是否有对应的 arp_buf 缓存
    buf_t *buf2 = (buf_t *) map_get(&arp_buf, arp_pkt->sender_ip);
    // 有缓存，说明 ARP 分组队列里面有待发送的数据包。
    if(buf2 != NULL) {
        // 将缓存的数据包 arp_buf 发送给以太网层，调用 ethernet_out() 函数
        ethernet_out(buf2, arp_pkt->sender_mac, NET_PROTOCOL_IP);
        // 调用 map_delete() 函数将这个缓存的数据包删除。
        map_delete(&arp_buf, arp_pkt->sender_ip);
    } else {
        // 无缓存：接收报文的 IP 地址没有对应的 arp_buf 缓存
        // 还需要判断接收到的报文是否为 ARP_REQUEST 请求报文，并且该请求报文的 target_ip 是本机的 IP。
        if(arp_pkt->opcode16 == swap16(ARP_REQUEST) && memcmp(arp_pkt->target_ip, net_if_ip, NET_IP_LEN) == 0) {
            // 若是，则认为是请求本主机 MAC 地址的 ARP 请求报文，调用 arp_resp() 函数回应一个响应报文。
            arp_resp(arp_pkt->sender_ip, arp_pkt->sender_mac);
        }
    }
}

```

3.4 ARP 响应函数（即 arp_resp 函数）的设计与实现

设计流程如下：

1. 初始化缓冲区：调用用 buf_init 函数对 tx_buf 进行初始化，传入的数据初始长度为 ARP 报文的长度（即 arp_pkt_t 的大小）。
2. 填写 ARP 报头首部：按照 ARP 协议规范，准确填写 ARP 报头首部信息。用预先定义好的初始 ARP 包 arp_init_pkt 进行填充，修改 ARP 的操作类型为 ARP_REPLY，并修改目的 IP 地址为 target_ip、目的 MAC 地址为 target_mac。值得注意的是，pkt->opcode16 的大小为两个字节，因此需要进行大小端转换；而 pkt->target_ip、pkt->target_mac 是单个元素大小为一个字节的 uint8_t 数组，因此不需要进行大小端转换
3. 发送 ARP 报文：调用 ethernet_out() 函数将填充好的 ARP 报文发送出去。

代码实现如下：

```
76  /**
77   * @brief 发送一个arp响应
78   *
79   * @param target_ip 目标ip地址
80   * @param target_mac 目标mac地址
81   */
    解释代码 | 注释代码 | 生成单测 | ×
82  void arp_resp(uint8_t *target_ip, uint8_t *target_mac) {
83      // TO-DO
84      // Step1: 初始化缓冲区：
85      // 调用 buf_init() 函数初始化 txbuf。
86      buf_init(&txbuf, sizeof(arp_pkt_t));
87      // Step2: 填写 ARP 报头首部：
88      // 按照 ARP 协议规范，准确填写 ARP 报头首部信息。
89      arp_pkt_t *pkt = (arp_pkt_t *) txbuf.data;
90      memcpy(pkt, &arp_init_pkt, sizeof(arp_pkt_t));
91      pkt->opcode16 = swap16(ARP_REPLY);
92      memcpy(pkt->target_ip, target_ip, NET_IP_LEN);
93      memcpy(pkt->target_mac, target_mac, NET_MAC_LEN);
94      // Step3: 发送 ARP 报文：
95      // 调用 ethernet_out() 函数将填充好的 ARP 报文发送出去。
96      ethernet_out(&txbuf, target_mac, NET_PROTOCOL_ARP);
97  }
```

3. IP 协议详细设计

（描述 IP 数据包的封装与解封装、IP 数据包的分片、校验和计算等。）

【总体设计思路】

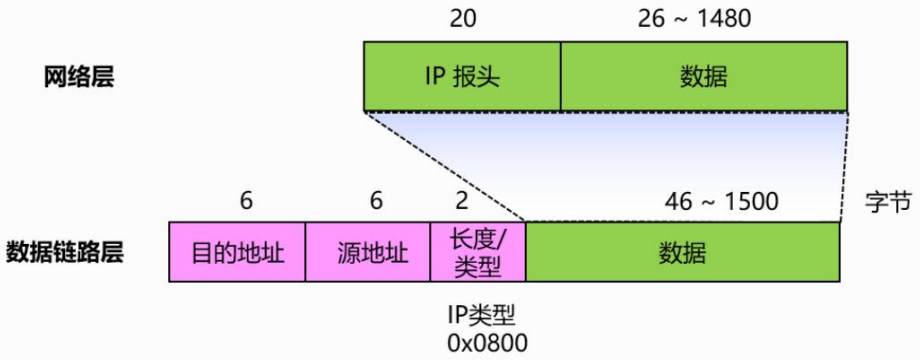
IP 协议属于网络层的核心协议，提供一种无连接、尽力交付的数据传输方式。这意味着 IP 协议并不保证数据报文一定能够成功抵达目的地，也不在路由器中保存任何与传输状态相关的连接信息。每个 IP 数据报都是独立处理的，可能出现乱序到达的情况。正是由于这种设计，IP 协议能够在不同类型的数据链路之间实现终端节点的互联互通。

IP 协议主要包括三个功能模块：IP 地址分配与识别、数据报的分片与重组、路由转发。值得说明的是，当前实验不涉及路由转发部分的实现。

因此，本实验的重点任务集中在以下几个方面：

- 在接收数据时，首先需要解析 IP 头部以提取关键信息，然后将 IP 头部从数据中剥离；
- 在发送数据时，要构造 IP 头部，填充各字段内容，并在必要时对数据包进行分片处理，最后完成数据包的发送操作；
- 由于 IP 首部中包含校验和字段，为确保数据完整性，还需实现用于计算校验和的函数模块。

网络层位于数据链路层之上，其数据包的结构如下所示：



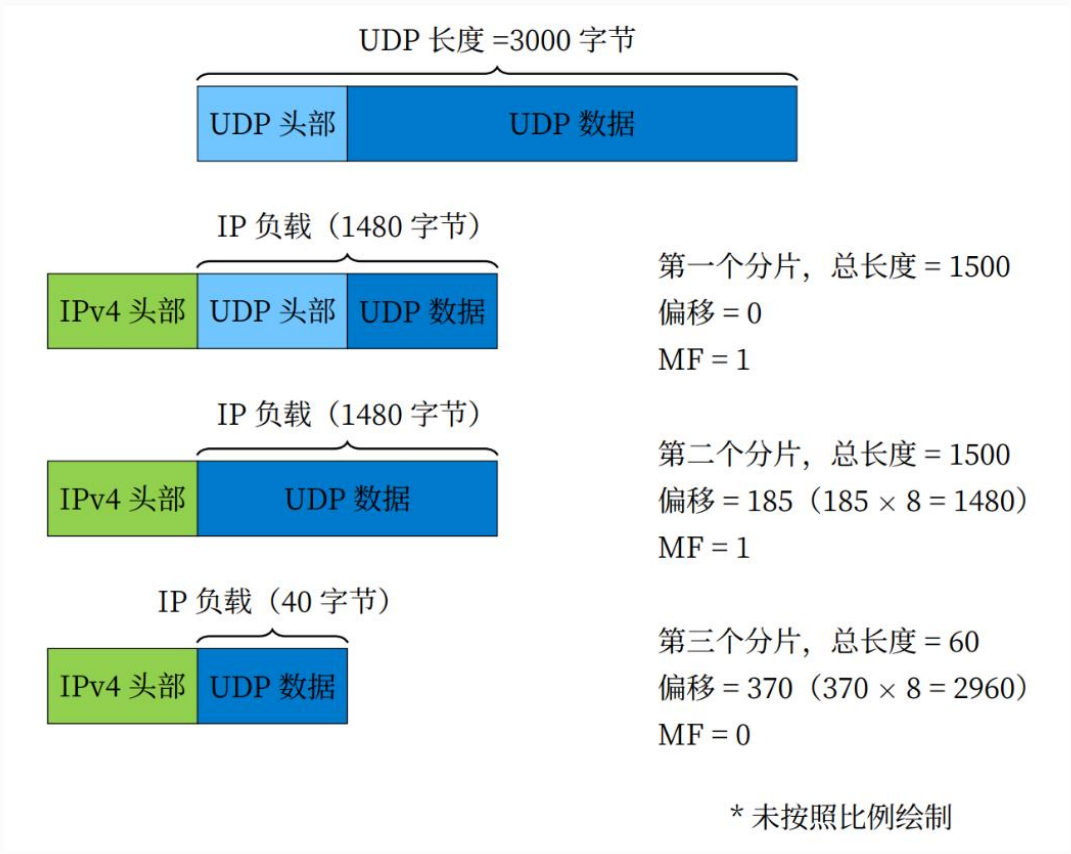
本实验是基于 IPv4 协议展开的，下面将重点介绍 IPv4 版本的 IP 协议。IPv4 的头部结构长度固定为 20 字节，如果包含可变长的选项部分，最多可达 60 字节。

因此，本小节实验在填充或解析 IP 头部的时候，需要考虑的字段主要有：版本号、头部长度、区分服务 TOS、总长度、标识 id、标志、片偏移、生存时间、协议、头部校验和、源 IP 地址、目的 IP 地址等。

【IP 分片】

根据前面的内容我们了解到，IPv4 协议通过 标识（Identification）、标志（Flags） 和片偏移（Fragment Offset） 这三个字段，实现了对大数据报的有效分片与重组。如下图所示，一个总长度为 3000 字节的 UDP 报文（其中 8 字节为 UDP 首部，剩余 2992 字节为负载）被拆分成了三个 IPv4 分片（每个 IP 报文不含选项字段）。

值得注意的是：UDP 头部仅包含在第一个分片中，后续的分片只包含原始负载的其余部分。这些分片在目标主机上通过上述字段进行正确地重组，还原出完整的 UDP 数据报。



【检验和计算】

IP 头部校验和的作用：

- 仅校验 IP 头部，不涉及负载数据；
- 因为 IP 报文转发时头部会改变（如 TTL 递减），负载通常不变；
- 这种设计能 提升路由器的处理效率；
- 上层协议（如 TCP、UDP）自行负责其数据部分的完整性校验。

反码求和（用于计算校验和）步骤如下：

1. 把数据划分为一组组 16bit 半字；
2. 执行逐个加法，如果有进位则 加回结果低位；
3. 若最终加和仍产生进位，再 加 1；
4. 对最终结果取 按位反码，作为校验值。

校验流程：

- 发送端：先将校验字段设为 0，计算校验和后写入字段；
- 接收端：临时清空校验字段，重新计算校验值，与原值比对是否一致。

若一致，认为数据正确；否则，报文被判定为出错并丢弃。

4.1 IP 数据报输入处理函数（即 ip_in 函数）的设计与实现

void ip_in(buf_t *buf, uint8_t *src_mac)

设计流程如下：

1. 检查数据包长度：若数据包的长度小于 IP 头部长度，表明该数据包不完整，将其丢弃，不予处理。

2. 进行报头检测：对 IP 报头进行详细检查，检查内容至少涵盖：IP 头部的版本号是否为 IPv4，总长度字段是否小于或等于收到的数据包长度等。若不符合这些要求，说明该数据包存在异常，将其丢弃。

3. 校验头部校验和：先把 IP 头部的头部校验和字段用其他变量保存起来，接着将该头部校验和字段置为 0。随后调用 checksum16 函数来计算头部校验和，将计算结果与 IP 头部原本的首部校验和字段进行对比。若不一致，说明数据包在传输过程中可能出现损坏，将其丢弃；若一致，则再将该头部校验和字段恢复成原来的值。

4. 对比目的 IP 地址：对比目的 IP 地址是否为本机的 IP 地址。若不是，说明该数据包并非发送给本机，将其丢弃。

5. 去除填充字段：如果接收到的数据包的长度大于 IP 头部的总长度字段，说明该数据包存在填充字段。此时可调用 buf_remove_padding() 函数去除填充字段。

6. 去掉 IP 报头：调用 buf_remove_header() 函数去掉 IP 报头，以便后续将数据传递给上层协议。

7. 向上层传递数据包：调用 net_in() 函数向上层传递数据包。若遇到不能识别的协议类型，调用 icmp_unreachable() 函数返回 ICMP 协议不可达信息。需要特别注意的是，在 Step 6 中已经去掉了 IP 报头，因此在发出 ICMP 协议报文前，需重新加入 IP 报头，以确保报文格式的完整性。考虑到目前大家还未实现 ICMP 协议，只需调用 icmp_unreachable() 这个空函数，并传入 ICMP_CODE_PROTOCOL_UNREACH（ICMP 协议不可达信息）即可。

说明与注意事项：

- 大部分处理过程实际上是依据 IP 头部的结构逐项提取字段信息，操作流程相对清晰，基本上是“对照格式完成解析”
- 为了支持 ICMP 协议中“不可达”类型的消息发送，建议在开始解析前就对原始 IP 头部进行一次备份，以便在后续需要构造 ICMP 报文时直接使用
- 在进行校验和计算前，应先将 IP 头部中的校验和字段清零，以确保计算结果准确。计算完成后，再将计算出的校验值写入对应字段（如果校验失败，则该报文可直接丢弃，无需恢复）；
- 类似于之前处理协议栈中其他协议的情况，仍需特别留意所有多字节字段的大小端格式转换，确保网络传输的正确性。

代码实现如下：

```c
/**
 * @brief 处理一个收到的数据包
 *
 * @param buf 要处理的数据包
 * @param src_mac 源mac地址
 */
解释代码 | 注释代码 | 生成单测 | ×
void ip_in(buf_t *buf, uint8_t *src_mac) {
    // TO-DO
    // Step1: 检查数据包长度：
    // 数据包的长度小于 IP 头部长度，丢弃
    if(buf->len < sizeof(ip_hdr_t)){
        return;
    }
    // Step2: 进行 IP 报头检测
    // IP 头部的版本号是否为 IPv4
    // 总长度字段是否小于或等于收到的数据包长度
    ip_hdr_t *hdr = (ip_hdr_t *) buf->data;
    if(hdr->version != IP_VERSION_4) {
        return;
    }
    // 总长度字段是否小于或等于收到的数据包长度
    if(swap16(hdr->total_len16) > buf->len) {
        return;
    }
    // Step3: 校验头部校验和
    // 先把 IP 头部的头部校验和字段用其他变量保存起来，接着将该头部校验和字段置为 0。
    uint16_t hdr_checksum16_backup = hdr->hdr_checksum16;
    hdr->hdr_checksum16 = 0;
    // 随后调用 checksum16 函数来计算头部校验和，将计算结果与 IP 头部原本的首部校验和字段进行对比。
    uint16_t hdr_checksum16 = checksum16((uint16_t *) hdr, sizeof(ip_hdr_t));
    // 若不一致，说明数据包在传输过程中可能出现损坏，将其丢弃
    if(hdr_checksum16 != hdr_checksum16_backup) {
        return;
    }
    // 若一致，则再将该头部校验和字段恢复成原来的值。
    hdr->hdr_checksum16 = hdr_checksum16_backup;

    // Step4 对比目的 IP 地址：
    // IP 地址不是本机的 IP 地址，将其丢弃
    if(memcmp(hdr->dst_ip, net_if_ip, NET_IP_LEN) != 0) {
        return;
    }

    // buf备份， 发送ICMP协议不可达信息
    buf_t copy_buf;
    buf_copy(&copy_buf, buf, buf->len);

    // Step5: 去除填充字段
    // 接收到的数据包的长度大于 IP 头部的总长度字段
    // 调用 buf_remove_padding() 函数去除填充字段。
    if(buf->len > swap16(hdr->total_len16)) {
        buf_remove_padding(buf, buf->len - swap16(hdr->total_len16));
    }
    // Step6: 去掉 IP 报头,调用 buf_remove_header()
    uint8_t protocol = hdr->protocol;
    uint8_t *src_ip = hdr->src_ip;
    buf_remove_header(buf, hdr->hdr_len * 4);
    // Step7: 向上层传递数据包
    // 调用 net_in() 函数向上层传递数据包
    int flag = net_in(buf, protocol, src_ip);
    // 若遇到不能识别的协议类型，调用 icmp_unreachable() 函数返回 ICMP 协议不可达信息。
    if(flag == -1) {
        icmp_unreachable(&copy_buf, src_ip, ICMP_CODE_PROTOCOL_UNREACH);
    }
    // 需要特别注意的是，在 Step 6 中已经去掉了 IP 报头
    // 在发出 ICMP 协议报文前，需重新加入 IP 报头，以确保报文格式的完整性。
    // 调用 icmp_unreachable() 这个空函数，并传入 ICMP_CODE_PROTOCOL_UNREACH（ICMP 协议不可达信息）

}
```

4.2 IP 检验和算法处理函数（即 checksum16 函数）的设计与实现

uint16_t checksum16(uint16_t *data, size_t len)

设计流程如下：

1. 按 16 位分组相加：把 data 看成是每 16 个 bit（即 2 个字节）组成一个数，依次相加。由于 16 位加法的结果可能会超过 16 位，因此加法结果需要用 32 位数来保存。
2. 处理剩余 8 位：如果最后还剩 8 个 bit 值，也要将这个 8bit 值与之前的和相加。
3. 循环处理高 16 位：判断相加后 32bit 结果值的高 16 位是否为 0。如果不为 0，则将高 16 位和低 16 位相加，如此循环，直至高 16 位为 0 为止。
4. 取反得到校验和：将上述处理后的和（低 16 位）取反，即得到校验和。

说明与注意事项：

- 校验和的核心计算逻辑是：将 IP 头部按每两个字节划分为一组，并将每组的数值累加（如果最后只剩下一个字节，也需要作为一组处理）。整个累加结果以 32 位整数存储，然后反复将高 16 位与低 16 位相加，直到高位为 0 为止。最终，将得到的低 16 位数值按位取反，便可得到最终的校验和。
- 尽管校验和是 16 位数值，但在本实验中无需处理大小端转换的问题。原因在于：整个加法过程使用了 32 位累加器，任何高位溢出的部分都会被重新加回到结果中。因此，即使在大小端不一致的系统中，最终的校验和结果也不会受到影响。

总结：

执行反码求和的过程中，字节序对最终结果没有影响，因此可以忽略大小端的问题。

代码实现如下：

```
65  /**
66   * @brief 计算16位校验和
67   *
68   * @param buf 要计算的数据包
69   * @param len 要计算的长度
70   * @return uint16_t 校验和
71   */
     解释代码 | 注释代码 | 生成单测 | ×
72  uint16_t checksum16(uint16_t *data, size_t len) {
73      // TO-DO
74      // Step1: 按 16 位分组相加：
75      // 把 data 看成是每 16 个 bit（即 2 个字节）组成一个数，依次相加。
76      // 由于 16 位加法的结果可能会超过 16 位，因此加法结果需要用 32 位数来保存。
77      uint32_t sum = 0;
78
79      for(int i = 0; i < len; i += 2) {
80          if(i == len -1) {
81              // Step2: 处理剩余 8 位
82              // 如果最后还剩 8 个 bit 值，也要将这个 8bit 值与之前的和相加。
83              sum += *((uint8_t *)data + i);
84          } else {
85              sum += data[i / 2];
86          }
87      }
88      // Step3 循环处理高 16 位：
89      // 判断相加后 32bit 结果值的高 16 位是否为 0。
90      // 如果不为 0，则将高 16 位和低 16 位相加，如此循环，直至高 16 位为 0 为止。
91      while (sum >> 16) {
92          sum = (sum & 0xFFFF) + (sum >> 16);
93      }
94      // Step4 取反得到校验和：
95      // 将上述处理后的和（低 16 位）取反，即得到校验和。
96      return (uint16_t)(~sum);
97  }
```

4.3 IP 数据报输出处理函数（即 ip_out 函数）的设计与实现

void ip_out(buf_t *buf, uint8_t *ip, net_protocol_t protocol)

设计流程如下：

1. 检查数据报包长：首先检查从上层传递下来的数据报包长是否大于 IP 协议最大负载包长（1500 字节（MTU）减去 IP 首部长度）。

2. 分片处理：若数据报包长超过 IP 协议最大负载包长，则需要进行分片发送。具体操作如下：

   2.1 首先调用 buf_init() 初始化一个 ip_buf。

   2.2 将数据报包长截断，每个截断后的包长等于 IP 协议最大负载包长（1500 字节 - IP 首部长度），调用 ip_fragment_out() 函数发送出去。

   2.3 如果截断后最后的一个分片小于或等于 IP 协议最大负载包长，调用 buf_init() 初始化一个 ip_buf，其大小等于该分片大小，再调用 ip_fragment_out() 函数发送出去。需要注意的是，最后一个分片的 MF（更多分片标志）应置为 0。

3. 直接发送：若数据报包长没有超过 IP 协议最大负载包长，则直接调用 ip_fragment_out() 函数发送出去。

说明与注意事项：

- 由于受最大传输单元（MTU）的限制，当数据报长度超过 MTU 时必须进行分片传输。而在实际网络环境中，IP 分片后的数据报往往远小于理论上的最大值。因此，建议首先实现一个用于发送单个 IP 分片的函数 ip_fragment_out，然后在 ip_out 函数中根据实际情况调用它来完成整个数据报的发送。
- IP 头部中各字段的填充逻辑已经封装在 ip_fragment_out 函数内部，因此 ip_out 函数只需分别处理是否需要分片的两种情况即可。在处理分片发送过程中，尤其要注意，对于最后一个分片，应确保将“更多分片”（MF）标志位置为 0，以表明分片传输已完成。

代码实现如下：

```c
/**
 * @brief 处理一个要发送的ip数据包
 *
 * @param buf 要处理的包
 * @param ip 目标ip地址
 * @param protocol 上层协议
 */
解释代码 | 注释代码 | 生成单测 | ×
void ip_out(buf_t *buf, uint8_t *ip, net_protocol_t protocol) {
    // TO-DO
    // Step1 检查数据报包长
    // 首先检查从上层传递下来的数据报包长是否大于 IP 协议最大负载包长（1500 字节（MTU）减去 IP 首部长度）。
    int max_load_length = 1500 - sizeof(ip_hdr_t);
    static uint16_t id = 0;
    // Step2 分片处理
    // 若数据报包长超过 IP 协议最大负载包长，则需要进行分片发送。
    if (buf->len > max_load_length) {
        int offset = 0;
        while (offset < buf->len) {
            // 如果截断后最后的一个分片小于或等于 IP 协议最大负载包长，调用 buf_init() 初始化一个 ip_buf，其大小等于该分片大小
            int frag_size = (buf->len - offset > max_load_length) ? max_load_length : (buf->len - offset);
            // 首先调用 buf_init() 初始化一个 ip_buf。
            buf_t ip_buf;
            buf_init(&ip_buf, frag_size);
            memcpy(ip_buf.data, buf->data + offset, frag_size);
            // 需要注意的是，最后一个分片的 MF（更多分片标志）应置为 0。
            int more_fragment = (offset + frag_size < buf->len) ? 1 : 0; // 最后一个分片 MF = 0

            ip_fragment_out(&ip_buf, ip, protocol, id, offset / 8, more_fragment); // 注意 offset 单位为 8 字节
            offset += frag_size;
        }
    } else {
        // Step3 直接发送：
        // 若数据报包长没有超过 IP 协议最大负载包长
        // 直接调用 ip_fragment_out() 函数发送出去。
        ip_fragment_out(buf, ip, protocol, id, 0, 0);
    }
    id ++;
}
```

4.4 IP 分片输出处理函数（即 ip_fragment_out 函数）的设计与实现

void ip_fragment_out(buf_t *buf, uint8_t *ip, net_protocol_t protocol, int id, uint16_t offset, int mf)

设计流程如下：

1. 增加头部缓存空间：调用 buf_add_header() 函数增加 IP 数据报头部缓存空间，为后续填写头部信息做准备。
2. 填写头部字段：按照 IP 协议的规范，填写 IP 数据报头部的各个字段，如版本号、首部长度、区分服务、总长度等。
3. 计算并填写校验和：先把 IP 头部的首部校验和字段填为 0，再调用 checksum16 函数计算校验和，然后把计算出来的校验和填入首部校验和字段。
4. 发送数据：调用 arp_out() 函数将封装后的 IP 头部和数据发送出去。

代码实现如下：

```
77  /**
78   * @brief 处理一个要发送的ip分片
79   *
80   * @param buf 要发送的分片
81   * @param ip 目标ip地址
82   * @param protocol 上层协议
83   * @param id 数据包id
84   * @param offset 分片offset，必须被8整除
85   * @param mf 分片mf标志，是否有下一个分片
86   */
     解释代码 | 注释代码 | 生成单测 | ×
87  void ip_fragment_out(buf_t *buf, uint8_t *ip, net_protocol_t protocol, int id, uint16_t offset, int mf) {
88      // TO-DO
89      // Step1: 调用 buf_add_header() 函数增加 IP 数据报头部缓存空间
90      buf_add_header(buf, sizeof(ip_hdr_t));
91      // Step2: 填写 IP 数据报头部字段
92      ip_hdr_t *hdr = (ip_hdr_t *) buf->data;
93      // 版本号
94      hdr->version = IP_VERSION_4;
95      //首部长度
96      hdr->hdr_len = sizeof(ip_hdr_t) / 4;
97      // 区分服务
98      hdr->tos = 0;
99      // 总长度
100     hdr->total_len16 = swap16(buf->len);
101     hdr->id16 = swap16(id);
102     hdr->flags_fragment16 = swap16((mf ? IP_MORE_FRAGMENT : 0) | offset);
103     hdr->ttl = 64;
104     hdr->protocol = protocol;
105     memcpy(hdr->src_ip, net_if_ip, NET_IP_LEN);
106     memcpy(hdr->dst_ip, ip, NET_IP_LEN);
107     // Step3: 计算并填写校验和
108     // 先把 IP 头部的首部校验和字段填 0
109     // 调用 checksum16 函数计算校验和
110     // 然后把计算出来的校验和填入首部校验和字段
111     hdr->hdr_checksum16 = 0;
112     uint16_t new_checksum = checksum16((uint16_t *)hdr, sizeof(ip_hdr_t));
113     hdr->hdr_checksum16 = new_checksum;
114     // Step4: 调用 arp_out 函数将封装后的 IP 头部和数据发送出去
115     arp_out(buf, ip);
116 }
```

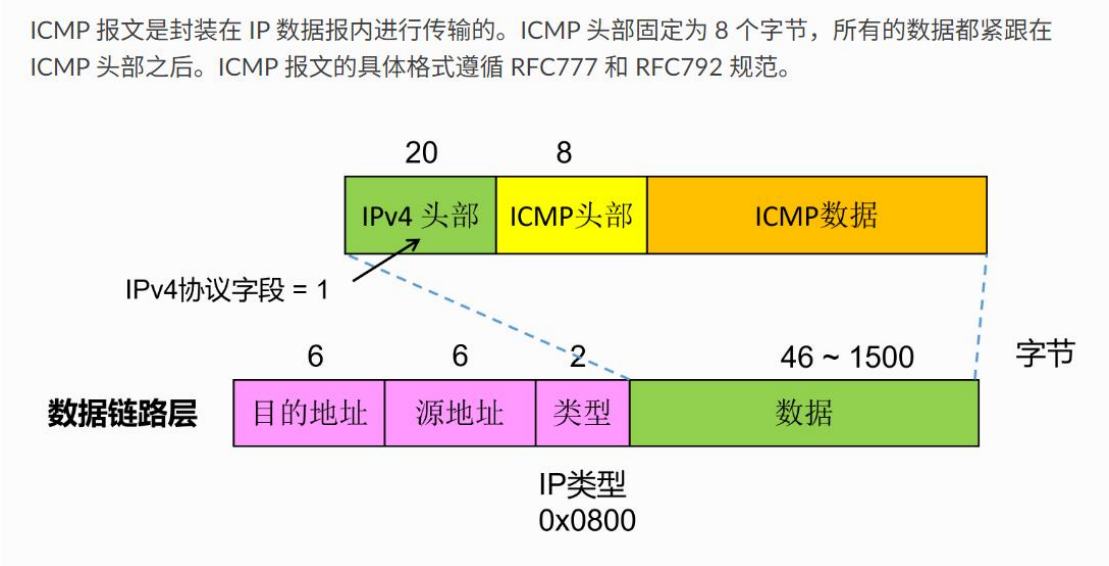
4. ICMP 协议详细设计

（解释如何处理 ICMP 请求和响应，以及如何利用 ICMP 报文进行网络故障诊断等。）

【总体设计思路】

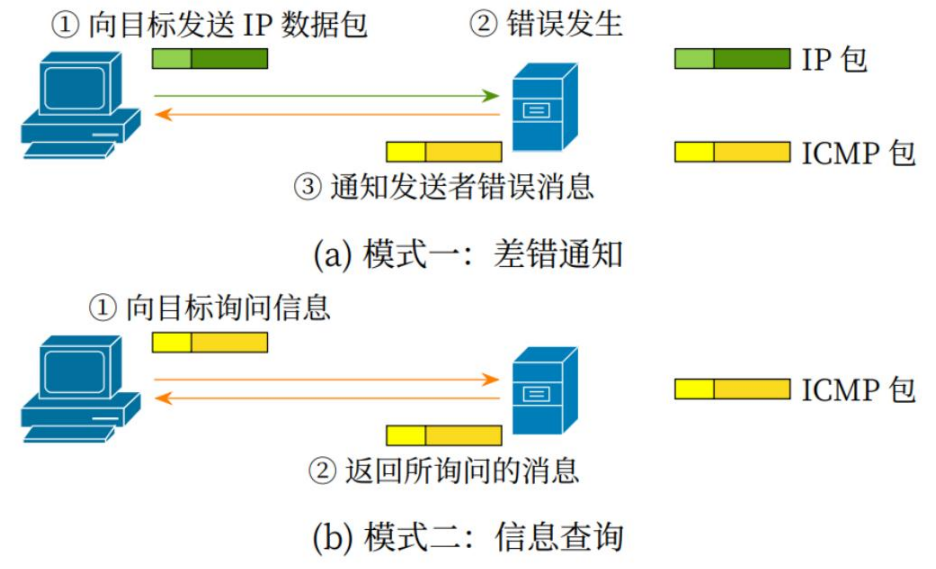
上一部分我们实现了 IP 协议的基本功能，但在实际网络通信中，光有 IP 协议还不足以应对所有情况。例如，IP 协议本身并没有机制来告知发送端某个数据包未能成功到达目的地，也无法执行网络故障的诊断任务。为了弥补这些不足，ICMP 协议应运而生，它通常与 IP 协议配合使用，用于提供与 IP 配置、数据包处理相关的控制和诊断能力。ICMP 的核心作用包括：确认 IP 包是否成功送达目标地址、通知发送过程当中 IP 包被废弃的具体原因、改善网络设置等等。

值得注意的是，ICMP 报文并非独立传输，而是作为数据负载被封装在 IP 数据报中进行传送，其结构如图所示：



ICMP 的消息大致可以分为两类：

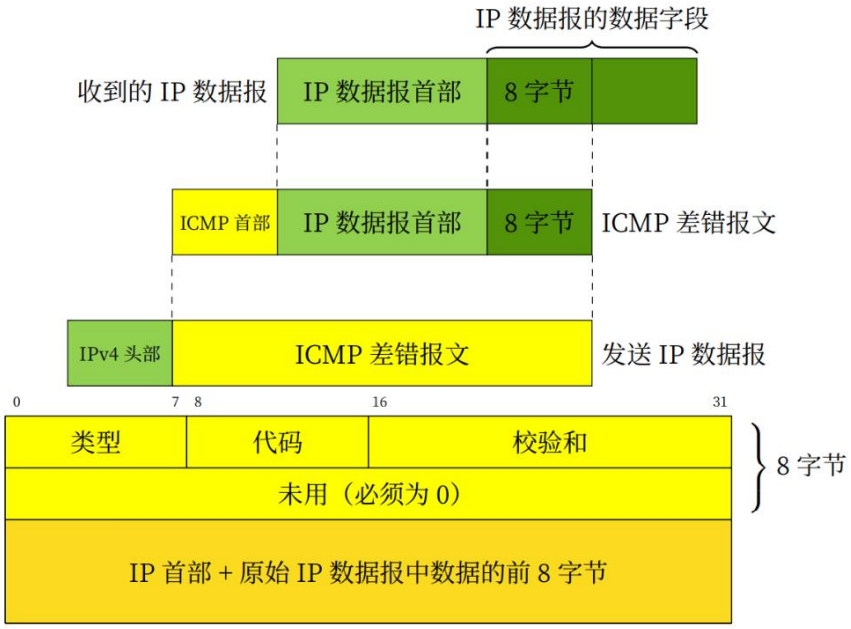
- 错误消息（差错报文，error message）：用于通知通信过程中出现的错误原因。
- 查询消息（询问或信息报文，information message）：主要用于网络诊断。



【ICMP 差错报文结构】

ICMP 差错报文中通常会保留原始 IP 数据报中的前 8 个字节。这是因为在 TCP 和 UDP 协议中，其首部的前 8 字节均包含了源端口号信息。这样一来，接收到 ICMP 差错报文的系统模块就可以依据这些信息，将该报文追溯到对应的上层协议和具体的用户进程。

其中，协议类型可以通过原始 IP 头部的 protocol 字段来识别，而具体的用户进程则可通过这 8 字节中 TCP 或 UDP 头部中的端口号进行定位。



【ICMP 查询报文结构】

1. 类型（Type）：用于说明ICMP报文的作用及格式；
2. 代码（Code）：于详细说明某种ICMP报文的类型；
3. 16位校验和（Checksum）：涵盖了整个报文，其校验和的计算方法和IP协议校验和算法是一样的；
4. 标识符（Identifier）：如果是ICMP请求报文，该字段在Linux/macOS中用的是进程ID。如果是ICMP应答报文，则只需拷贝来自ICMP请求报文的标识符字段；
5. 序列号（Sequence number）：如果是ICMP请求报文，该字段在Linux/macOS中是从0递增的，每个进程独立。如果是ICMP应答报文，则只需拷贝来自ICMP请求报文的序列号字段；
6. 数据（Optional data）：如果是ICMP请求报文，该字段包括时间戳以及一串填充数据。如果是ICMP应答报文，则只需拷贝来自ICMP请求报文的数据字段。

以下是常见的ICMP报文“类型”和“代码”字段用途。

TYPE	CODE	Description
0	0	查询报文：Echo Reply——回显应答（Ping应答）
3	0	差错报文：Network Unreachable——网络不可达
3	1	差错报文：Host Unreachable——主机不可达
3	2	差错报文：Protocol Unreachable——协议不可达
3	3	差错报文：Port Unreachable——端口不可达
8	0	查询报文：Echo request——回显请求（Ping请求）
11	0	差错报文：TTL equals 0 during transit——传输期间生存时间为0
12	0	差错报文：IP header bad (catchall error)——坏的IP首部（包括各种差错）

5.1 ICMP 数据报输入处理函数（即 icmp_in 函数）的设计与实现

void icmp_in(buf_t *buf, uint8_t *src_ip)

设计流程如下：

1. 报头检测：首先检查接收到的数据包长度。若包长小于 ICMP 头部长度，表明数据包不完整，将其丢弃，不做后续处理。
2. 查看 ICMP 类型：接着查看该 ICMP 报文的类型，重点关注是否为回显请求类型。
3. 回送回显应答：如果该报文的 ICMP 类型是回显请求，则调用 icmp_resp() 函数回送一个回显应答（ping 应答）。

代码实现如下：

```
36  /**
37   * @brief 处理一个收到的数据包
38   *
39   * @param buf 要处理的数据包
40   * @param src_ip 源ip地址
41   */
     解释代码 | 注释代码 | 生成单测 | ×
42  void icmp_in(buf_t *buf, uint8_t *src_ip) {
43      // TO-DO
44      // Step1: 报头检测，检查接收到的数据包长度
45      // 若包长小于 ICMP 头部长度，表明数据包不完整，丢弃
46      if(buf->len < sizeof(icmp_hdr_t)) {
47          return;
48      }
49      // Step2: 查看 ICMP 报文的类型：
50      // 重点关注是否为回显请求类型
51      icmp_hdr_t *hdr = (icmp_hdr_t *) buf->data;
52      // Step3: 回送回显应答
53      // 如果报文的 ICMP 类型是回显请求
54      // 则调用 icmp_resp() 函数回送一个回显应答（ping 应答）。
55      if(hdr->type == ICMP_TYPE_ECHO_REQUEST) {
56          icmp_resp(buf, src_ip);
57      }
58  }
```

5.2 ICMP 响应报文发送函数（即 icmp_resp 函数）的设计与实现

static void icmp_resp(buf_t *req_buf, uint8_t *src_ip)

设计流程如下：

1. 初始化并封装数据：调用 buf_init() 函数来初始化 txbuf。之后，封装 ICMP 报头和数据，其中数据部分直接拷贝来自接收的回显请求报文中的数据，以保证信息的一致性。
2. 填写校验和：ICMP 的校验和计算方法与 IP 协议校验和算法相同。按照此算法计算出校验和，并将其填写到相应位置，确保报文在传输过程中的完整性。
3. 发送数据报：调用 ip_out() 函数将封装好且填写了校验和的 ICMP 数据报发送出去。

代码实现如下：

```
/**
 * @brief 发送icmp响应
 *
 * @param req_buf 收到的icmp请求包
 * @param src_ip 源ip地址
 */
解释代码 | 注释代码 | 生成单测 | ×
static void icmp_resp(buf_t *req_buf, uint8_t *src_ip) {
    // TO-DO
    // Step1: 初始化并封装数据
    // 调用 buf_init() 函数来初始化 txbuf
    buf_init(&txbuf, req_buf->len);
    // 封装 ICMP 报头和数据
    // 数据部分直接拷贝来自接收的回显请求报文中的数据，保证信息的一致性。
    memcpy(txbuf.data, req_buf->data, req_buf->len);
    icmp_hdr_t *hdr = (icmp_hdr_t *) txbuf.data;
    icmp_hdr_t *req_hdr = (icmp_hdr_t *) req_buf->data;
    hdr->type = ICMP_TYPE_ECHO_REPLY;
    hdr->code = 0;
    hdr->id16 = req_hdr->id16;
    hdr->seq16 = req_hdr->seq16;
    // Step2: 填写校验和
    // ICMP 的校验和计算方法与 IP 协议校验和算法相同。
    // 并将校验和其填写到相应位置，确保报文在传输过程中的完整性。
    hdr->checksum16 = 0;
    hdr->checksum16 = checksum16((uint16_t *)txbuf.data, txbuf.len);
    // Step3: 发送数据报
    // 调用 ip_out() 函数将封装好且填写了校验和的 ICMP 数据报发送出去。
    ip_out(&txbuf, src_ip, NET_PROTOCOL_ICMP);
}
```

5.3 ICMP 差错报文发送函数（即 icmp_unreachable 函数）的设计与实现

void icmp_unreachable(buf_t *recv_buf, uint8_t *src_ip, icmp_code_t code)

本实验仅要求输出 icmp unreachable 的差错报文，且仅支持如下差错代码：

```
typedef enum icmp_code
{
    ICMP_CODE_PROTOCOL_UNREACH = 2, // 协议不可达
    ICMP_CODE_PORT_UNREACH = 3      // 端口不可达
} icmp_code_t;
```

设计流程如下：

1. 初始化并填写报头：首先调用 buf_init() 函数来初始化 txbuf。然后，按照 ICMP 协议规范填写 ICMP 报头首部，包含类型、代码等关键信息。

2. 填写数据与校验和：接着，填写 ICMP 数据部分，该部分要包含 IP 数据报首部和 IP 数据报的前 8 个字节的数据字段。完成数据填写后，使用与 IP 协议相同的校验和算法计算校验和，并将其填写到相应位置。

3. 发送数据报：调用 ip_out() 函数将封装好且填写了校验和的 ICMP 差错数据报发送出去。

代码实现如下：

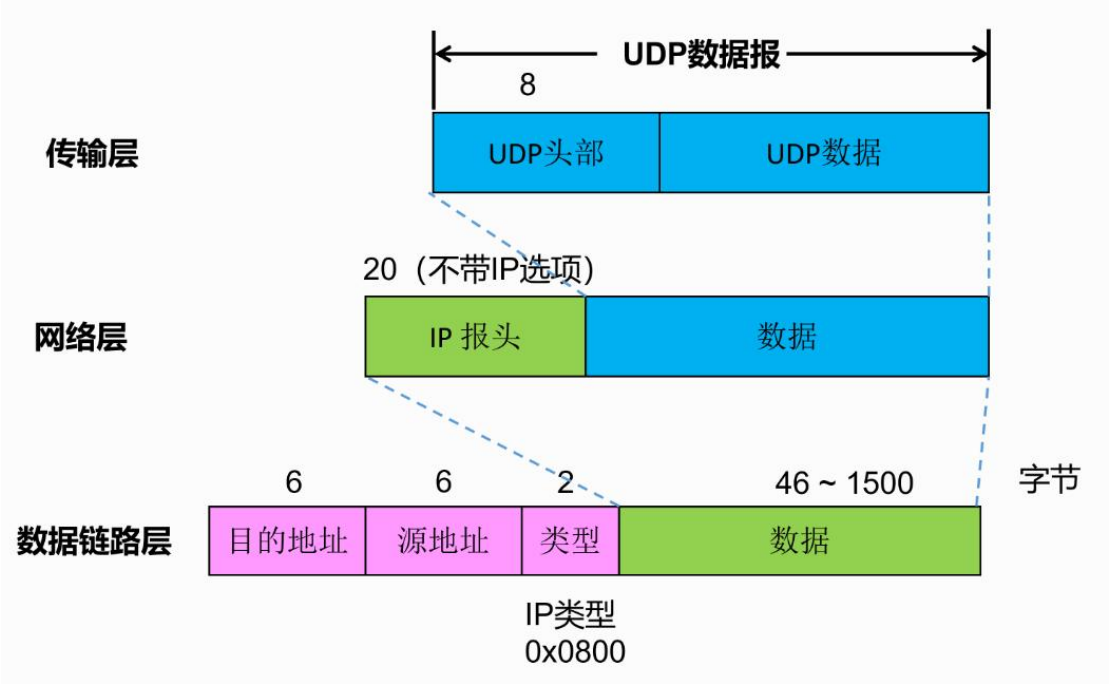
```
/**
 * @brief 发送icmp不可达
 *
 * @param recv_buf 收到的ip数据包
 * @param src_ip 源ip地址
 * @param code icmp code，协议不可达或端口不可达
 */
解释代码 | 注释代码 | 生成单测 | ×
void icmp_unreachable(buf_t *recv_buf, uint8_t *src_ip, icmp_code_t code) {
    // TO-DO
    // Step1 初始化并填写报头：
    // 首先调用 buf_init() 函数来初始化 txbuf。
    // 报头 + IP头 + 前8字节
    buf_init(&txbuf, sizeof(icmp_hdr_t) + sizeof(ip_hdr_t) + 8);
    // 然后，按照 ICMP 协议规范填写 ICMP 报头首部，包含类型、代码等关键信息。
    icmp_hdr_t *icmp_hdr = (icmp_hdr_t *)txbuf.data;
    icmp_hdr->type = ICMP_TYPE_UNREACH;
    icmp_hdr->code = code;
    icmp_hdr->id16 = 0;
    icmp_hdr->seq16 = 0;

    // Step2 填写数据与校验和：
    // 该部分要包含 IP 数据报首部
    ip_hdr_t *ip_hdr = (ip_hdr_t *)(icmp_hdr + 1);
    ip_hdr_t *recv_ip_hdr = (ip_hdr_t *)recv_buf->data;
    memcpy(ip_hdr, recv_ip_hdr, sizeof(ip_hdr_t));
    //IP 数据报的前 8 个字节的数据字段
    uint8_t *data = (uint8_t *)(ip_hdr + 1);
    uint8_t *recv_ip_data = (uint8_t *)(recv_ip_hdr + 1);
    memcpy(data, recv_ip_data, 8);
    // 使用与 IP 协议相同的校验和算法计算校验和，并将其填写到相应位置。
    icmp_hdr->checksum16 = 0;
    icmp_hdr->checksum16 = checksum16((uint16_t *)txbuf.data, sizeof(icmp_hdr_t) + sizeof(ip_hdr_t) + 8);

    // Step3 发送数据报：
    // 调用 ip_out() 函数将封装好且填写了校验和的 ICMP 差错数据报发送出去。
    ip_out(&txbuf, src_ip, NET_PROTOCOL_ICMP);
}
```

UDP 协议详细设计

（描述 UDP 数据包的封装与解封装、UDP 校验和计算等。）

【总体设计思路】

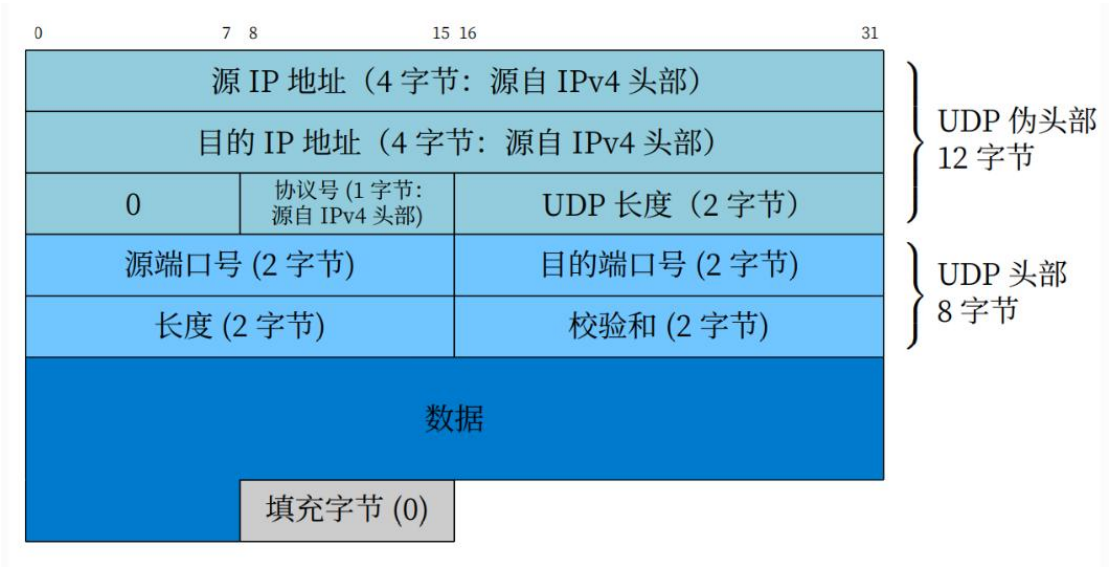
UDP 协议是 TCP/IP 协议族中典型的传输层协议之一。它是一种不提供可靠性保障的数据报协议，适合应用在对传输速率和实时性要求较高的场景中，例如实时通信或广播式数据传输。UDP 报文的数据结构以及其头部格式通常如下图所示。



因此，在处理 UDP 协议时，无论是解析还是构造 UDP 报文头部，都需要关注以下关键字段：源端口号、目的端口号、数据长度以及校验和。

另一方面，UDP 的校验和计算方式与 IP 协议的不同，不能直接套用 IP 校验方法。因为 UDP 校验和除了覆盖 UDP 的头部和数据部分外，还需要引入一个额外的“伪首部”（Pseudo Header）。这个伪首部是临时添加在 UDP 报文前的 12 字节信息，仅用于校验和的计算过程，在实际发送过程中不会被包括在 UDP 数据报中，也不会计入 UDP 长度字段。

伪首部的具体结构如下图所示：



之所以在计算 UDP 校验和时还需引入伪头部，是因为仅靠目标端口并不足以准确标识一次通信过程。网络中通常采用五元组（源 IP、目标 IP、协议号、源端口和目标端口）来唯一标识一条连接路径。若其中任意字段在传输过程中出现损坏，就有可能导致数据包被错误地分发：目标应用程序可能接收不到应有的数据，而其他无关应用则可能错误地接收到它。为增强数据传输的可靠性，UDP 校验和计算时引入了伪头部，以协助识别通信双方和协议类型。

因此，本小节实验的主要工作包括：在接收数据包时，正确解析 UDP 头部，提取关键信息并去除该头部；在发送数据时，填充必要的 UDP 字段，构造完整数据报后完成发送。此外，还需实现用于生成 UDP 校验和的伪头部构造与计算函数。

6.1 UDP 数据报输出处理函数（即 udp_out 函数）的设计与实现

void udp_out(buf_t *buf, uint16_t src_port, uint8_t *dst_ip, uint16_t dst_port)
设计流程如下：
1. 添加 UDP 报头：调用 buf_add_header() 函数，为待发送的数据添加 UDP 报头，为后续填充首部字段做准备。
2. 填充 UDP 首部字段：按照 UDP 协议规范，依次填充 UDP 首部的各个字段，如源端口号、目的端口号、长度等。
3. 计算并填充校验和：先将校验和字段填充为 0，然后调用 transport_checksum() 函数计算 UDP 数据报的校验和。计算完成后，将得到的校验和结果填入校验和字段。
4. 发送 UDP 数据报：调用 ip_out() 函数，将封装好的 UDP 数据报发送出去。

代码实现如下：

```
64  /**
65   * @brief 处理一个要发送的数据包
66   *
67   * @param buf 要处理的包
68   * @param src_port 源端口号
69   * @param dst_ip 目的ip地址
70   * @param dst_port 目的端口号
71   */
     解释代码 | 注释代码 | 生成单测 | ×
72  void udp_out(buf_t *buf, uint16_t src_port, uint8_t *dst_ip, uint16_t dst_port) {
73      // TO-DO
74      // Step1: 添加 UDP 报头：
75      // 调用 buf_add_header() 函数，为待发送的数据添加 UDP 报头
76      buf_add_header(buf, sizeof(udp_hdr_t));
77      // Step2 填充 UDP 首部字段：
78      // 依次填充 UDP 首部的各个字段，如源端口号、目的端口号、长度等。
79      udp_hdr_t *hdr = (udp_hdr_t *) buf->data;
80      hdr->src_port16 = swap16(src_port);
81      hdr->dst_port16 = swap16(dst_port);
82      hdr->total_len16 = swap16(buf->len);
83      // Step3 计算并填充校验和：
84      // 先将校验和字段填充为 0
85      hdr->checksum16 = 0;
86      // 然后调用 transport_checksum() 函数计算 UDP 数据报的校验和
87      // 计算完成后，将得到的校验和结果填入校验和字段。
88      hdr->checksum16 = transport_checksum(
89          NET_PROTOCOL_UDP,
90          buf,
91          net_if_ip,
92          dst_ip
93      );
94      // Step4 发送 UDP 数据报：
95      // 调用 ip_out() 函数，将封装好的 UDP 数据报发送出去。
96      ip_out(buf, dst_ip, NET_PROTOCOL_UDP);
97  }
```

6.2 UDP 数据报输入处理函数（即 udp_in 函数）的设计与实现

void udp_in(buf_t *buf, uint8_t *src_ip)

设计流程如下：

1. 包检查：对接收到的数据报进行检查，判断数据报的长度是否小于 UDP 首部长度，或者接收到的包长度是否小于 UDP 首部长度字段给出的长度。如果满足上述条件之一，则说明数据报不完整或存在异常，将其丢弃，不做进一步处理。
2. 重新计算校验和：先把首部的校验和字段保存起来，然后将该字段填充为 0。接着调用 transport_checksum() 函数重新计算校验和。将计算得到的校验和值与接收到的 UDP 数据报的校验和进行比较，如果两者不一致，则说明数据报在传输过程中可能发生了错误，将其丢弃。
3. 查询处理函数：调用 map_get() 函数，在 udp_table 中查询是否有该目的端口号对应的处理函数（回调函数）。
4. 处理未找到处理函数的情况：如果在 udp_table 中没有找到对应的处理函数，则调用 buf_add_header() 函数增加 IPv4 数据报头部，再调用 icmp_unreachable() 函数发送一个端口不可达的 ICMP 差错报文，通知发送方该端口不可用。
5. 调用处理函数：如果在 udp_table 中能找到对应的处理函数，则去掉 UDP 报头，调用该处理函数对数据进行相应的处理。

代码实现如下：

```
12  /**
13   * @brief 处理一个收到的udp数据包
14   *
15   * @param buf 要处理的包
16   * @param src_ip 源ip地址
17   */
    解释代码 | 注释代码 | 生成单测 | ×
18  void udp_in(buf_t *buf, uint8_t *src_ip) {
19      // TO-DO
20      // Step1 包检查
21      // 判断数据报的长度是否小于 UDP 首部长度
22      if (buf->len < sizeof(udp_hdr_t)) {
23          return;
24      }
25      // 或者接收到的包长度是否小于 UDP 首部长度字段给出的长度
26      udp_hdr_t *hdr = (udp_hdr_t *)buf->data;
27
28      // 报文长度字段是网络字节序，需要转换
29      uint16_t total_len = swap16(hdr->total_len16);
30      if (buf->len < total_len) {
31          return;
32      }
33      // Step2 重新计算校验和
34      // 先把首部的校验和字段保存起来
35      uint16_t checksum_backup = hdr->checksum16;
36      // 然后将该字段填充为 0，接着调用 transport_checksum() 函数重新计算校验和
37      hdr->checksum16 = 0;
38      uint16_t checksum = transport_checksum(NET_PROTOCOL_UDP, buf, src_ip, net_if_ip);
39      // 将计算得到的校验和值与接收到的 UDP 数据报的校验和进行比较
40      if (checksum != checksum_backup) {
41          // 如果两者不一致，则说明数据报在传输过程中可能发生了错误，将其丢弃。
42          return;
43      }
44      hdr->checksum16 = checksum;
```

```
45      // Step3 查询处理函数：
46      // 调用 map_get() 函数，在 udp_table 中查询是否有该目的端口号对应的处理函数（回调函数）。
47      uint16_t dst_port = swap16(hdr->dst_port16);
48      udp_handler_t *handler = map_get(&udp_table, &dst_port);
49      // Step4 处理未找到处理函数的情况：
50      // 如果在 udp_table 中没有找到对应的处理函数
51      if(handler == NULL) {
52          // 调用 buf_add_header() 函数增加 IPv4 数据报头部
53          // 再调用 icmp_unreachable() 函数发送一个端口不可达的 ICMP 差错报文
54          // 通知发送方该端口不可用
55          buf_add_header(buf, sizeof(ip_hdr_t));
56          icmp_unreachable(buf, src_ip, ICMP_CODE_PORT_UNREACH);
57      } else {
58          // Step5 调用处理函数：如果在 udp_table 中能找到对应的处理函数
59          // 去掉 UDP 报头，调用该处理函数对数据进行相应的处理
60          buf_remove_header(buf, sizeof(udp_hdr_t));
61          (* handler)(buf->data, buf->len, src_ip, swap16(hdr->src_port16));
62      }
63  }
```

6.3 UDP 校验和计算函数（即 transport_checksum 函数）的设计与实现

uint16_t transport_checksum(uint8_t protocol, buf_t *buf, uint8_t *src_ip, uint8_t *dst_ip)
设计流程如下：

1. 增加 UDP 伪头部：调用 buf_add_header() 函数，为 UDP 数据报增加 UDP 伪头部，以便进行校验和计算。
2. 暂存 IP 头部：将被 UDP 伪头部覆盖的 IP 头部拷贝出来，暂存起来，避免其被 UDP 伪头部覆盖。
3. 填写 UDP 伪头部字段：按照 UDP 伪头部的格式要求，填写 12 字节的字段，包括源 IP 地址、目的 IP 地址、协议号和 UDP 长度等。
4. 计算 UDP 校验和：对包含 UDP 伪头部、UDP 头部和 UDP 数据的数据块进行校验和计算。
5. 恢复 IP 头部：再将 Step2 中暂存的 IP 头部拷贝回来，确保 IP 头部信息的完整性。
6. 去掉 UDP 伪头部：调用 buf_remove_header() 函数，去掉之前添加的 UDP 伪头部，因为伪头部不会随 UDP 数据报一起传输。
7. 返回校验和值：返回计算得到的 UDP 校验和值。

代码实现如下：

```
/**
 * @brief 计算传输层协议（如TCP/UDP）的校验和
 *
 * @param protocol  传输层协议号（如NET_PROTOCOL_UDP、NET_PROTOCOL_TCP）
 * @param buf       待计算的数据包缓冲区
 * @param src_ip    源IP地址
 * @param dst_ip    目的IP地址
 * @return uint16_t 计算得到的16位校验和
 */
解释代码 | 注释代码 | 生成单测 | ×
uint16_t transport_checksum(uint8_t protocol, buf_t *buf, uint8_t *src_ip, uint8_t *dst_ip) {
    // TO-DO
    uint16_t checksum = 0;
    if(protocol == NET_PROTOCOL_UDP) {
        // Step1 增加 UDP 伪头部：
        // 调用 buf_add_header() 函数，为 UDP 数据报增加 UDP 伪头部
        uint16_t len_backup = swap16(buf->len);
        buf_add_header(buf, sizeof(peso_hdr_t));

        // Step2 暂存 IP 头部：
        // 将被 UDP 伪头部覆盖的 IP 头部拷贝出来，暂存起来，避免其被 UDP 伪头部覆盖。
        peso_hdr_t pre;
        memcpy(&pre, buf->data, sizeof(peso_hdr_t));

        // Step3 填写 UDP 伪头部字段：
        // 按照 UDP 伪头部的格式要求，填写 12 字节的字段
        // 包括源 IP 地址、目的 IP 地址、协议号和 UDP 长度等。
        peso_hdr_t* peso_hdr = (peso_hdr_t*)(buf->data);
        memcpy(peso_hdr->src_ip, src_ip, NET_IP_LEN);
        memcpy(peso_hdr->dst_ip, dst_ip, NET_IP_LEN);

        peso_hdr->placeholder = 0;
        peso_hdr->protocol = NET_PROTOCOL_UDP;
        peso_hdr->total_len16 = len_backup;
        // Step4 计算 UDP 校验和：
        // 对包含 UDP 伪头部、UDP 头部和 UDP 数据的数据块进行校验和计算。
        checksum = checksum16((uint16_t*)buf->data, buf->len);
        // Step5 恢复 IP 头部：
        // 再将 Step2 中暂存的 IP 头部拷贝回来，确保 IP 头部信息的完整性。
        memcpy(buf->data, &pre, sizeof(peso_hdr_t));
        // Step6 去掉 UDP 伪头部：
        // 调用 buf_remove_header() 函数，去掉之前添加的 UDP 伪头部
        // 因为伪头部不会随 UDP 数据报一起传输。
        buf_remove_header(buf, sizeof(peso_hdr_t));
        // Step7 返回校验和值：
        // 返回计算得到的 UDP 校验和值
```

5. TCP 协议详细设计

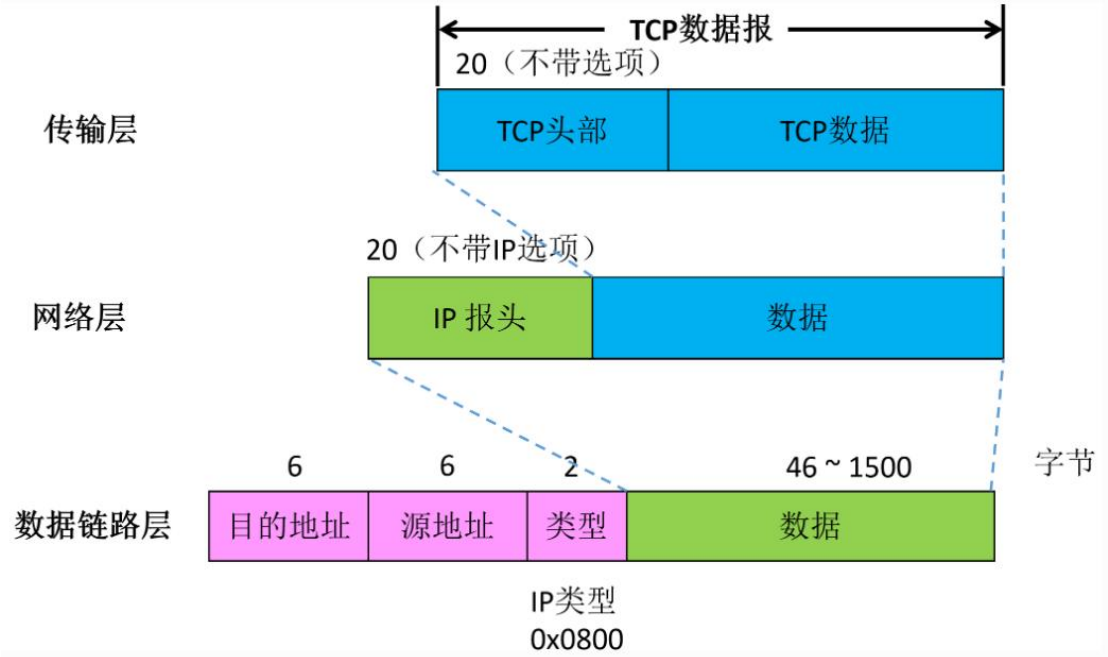
（描述 TCP 连接的建立与关闭过程（三次握手、四次挥手）等。解释如何处理 TCP 数据包的确认、连接状态等问题。）

【总体设计思路】

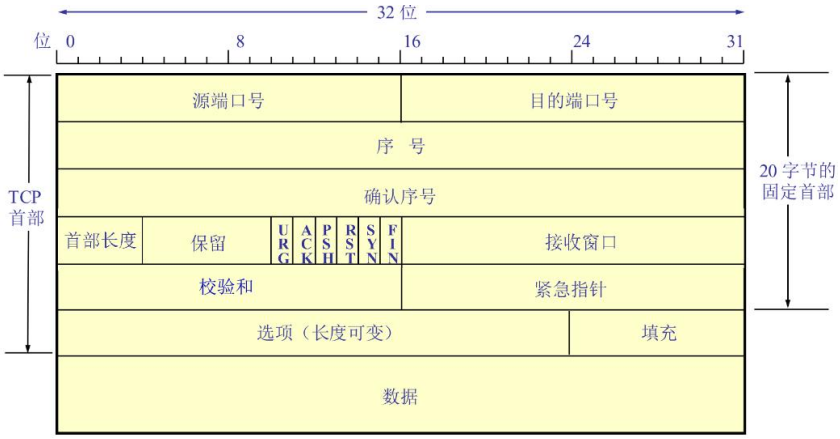
TCP 是一种面向连接、可靠的传输层协议，具备序列号、确认应答、超时重传、流量控制和拥塞控制等机制，确保数据有序且完整地传输。相比不可靠的 UDP，TCP 更适用于对可靠性要求高的应用，如文件传输和网页浏览。

本次实验中，我们聚焦于简化版 TCP 的服务端逻辑，重点实现连接建立与确认应答机制，确保基本的可靠数据传输功能。

【TCP 头部】



TCP 的头部结构长度通常为 20 字节，若包含可变长的选项部分，最长可达 60 字节。



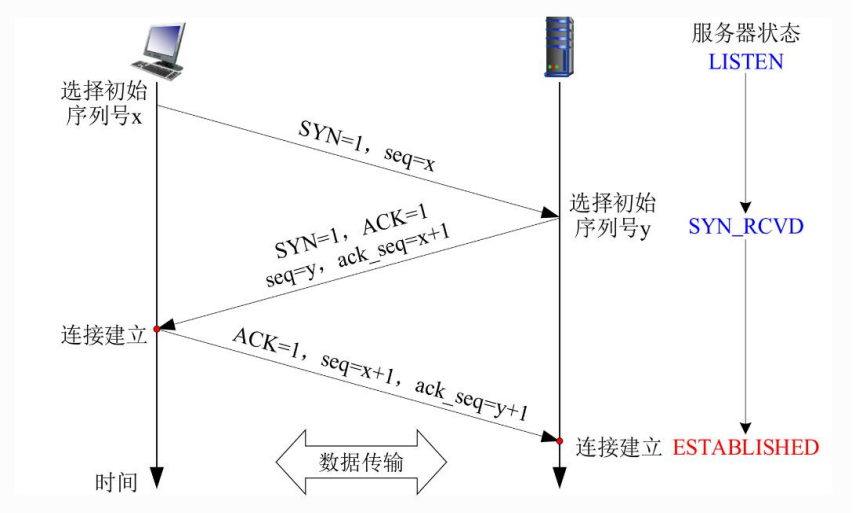
TCP 报文字段功能概览：

- 源端口号 & 目的端口号（16 位）：标识通信双方的应用程序，用于数据在多个进程间正确传递。
- 序号（32 位）：用于数据排序与丢包重传，表示该报文段中第一个字节的位置。
- 确认序号（32 位）：告知发送方下一个期望接收的字节序号，实现可靠传输。
- 首部长度（4 位）：指示 TCP 首部占用的长度，单位为 4 字节。
- 保留位（3 位）：保留供将来使用，当前设置为 0。
- 标志位（9 位）：控制连接状态与数据传输，重点包括：
  - ACK：确认应答；
  - SYN：建立连接；
  - FIN：释放连接；
  - RST：复位连接。
- 窗口大小（16 位）：表示接收方缓冲区可用大小，用于流量控制。
- 校验和（16 位）：进行差错检测，确保数据完整性。
- 紧急指针（16 位）：指向紧急数据的末尾（本实验中忽略）。
- 数据（可变）：承载实际传输的应用层内容。

【TCP 建立连接：三次“握手”】

TCP 三次握手建立连接的过程如下：

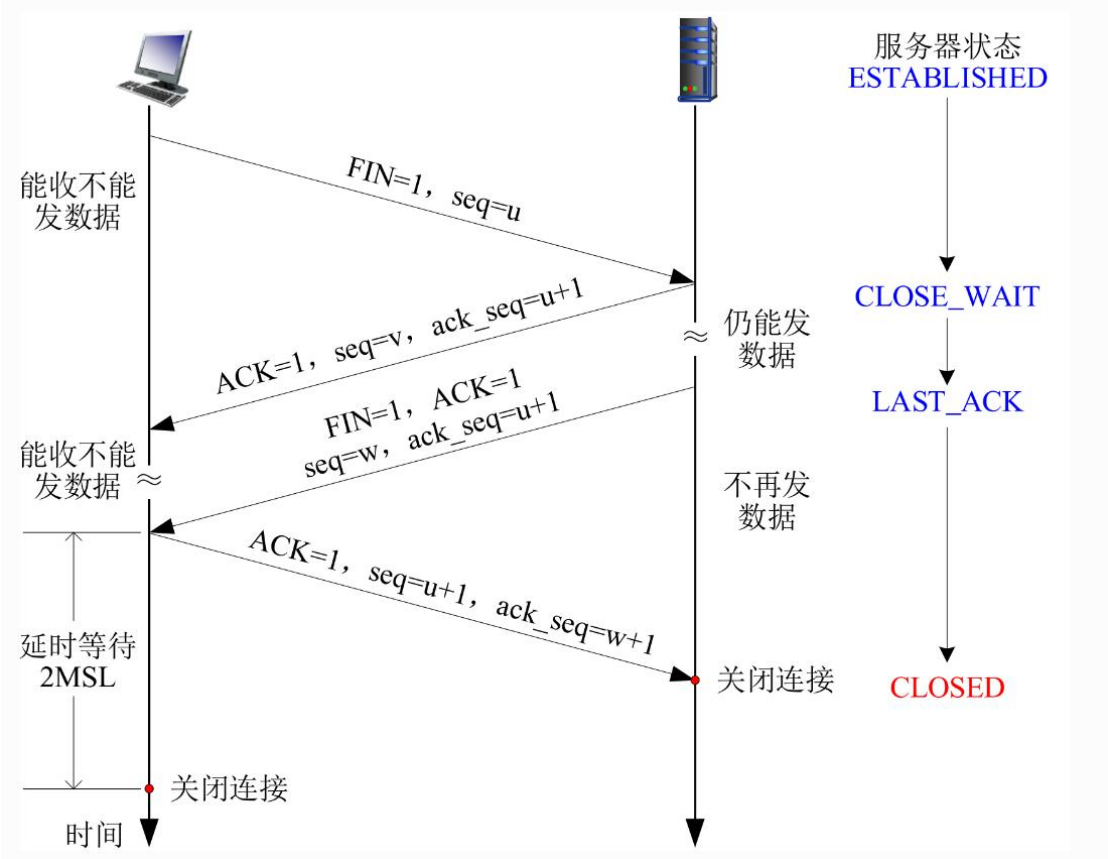
1. 服务端处于监听状态，等待来自客户端的连接请求。
2. 客户端发起连接请求：设置初始序列号为 x，并发送一个 SYN 报文段，表示希望建立连接。
3. 服务端响应请求：收到 SYN 后，进入 SYN_RCVD 状态，生成自身的初始序列号 y，并发送一个同时带有 SYN 和 ACK 标志的报文段，确认号为 x+1，表示已接收到客户端的请求。
4. 客户端确认连接：接收到 SYN+ACK 报文后，客户端进入 ESTABLISHED 状态，发送一个 ACK 报文段，序列号为 x+1，确认号为 y+1，表示连接确认成功，该报文可能携带实际数据。
5. 服务器接收到 ACK 报文 后，状态也转为 ESTABLISHED，至此连接正式建立，双方可以开始数据通信。



【TCP 断开连接：四次“挥手”】

TCP 四次挥手断开连接的过程如下：

1. 服务器处于 ESTABLISHED 状态，并等待客户端请求关闭连接。
2. 客户端发送 FIN 报文段，表示它希望关闭连接并释放资源。
3. 如果 服务器没有数据要发送，则直接进入关闭阶段；否则，服务器首先回复一个 ACK 报文段，确认关闭请求，并进入 CLOSE_WAIT 状态。
4. 服务器发送 FIN｜ACK 报文段 给客户端，表明它准备关闭连接，并进入 LAST_ACK 状态。
5. 客户端收到 FIN 报文后，回复一个 ACK 报文段，确认服务器的关闭请求。客户端随后进入等待状态，通常会等待 2 倍的最大报文生存时间（MSL），确保所有报文都被正确接收后，才能正式关闭连接。
6. 服务器收到客户端的 ACK 后，连接被完全关闭。



【TCP 连接对象】

解释如何处理 TCP 数据包的确认、连接状态等问题。）

实验框架可以串行地同时支持多个 TCP 连接。

TCP 连接的上下文信息通过 tcp_connection 结构体表示，定义于 tcp.h 文件：

```c
typedef struct tcp_connection {
    /* TCP connection states */
    tcp_state_t state;
    uint8_t not_send_empty_ack;

    /* TCP communication states */
    int port;
    uint32_t seq;  // 要发送的序列号
    uint32_t ack;  // 要发送的 ACK
} tcp_conn_t;
```

static map_t tcp_conn_table; // [src_ip, src_port, dst_port] -> tcp_conn

TCP 连接表 tcp_conn_table 是一个哈希表（map_t 类型），用于统一管理当前所有活跃的 TCP 连接，定义在 tcp.c 文件中。

它的键是一个三元组结构 tcp_key_t，由远程 IP 地址（remote_ip）、远程端口号（remote_port）以及本地端口号（host_port）组成，用于唯一标识一条 TCP 连接。与之对应的值则是该连接的状态对象 tcp_conn_t，用于保存连接的相关信息和上下文。

当系统接收到一个 TCP 报文时，可以通过调用 tcp_get_connection() 函数在连接表中查找对应的连接，若不存在则会创建新的连接项；而在连接结束时，系统则调用 tcp_close_connection() 函数，从连接表中移除指定连接，实现资源释放与管理。

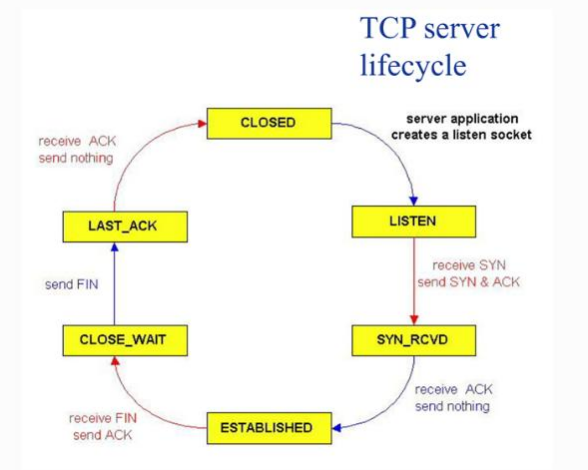
【TCP 连接状态】

本实验实现了一个简化版的服务端 TCP 协议（runnable），涵盖以下核心功能：

- 连接管理：通过状态机控制 TCP 连接的生命周期，支持“三次握手”建立连接与“四次挥手”释放连接。
- 基础可靠传输：实现了序列号与确认号机制，具备初步的乱序包处理能力，确保上层应用接收到的数据是有序的。

不过，该协议也做出了一些简化，存在如下限制：

- 缺乏流量与拥塞控制：窗口大小为固定值，未实现慢启动、拥塞避免等机制。
- 可靠性有限：未支持超时重传，乱序包处理也较为基础，因此仅适用于网络状况较好的场景。
- 仅支持服务端逻辑：没有实现客户端功能，无法主动向其他服务器发起连接。



7.1 TCP 数据报输出处理函数（即 tcp_out 函数）的设计与实现

void tcp_out(tcp_conn_t *tcp_conn, buf_t *buf, uint16_t src_port, uint8_t *dst_ip, uint16_t dst_port, uint8_t flags)

设计流程如下：

1. 添加 TCP 报头：首先调用 buf_add_header()函数添加 TCP 报头。
2. 填充 TCP 首部字段：将 win 字段设置为最大 TCP 窗口大小（TCP_MAX_WINDOW_SIZE），uptr 字段置为零。操作时需注意字节序问题。
3. 计算并填充校验和：先把校验和字段填充为 0，接着调用 transport_checksum() 函数计算校验和，最后将计算得到的校验和结果填入校验和字段。
4. 发送 TCP 数据报：调用 ip_out() 函数，将处理好的 TCP 数据报发送出去。

代码实现如下：

```
/**
 * @brief 填写 TCP 报文头并发送
 *
 * @param tcp_conn  指向当前 TCP 连接的指针，用于获取和更新序列号、确认号、窗口大小等状态信息
 * @param buf       数据缓冲区，payload 为要发送的数据
 * @param src_port  源端口号
 * @param dst_ip    目标IP地址
 * @param dst_port  目标端口号
 * @param flags     TCP 标志位
 */
解释代码 | 注释代码 | 生成单测 | ×
void tcp_out(tcp_conn_t *tcp_conn, buf_t *buf, uint16_t src_port, uint8_t *dst_ip, uint16_t dst_port, uint8_t flags) {
    /* ============================== TODO 1 BEGIN ============================== */
    // Step1 添加 TCP 报头
    // 首先调用buf_add_header()函数添加TCP报头
    buf_add_header(buf, sizeof(tcp_hdr_t));
    // Step2 填充 TCP 首部字段：
    // 将 win 字段设置为最大 TCP 窗口大小（TCP_MAX_WINDOW_SIZE）
    // uptr 字段置为零。操作时需注意字节序问题。
    tcp_hdr_t *hdr = (tcp_hdr_t *) buf->data;
    hdr->src_port16 = swap16(src_port);
    hdr->dst_port16 = swap16(dst_port);
    hdr->seq = swap32(tcp_conn->seq);
    hdr->ack = swap32(tcp_conn->ack);
    hdr->doff = (sizeof(tcp_hdr_t) / 4) << 4;
    hdr->flags = flags;
    hdr->win = TCP_MAX_WINDOW_SIZE;
    hdr->uptr = 0;
    // Step3 计算并填充校验和：
    // 先把校验和字段填充为 0，接着调用 transport_checksum() 函数计算校验和
    // 最后将计算得到的校验和结果填入校验和字段
    hdr->checksum16 = 0;
    hdr->checksum16 = transport_checksum(
        NET_PROTOCOL_TCP,
        buf,
        net_if_ip,
        dst_ip
    );
    // Step4 发送 TCP 数据报：
    // 调用 ip_out() 函数，将处理好的 TCP 数据报发送出去
    ip_out(buf, dst_ip, NET_PROTOCOL_TCP);
    /* ============================== TODO 1 END ============================== */
}
```

7.2 TCP 数据报输入处理函数（即 tcp_in 函数）的设计与实现

void tcp_in(buf_t *buf, uint8_t *src_ip)

设计流程如下：

1. 更新连接状态与回复报文标志：依据接收包的数据，更新当前 TCP 连接的内部状态，并填写回复报文的标志部分。

2. 数据交付与错误处理：如果接收报文携带数据，则将数据部分交付给上层应用。调用 map_get()函数查询 tcp_handler_table 是否有该目的端口号对应的处理函数（回调函数）。如果没有找到，则调用 buf_add_header()函数增加 IPv4 数据报头部，再调用 icmp_unreachable()函数发送一个端口不可达的 ICMP 差错报文。如果能找到，则去掉 TCP 报头，调用处理函数来做相应处理。

3. 发送回复报文并更新序列号：调用 tcp_out() 函数发送回复报文，同时更新 TCP 连接的序列号。详细步骤可参考代码注释。

代码实现如下：（代码本身具有注释，可以解释大部分意思）

```
186     /* ================================ TODO 2 BEGIN ================================ */
187     /* Step1 : 根据接收包数据更新当前TCP连接内部状态，并填写回复报文的标志部分。 */
188
189     uint8_t send_flags = 0;  // 回复报文的标志位字段
190
191      // 根据当前 TCP 连接的状态进行不同的处理
192     switch (tcp_conn->state) {
193         case TCP_STATE_LISTEN:
194             // TODO: 仅在收到连接报文时（SYN报文）才做出处理，否则直接返回
195             if(!TCP_FLG_ISSET(recv_flags, TCP_FLG_SYN)) {
196                 return;
197             }
198             // TODO: 初始化 TCP 连接上下文（tcp_conn结构体）的seq字段
199             tcp_conn->seq = tcp_generate_initial_seq();
200
201             // TODO: 填写 TCP 连接上下文（tcp_conn结构体）的ack字段
202             tcp_conn->ack = remote_seq + 1;
203
204             // TODO: 填写回复标志 send_flags
205             send_flags = TCP_FLG_SYN | TCP_FLG_ACK;
206
207             // TODO: 进行状态转移
208             tcp_conn->state = TCP_STATE_SYN_RECEIVED;
209
210             break;
211
212         case TCP_STATE_SYN_RECEIVED:
213             // TODO: 仅在收到确认报文时（ACK报文）才做出处理，否则直接返回
214             if(!TCP_FLG_ISSET(recv_flags, TCP_FLG_ACK)) {
215                 return;
216             }
217
218             // TODO: 进行状态转移
219             tcp_conn->state = TCP_STATE_ESTABLISHED;
220
221             break;
```

```
223         case TCP_STATE_ESTABLISHED:
224             // 未收到顺序包，丢弃并发送重复 ACK
225             if (remote_seq != tcp_conn->ack) {
226                 buf_init(&txbuf, 0);
227                 tcp_out(tcp_conn, &txbuf, host_port, remote_ip, remote_port, TCP_FLG_ACK);
228                 return;
229             }
230             // TODO: 计算接收到的数据长度，更新 ACK
231             uint32_t data_len = buf->len - tcp_hdr_sz;
232 
233             // TODO: 如果接收报文携带数据，则填写回复标志 send_flags 发送ACK
234             tcp_conn->ack += bytes_in_flight(data_len, recv_flags);
235 
236             // TODO: 如果收到 FIN 报文，则增加 send_flags 相应标志位，并且进行状态转移
237             if(data_len > 0 || TCP_FLG_ISSET(recv_flags, TCP_FLG_FIN)) {
238                 send_flags |= TCP_FLG_ACK;
239             }
240 
241             if (TCP_FLG_ISSET(recv_flags, TCP_FLG_FIN)) {
242                 tcp_conn->state = TCP_STATE_LAST_ACK;
243                 send_flags |= TCP_FLG_FIN;
244             }
245             break;
246 
247         case TCP_STATE_LAST_ACK:
248             // TODO: 仅在收到确认报文时（ACK报文）才做出处理，否则直接返回
249             if (!TCP_FLG_ISSET(recv_flags, TCP_FLG_ACK)) {
250                 return;
251             }
252             // TODO: 关闭 TCP 连接
253             tcp_close_connection(remote_ip, remote_port, host_port);
254 
255             break;
256 
257         default:
258             printf("do not support state %d\n", tcp_conn->state);
259             break;
260     }
```

```
262     /* Step2 ：如果接收报文携带数据，则将数据部分交付给上层应用 */
263     // TODO
264     if (tcp_conn->state == TCP_STATE_ESTABLISHED && buf->len > tcp_hdr_sz) {
265         // 将 TCP payload 数据交付给应用层（使用 handler
266         // 调用map_get()函数查询tcp_handler_table是否有该目的端口号对应的处理函数（回调函数）
267         tcp_handler_t *handler = map_get(&tcp_handler_table, &host_port);
268         // 能找到回调函数
269         if (handler) {
270             size_t data_len = buf->len - tcp_hdr_sz;
271             uint8_t *data_ptr = buf->data + tcp_hdr_sz;
272             // 去掉 TCP 头部
273             buf_remove_header(buf, tcp_hdr_sz);
274 
275             // 调用处理函数来做相应处理
276             (*handler)(tcp_conn, data_ptr, data_len, remote_ip, remote_port);
277         } else {    // 如果没有找到
278             // 调用buf_add_header()函数增加IPv4数据报头部
279             buf_add_header(buf, TCP_HEADER_LEN);
280             // 调用icmp_unreachable()函数发送一个端口不可达的ICMP差错报文
281             icmp_unreachable(buf, remote_ip, ICMP_CODE_PORT_UNREACH);
282         }
283     }
```

```
285     /* Step3 ： 调用tcp_out()发送回复报文，更新TCP连接序列号。 */
286     // 如果无需回复，则接收逻辑结束
287     if (send_flags == 0)
288         return;
289     // 如果 send_flags 只标识了 ACK 字段，并且应用程序已通过 tcp_send() 发送顺带 ACK，则无需再进行回复
290     if (bytes_in_flight(0, send_flags) == 0 && tcp_conn->not_send_empty_ack) {
291         assert(TCP_FLG_ISSET(send_flags, TCP_FLG_ACK));
292         tcp_conn->not_send_empty_ack = 0;
293         return;
294     }
295 
296     // TODO:  初始化一个新的缓冲区，发送回复报文
297     buf_t tx_buf;
298     buf_init(&tx_buf, 0);
299     tcp_out(tcp_conn, &tx_buf, host_port, remote_ip, remote_port, send_flags);
300     // TODO: 更新序列号
301     tcp_conn->seq += bytes_in_flight(0, send_flags);
302     /* ================================ TODO 2 END ================================ */
303 }
```

7.3 TCP 校验和计算函数（即 transport_checksum 函数）的设计与实现

uint16_t transport_checksum(uint8_t protocol, buf_t *buf, uint8_t *src_ip, uint8_t *dst_ip)
设计流程如下：
1. 获取 TCP 段长度：首先记录当前 TCP 段（包括 TCP 头部和数据部分）的总长度，为后续构造伪头部和计算校验和做准备
2. 定位 TCP 伪头部位置：通过向 buf->data 的起始地址前移 12 字节（伪头部长度），确定伪头部插入的位置。
3. 暂存被覆盖数据（IP 头部等）：将被伪头部覆盖的原始内容（可能是 IP 头部）暂存到 pre 中，以便后续恢复，防止原始数据丢失。
4. 填写 TCP 伪头部字段：按照 TCP 校验和伪头部的格式规范，填写源 IP、目标 IP、协议号（TCP）、TCP 段长度等字段。其中长度字段需要大小端转换。
5. 计算 TCP 校验和：对从伪头部开始的整个数据块（伪头部 + TCP 头部 + TCP 数据）进行校验和计算。注意，这里使用的是标准的 16 位求和函数 checksum16。
6. 恢复被覆盖的原始数据：将之前暂存的原始内容重新拷贝回来，恢复 buf 中伪头部之前的数据，确保 IP 层数据不受影响。

代码实现如下：

```
154         } else if(protocol == NET_PROTOCOL_TCP) {
155             uint16_t len = (uint16_t)buf->len;
156 
157             // 获取 TCP 校验和所需的伪头部位置
158             peso_hdr_t* peso_hdr = (peso_hdr_t*)(buf->data - sizeof(peso_hdr_t));
159 
160             peso_hdr_t pre; //暂存被覆盖的IP头
161 
162             // 备份原始伪头部内容，防止后续计算修改原数据
163             memcpy(&pre, peso_hdr, sizeof(peso_hdr_t));
164 
165             // 填写伪头部的源 IP 和目的 IP 地址
166             memcpy(peso_hdr->src_ip, src_ip, NET_IP_LEN);
167             memcpy(peso_hdr->dst_ip, dst_ip, NET_IP_LEN);
168 
169             // 伪头部中的保留字段设为 0
170             peso_hdr->placeholder = 0;
171 
172             // 指定协议类型为 TCP
173             peso_hdr->protocol = NET_PROTOCOL_TCP;
174 
175             // 设置 TCP 段长度（需要大小端转换）
176             peso_hdr->total_len16 = swap16(len);
177 
178             // 计算 TCP 校验和，包含 TCP 数据 + 伪头部
179             checksum = checksum16((uint16_t*)peso_hdr, len + sizeof(peso_hdr_t));
180 
181             // 将伪头部恢复为原内容
182             memcpy(peso_hdr, &pre, sizeof(peso_hdr_t));
183         }
184         return checksum;
185     }
```

6. web 服务器详细设计

（描述 web 服务器的请求处理流程和响应等。）

【总体设计思路】

本次实验要求我们基于已有的简化版 TCP 协议，构建一个简单的 Web 服务器。该服务器应能接收并解析基本的 HTTP/1.1 请求，并返回对应的网页资源，从而实现网页的访问与展示功能。核心目标是搭建 TCP 之上的 HTTP 服务能力，完成从底层通信到应用层响应的整体流程。

【HTTP 协议概要】

HTTP（超文本传输协议）是 Web 应用层的通信协议，采用客户端-服务器架构，基于 TCP 实现可靠的数据传输。客户端发送请求，服务器返回网页、图片等资源作为响应。HTTP 本身是无状态的，每次请求彼此独立。

早期的 HTTP/1.0 每次请求都需建立和关闭一次 TCP 连接，效率较低。HTTP/1.1 引入了持久连接（默认开启 keep-alive），允许在一个连接上多次请求和响应，显著提升了通信效率，也是目前最常用的 HTTP 协议版本。

【HTTP 交互过程】

HTTP 服务器的工作流程如下所示：

- **监听端口等待请求：**HTTP 服务器默认监听在 80 端口，等待客户端（如浏览器）发起连接。
- **建立 TCP 连接：**当客户端发出连接请求后，服务器响应并通过三次握手过程，成功建立 TCP 连接。
- **客户端发送请求报文：**连接建立后，客户端通过该连接发送 HTTP 请求报文，请求的 URL 指明了所需资源的路径。通常包括以下过程：
  - 客户端最初会请求主页资源（如 index.html），该页面可能包含多个其他对象的引用；
  - 浏览器根据页面内容中的链接，发起多个并行请求，下载网页中包含的图片、视频、脚本等资源。
- **服务器返回响应内容：**服务器接收到请求后，对其进行解析，根据请求生成响应报文，将所请求的资源通过 TCP 连接返回给客户端。
- **关闭 TCP 连接：**资源传输完成后，客户端主动发起连接释放（发送 FIN 报文），双方完成四次挥手，释放网络资源。

【HTTP1.1 协议格式】

HTTP 协议有两类消息：请求消息（request）和响应消息（response）

3.2.2.1. 请求消息

```
GET /somedir/page.html HTTP/1.1<CR><LF> # 请求行，包含GET、URL、协议版本
Host: www.someschool.edu<CR><LF>        # 首部行（本实验不涉及）
User-agent: Mozilla/4.0<CR><LF>
Connection: close<CR><LF>
Accept-language:fr<CR><LF>
<CR><LF>
(extra carriage return, line feed)      # 请求体（本实验不涉及）
```

注：<CR>和<LF>分别指回车符（Carriage Return）和换行符（Line Feed），即\r和\n。

3.2.2.2. 响应消息

```
HTTP/1.1 200 OK<CR><LF>          # 状态行(协议版本，  状态码，相应状态信息)
Connection: Keep-Alive<CR><LF>   # HTTP/1.1 默认开启长连接
Content-Type: text/html<CR><LF>  # 响应体的MIME类型
Content-Length: 6821<CR><LF>     # 响应体长度
<CR><LF>                         # (注意此处需要有一个<CR><LF>)
data data data data data ...     # 数据
```

其中，状态代码可以是“200 OK”、“404 NOT FOUND”等。

8.1 Web 发送 404 响应的代码 Step1 设计与实现

设计流程如下：

1. 发送 HTTP 状态行：构造 HTTP 响应的第一行，即状态行（Status Line），指示客户端请求的资源不存在（404）。状态行格式为：HTTP/版本 状态码 状态描述。使用 tcp_send() 发送构造好的字符串。
2. 发送 HTTP 连接信息：表明当前连接在响应后不关闭（Keep-Alive），允许复用 TCP 连接。这是一个可选的 HTTP 首部字段，用于控制连接行为。
3. 发送 HTTP 内容类型信息：指定响应体的数据格式为 HTML，并使用 UTF-8 编码。有助于客户端正确解释和渲染 HTML 页面内容。
4. 发送 HTTP 内容长度：计算并发送响应体的长度，以字节为单位。Content-Length 是 HTTP 协议中必须的字段之一，尤其是在 Keep-Alive 连接中，用于帮助客户端确定报文的边界。
5. 发送空行，分隔头部与正文：按照 HTTP 协议规范，空行标志着头部部分结束，正文（Body）部分即将开始。这是固定格式，必须严格发送 \r\n\r\n 的两个换行（当前这一句发送的是第二个换行的起始）。
6. 发送 HTTP 响应体内容：发送定义好的 404 页面内容，例如：<html><body><h1>404 Not Found</h1></body></html>。该内容作为 HTML 页面呈现给用户，说明请求资源不存在。

代码实现如下：

```
30  /**
31   * @brief 响应函数
32   *
33   * @param tcp_conn  指向当前 TCP 连接的指针
34   * @param url_path  资源文件路径
35   * @param port      本连接端口
36   * @param dst_ip    目标 IP 地址
37   * @param dst_port  目标端口
38   */
     解释代码 | 注释代码 | 生成单测 | ×
39  void http_respond(tcp_conn_t *tcp_conn, char *url_path, uint16_t port, uint8_t *dst_ip, uint16_t dst_port) {
40      FILE *file;
41      char file_path[HTTP_MAX_PATH_LENGTH];
42      memcpy(file_path, HTTP_RESOURCE_DIR, sizeof(HTTP_RESOURCE_DIR));
43
44      // 获取文件路径，打开文件
45      if (strcmp(url_path, "/") == 0) {  // 如果路径为 "/", 则默认打开 index.html
46          strcat(file_path, "/index.html");
47      } else {
48          strcat(file_path, url_path);  // 否则，文件路径为 "${HTTP_RESOURCE_DIR}${url_path}"
49      }
50      // 打开文件
51      file = fopen(file_path, "rb");
52
53      char resp_buffer[HTTP_MAX_RESPONSE_LENGTH] = {0};
54
55      // 文件不存在时发送 404 响应
56      if (!file) {
57          // HTTP 404 响应请求体
58          char *not_found_body = "<HTML><TITLE>Not Found</TITLE>\r\n"
59                                 "The resource specified\r\n"
60                                 "is unavailable or nonexistent.\r\n"
61                                 "</BODY></HTML>\r\n";
62          /* Step1 : 发送 HTTP 404 请求头 */
63          // TODO: 发送 HTTP 状态行
64          sprintf(resp_buffer, "HTTP/1.1 404 NOT FOUND\r\n");
65          tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
66
```

```
66
67          // 发送 HTTP 连接信息
68          sprintf(resp_buffer, "Connection: Keep-Alive\r\n");
69          tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
70
71          // TODO: 发送 HTTP 内容类型
72          sprintf(resp_buffer, "Content-Type: text/html; charset = utf-8\r\n");
73          tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
74
75          // TODO: 发送 HTTP 内容长度
76          sprintf(resp_buffer, "Content-Length: %lu\r\n", strlen(not_found_body));
77          tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
78
79          // TODO: 发送 HTTP 响应头与响应体的分隔符
80          sprintf(resp_buffer, "\r\n");
81          tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
82
83          // TODO: 发送 HTTP 响应体
84          tcp_send(tcp_conn, (uint8_t *)not_found_body, strlen(not_found_body), port, dst_ip, dst_port);
85
86          return;
87      }
```

8.2 Web 发送 200 响应的代码 Step2 和 Step3 设计与实现

设计流程如下：

1. 发送 HTTP 状态行：构造并发送 HTTP 状态行，表明请求成功，服务器将返回所请求的资源。状态行为 HTTP/版本 状态码 状态描述 格式，此处为 200 OK。
2. 发送 HTTP 连接信息：表示此次通信之后连接不立即关闭，支持 TCP 长连接（Connection Reuse），减少后续开销。客户端和服务器都需支持 Keep-Alive 才有效。
3. 发送 HTTP 内容类型（MIME Type）：依据请求文件的类型（例如 .html, .jpg, .css）调用 http_get_mime_type() 获取对应的 MIME 类型。构造 Content-Type 字段，通知客户端如何解析接收的资源内容。
4. 发送 HTTP 内容长度：利用 fseek() 和 ftell() 获取待发送文件的总字节数。设置 Content-Length 字段，确保客户端知道将要接收多少数据。注意 fseek() 之后需要回到文件起始位置，以便后续读取。
5. 发送空行，标识响应头结束：HTTP 协议中，响应头与响应体之间必须由一行空行（\r\n）分隔。这是协议约定的格式，不可省略。
6. 发送 HTTP 响应体内容：使用循环读取本地文件内容，每次最多读取 resp_buffer 缓冲区的大小。返回值 bytes_read 表示实际读取到的字节数（可能小于缓冲区大小）。将每一块数据通过 tcp_send() 发送到客户端，实现完整响应体的传输。

代码实现如下：

```
89      /* Step2 : 发送 HTTP 请求头 */
90      // TODO: 发送 HTTP 状态行
91      sprintf(resp_buffer, "HTTP/1.1 200 OK\r\n");
92      tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
93
94      // 发送 HTTP 连接信息
95      sprintf(resp_buffer, "Connection: Keep-Alive\r\n");
96      tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
97
98      const char *content_type = http_get_mime_type(file_path);
99      // TODO: 发送 HTTP 内容类型，根据文件类型设置 MIME 类型
100     sprintf(resp_buffer, "Content-Type: %s\r\n", content_type);
101     tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
102
103     fseek(file, 0, SEEK_END);
104     size_t content_length = ftell(file);
105     fseek(file, 0, SEEK_SET);
106     // TODO: 发送 HTTP 内容长度
107     sprintf(resp_buffer, "Content-Length: %lu\r\n", content_length);
108     tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
109
110     // TODO: 发送 HTTP 响应头与响应体的分隔符
111     sprintf(resp_buffer, "\r\n");
112     tcp_send(tcp_conn, (uint8_t *)resp_buffer, strlen(resp_buffer), port, dst_ip, dst_port);
113
```

```
114     /* Step3 ：发送 HTTP 响应体 */
115     size_t bytes_read;
116     while ((bytes_read = fread(resp_buffer, 1, sizeof(resp_buffer), file)) > 0) {
117         // TODO: 每次发送读取的文件内容块
118         tcp_send(tcp_conn, (uint8_t *)resp_buffer, bytes_read, port, dst_ip, dst_port);
119     }
120 
121     // 后处理：关闭文件
122     fclose(file);
123 }
124 
```

二、 实验结果截图及分析

```
D:\code\net-lab\build>ctest
Test project D:/code/net-lab/build
    Start 1: eth_in
1/8 Test #1: eth_in ..........................   Passed    0.05 sec
    Start 2: eth_out
2/8 Test #2: eth_out .........................   Passed    0.03 sec
    Start 3: arp_test
3/8 Test #3: arp_test ........................   Passed    0.05 sec
    Start 4: ip_test
4/8 Test #4: ip_test .........................   Passed    0.06 sec
    Start 5: ip_frag_test
5/8 Test #5: ip_frag_test ....................   Passed    0.03 sec
    Start 6: icmp_test
6/8 Test #6: icmp_test .......................   Passed    0.05 sec
    Start 7: udp_test
7/8 Test #7: udp_test ........................   Passed    0.04 sec
    Start 8: tcp_test
8/8 Test #8: tcp_test ........................   Passed    0.05 sec

100% tests passed, 0 tests failed out of 8

Total Test time (real) =   0.40 sec
```

（请展示并详细分析实验结果的截图。可以利用 log 文件、通过 Wireshark 打开的 pcap 文件或 Wireshark 实时捕获的网络报文。）

1. Eth 协议实验结果及分析

（展示以太网帧捕获截图，分析帧的结构和内容是否符合预期。检查目的 MAC 地址、源 MAC 地址、协议类型字段以及数据部分）

1. 实验结果：与 Eth 协议相关的测试：eth_out 和 eth_in 的测试均已通过，如下图所示：

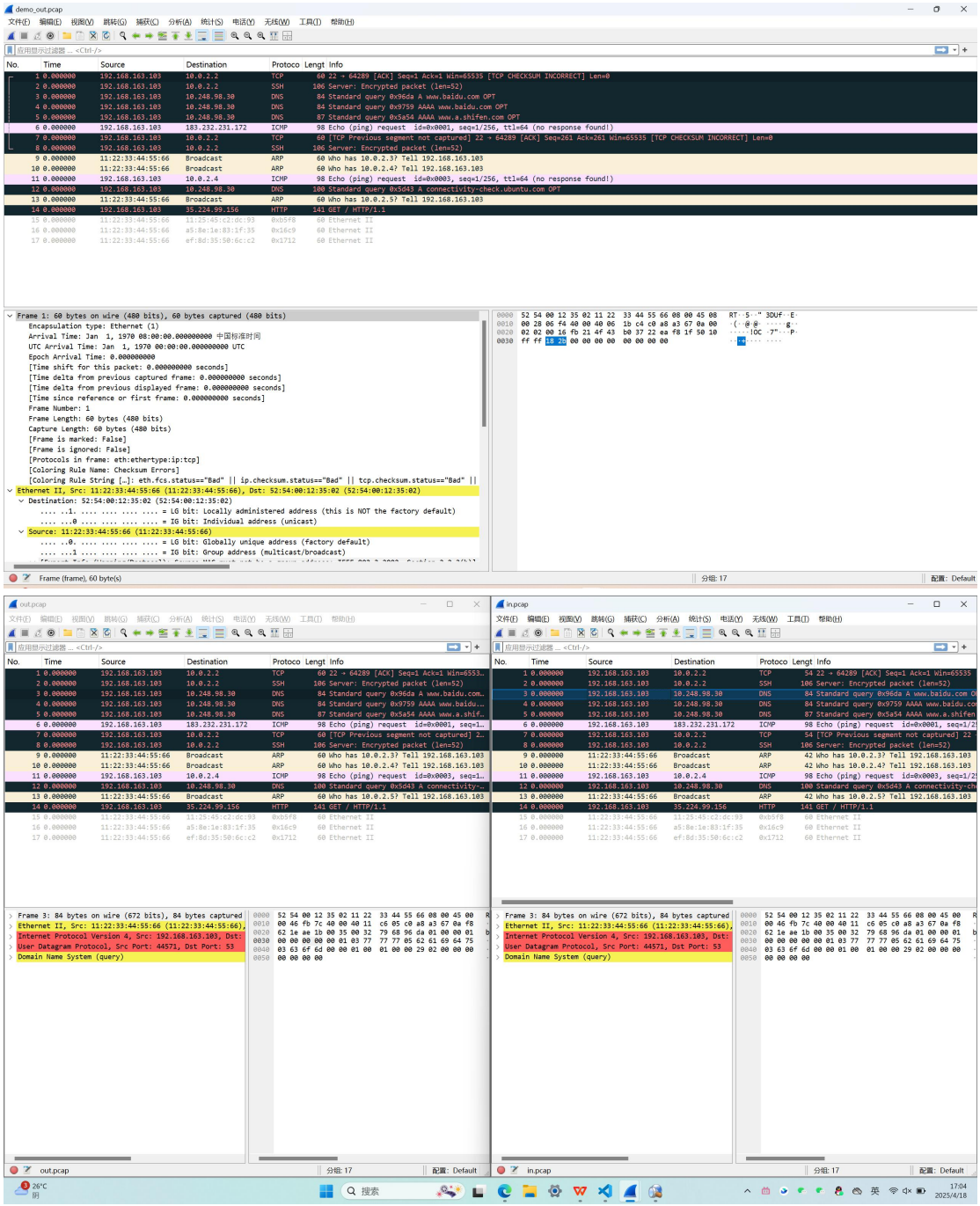
```
D:\code\net-lab\build>ctest -R eth_out
Test project D:/code/net-lab/build
    Start 2: eth_out
1/1 Test #2: eth_out .........................   Passed    0.56 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.58 sec
```

```
D:\code\net-lab\build>ctest -R eth_in
Test project D:/code/net-lab/build
    Start 1: eth_in
1/1 Test #1: eth_in ............................   Passed    0.42 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.43 sec
```

2. 展示以太网帧捕获截图，分析帧的结构和内容是否符合预期。

对于 eth_out 测试的以太网帧捕获：

通过与 demo_out 的对比，二者帧的结构和内容符合预期



Frame 3: 84 bytes on wire (672 bits), 84 bytes captured (672 bits)

Ethernet II, Src: 11:22:33:44:55:66 (11:22:33:44:55:66), Dst: 52:54:00:12:35:02 (52:54:00:12:35:02)

Destination: 52:54:00:12:35:02 (52:54:00:12:35:02)

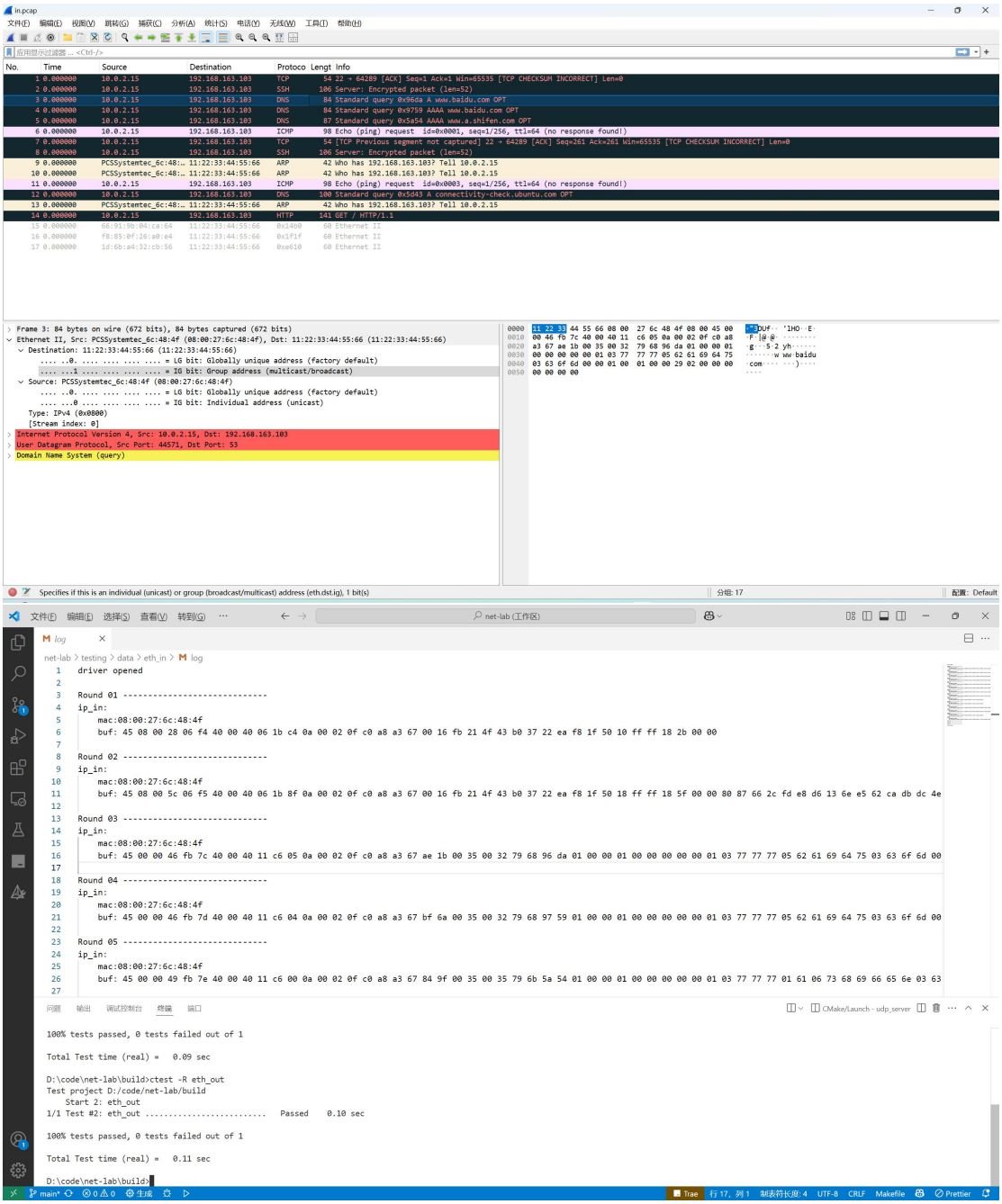
.... ..1. .... = LG bit: Locally administered address (this is NOT the factory default)

....0 .... = IG bit: Individual address (unicast)
Source: 11:22:33:44:55:66 (11:22:33:44:55:66)
.... ..0. .... = LG bit: Globally unique address (factory default)
....1 .... = IG bit: Group address (multicast/broadcast)
[Expert Info (Warning/Protocol): Source MAC must not be a group address: IEEE 802.3-2002, Section 3.2.3(b)]
[Source MAC must not be a group address: IEEE 802.3-2002, Section 3.2.3(b)]
[Severity level: Warning]
[Group: Protocol]
Type: IPv4 (0x0800)
[Stream index: 0]
Internet Protocol Version 4, Src: 192.168.163.103, Dst: 10.248.98.30
User Datagram Protocol, Src Port: 44571, Dst Port: 53
Domain Name System (query)

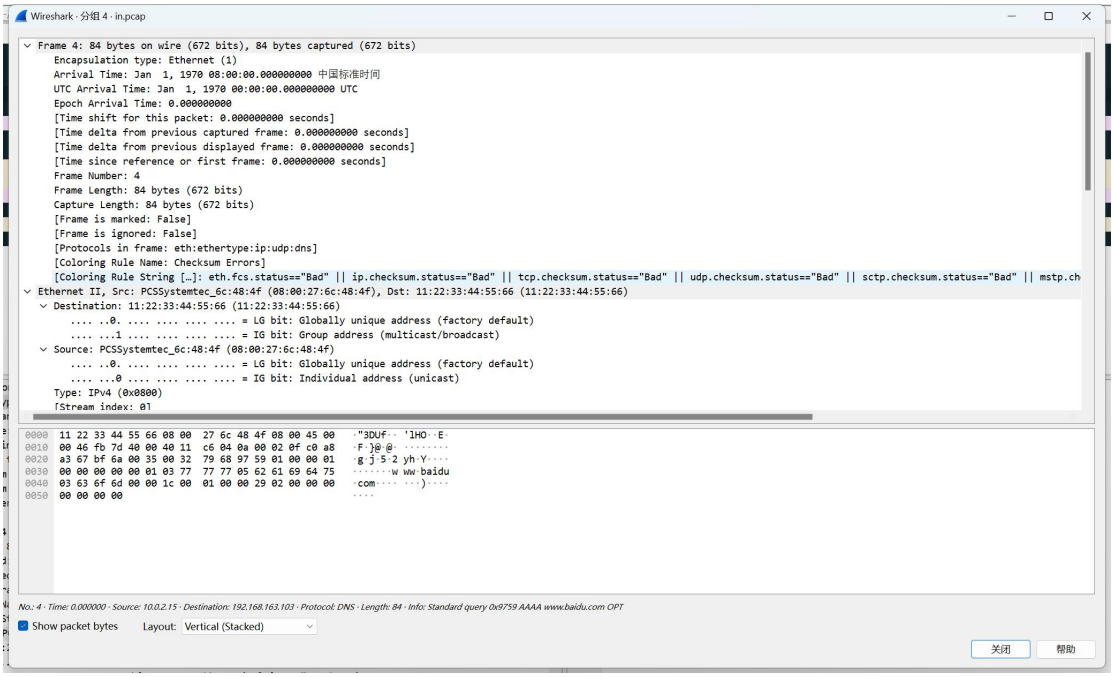
检查目的 MAC 地址、源 MAC 地址、协议类型字段以及数据部分
1. 目的 MAC 地址：52:54:00:12:35:02
2. 源 MAC 地址：11:22:33:44:55:66
3. 协议类型字段：IPv4（0x0800）
4. 数据部分：

协议层	大致长度	内容说明
Ethernet 头	14 字节	包含源 MAC、目的 MAC、EtherType
IPv4 头	20 字节	源 IP、目的 IP
UDP 头	8 字节	源端口、目的端口
DNS 查询数据	42 字节	DNS 报文

对于 eth_in 测试的以太网帧捕获：



以第 4 帧为例子：
对比之下发现，二者相同：



观察不难发现，in.pcap 中的第 4 个数据包确实被正确地解析、去除以太网头部，并被输出到 log 文件中，作为 Round 04 的 ip_in。Round 04 的 ip_in 的 mac、buf，也与 in.pcap 中的第 4 个数据包的内容相一致。

检查目的 MAC 地址、源 MAC 地址、协议类型字段以及数据部分

1. 目的 MAC 地址：11:22:33:44:55:66
2. 源 MAC 地址：08:00:27:6c:48:4f
3. 协议类型字段：IPv4（0x0800）
4. 数据部分：

以太网头部为 **14 字节**，从第 15 字节开始为数据部分：

- 总帧大小为：84 字节
- 所以**数据部分长度 = 84 - 14 = 70 字节**

数据部分内容结构如下：

协议	偏移位置	长度	内容
IPv4 Header	14 ~ 33	20 字节	Src IP: 10.0.2.15, Dst IP: 192.168.163.103
UDP Header	34 ~ 41	8 字节	Src Port: 49002, Dst Port: 53
DNS 查询数据	42 ~ 83	42 字节	DNS 查询报文

📌 总结：

复制代码

```
数据部分（Payload）起始字节：第14字节（偏移13）
长度：70 字节
结构：IPv4（20） + UDP（8） + DNS 查询（42）
```

2. ARP 协议实验结果及分析

（展示 ARP 请求和响应包的捕获截图，分析其请求和响应过程是否正常。检查 ARP 请求中的目标 IP 地址和发送方 MAC 地址，ARP 响应中的目标 MAC 地址。）

1. 实验结果：与 ARP 协议相关的测试：arp_test 的测试已通过，如下图所示：

```
D:\code\net-lab\build>ctest -R arp_test
Test project D:/code/net-lab/build
    Start 3: arp_test
1/1 Test #3: arp_test .........................   Passed    0.43 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.44 sec
```

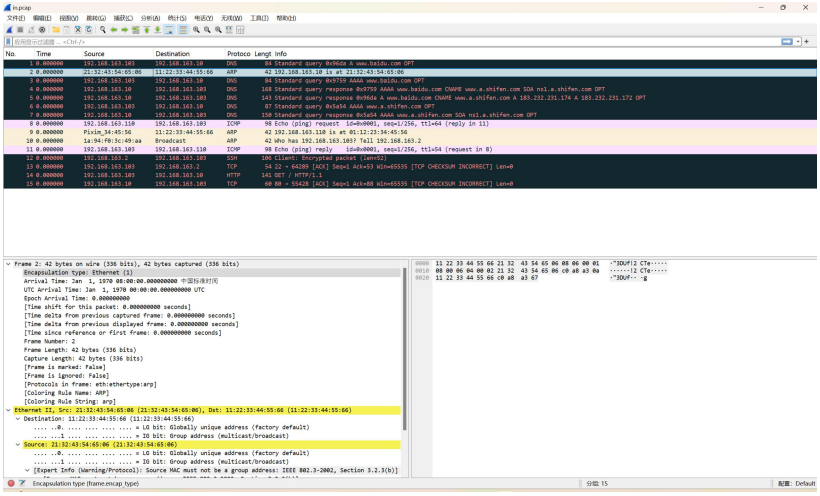
2. 展示以太网帧捕获截图，分析帧的结构和内容是否符合预期。

对于 arp_test 测试的以太网帧捕获：

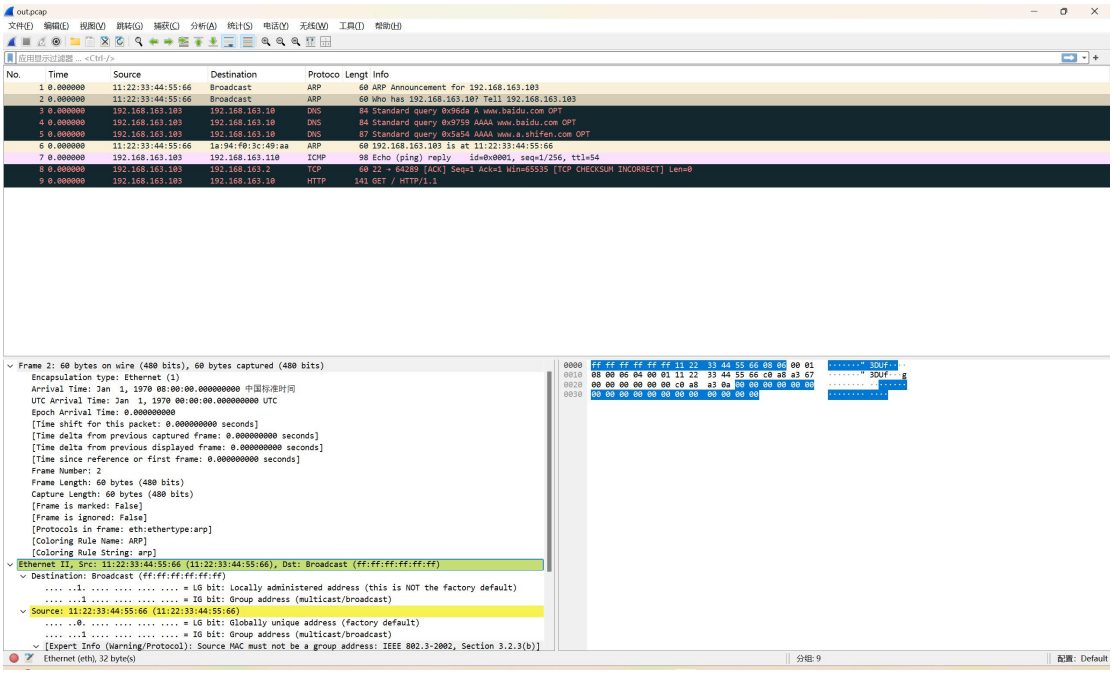
ARP 响应包：

字段	内容	说明
发送方 MAC 地址	21:32:43:54:65:06	IP 地址 192.168.163.10 所对应的 MAC
发送方 IP 地址	192.168.163.10	响应者的 IP 地址
目标 MAC 地址	11:22:33:44:55:66	ARP 请求者的 MAC
目标 IP 地址	192.168.163.103	请求者是想找这个 IP 的 MAC 地址
目的 MAC	11:22:33:44:55:66	单播

表示 IP 地址 192.168.163.103 想要获取 192.168.163.10 的 MAC 地址，而这个响应包明确回复了对应的 MAC 地址为 21:32:43:54:65:06。所以 ARP 响应过程是正常的。



ARP 请求包：



字段	内容
发送方 MAC 地址	11:22:33:44:55:66
发送方 IP 地址	192.168.163.103
目标 MAC 地址	00:00:00:00:00:00
目标 IP 地址	192.168.163.10
目的 MAC	ff:ff:ff:ff:ff:ff

主机 192.168.163.103 想要知道 IP 192.168.163.10 对应的 MAC 地址，于是向全网广播 ARP 请求。

ARP 请求和响应过程是正常的，功能完整闭环。

1. 请求者正确广播请求；
2. 目标主机收到后正确回应；
3. 响应内容与请求匹配；

3. IP 协议实验结果及分析

（展示 IP 数据包（包括分片）的捕获截图，分析 IP 头部字段的正确性。检查版本号、首部长度、总长度、标识、标志位、片偏移、TTL、协议类型等字段，同时分析分片机制是否准确。）

1.实验结果：与 IP 协议相关的测试：ip_test 和 ip_frag_test 的测试已通过，如下图所示：

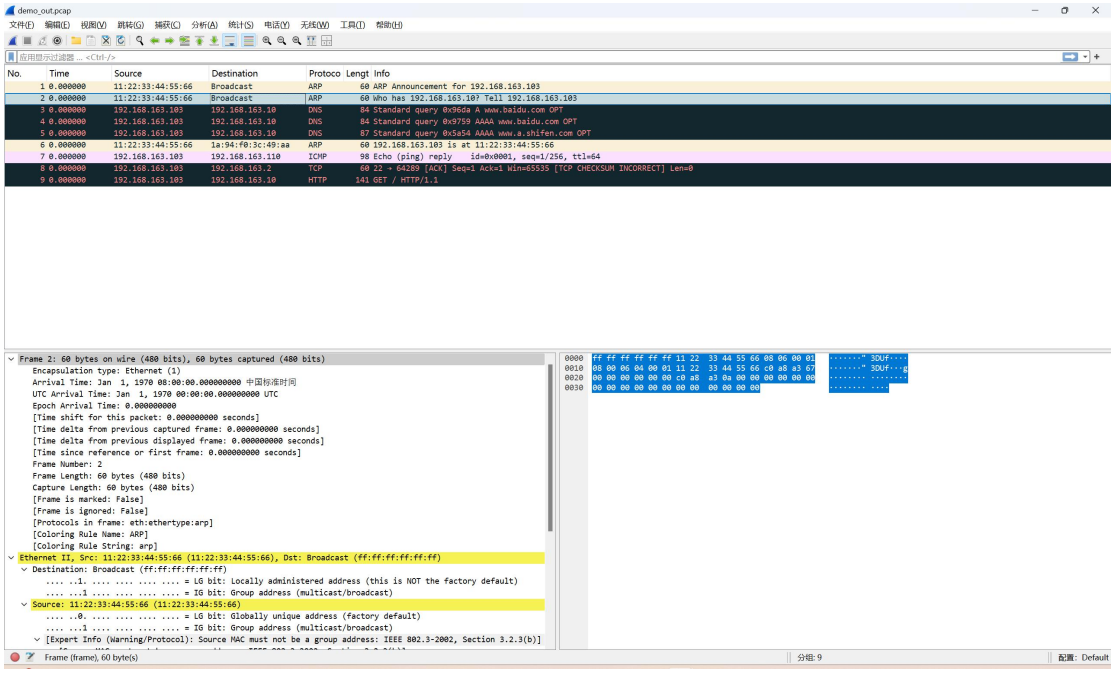
```
D:\code\net-lab\build>ctest -R ip_test
Test project D:/code/net-lab/build
    Start 4: ip_test
1/1 Test #4: ip_test ..........................   Passed    0.45 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.46 sec


D:\code\net-lab\build>ctest -R ip_frag_test
Test project D:/code/net-lab/build
    Start 5: ip_frag_test
1/1 Test #5: ip_frag_test ......................   Passed    0.42 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.43 sec
```

2.

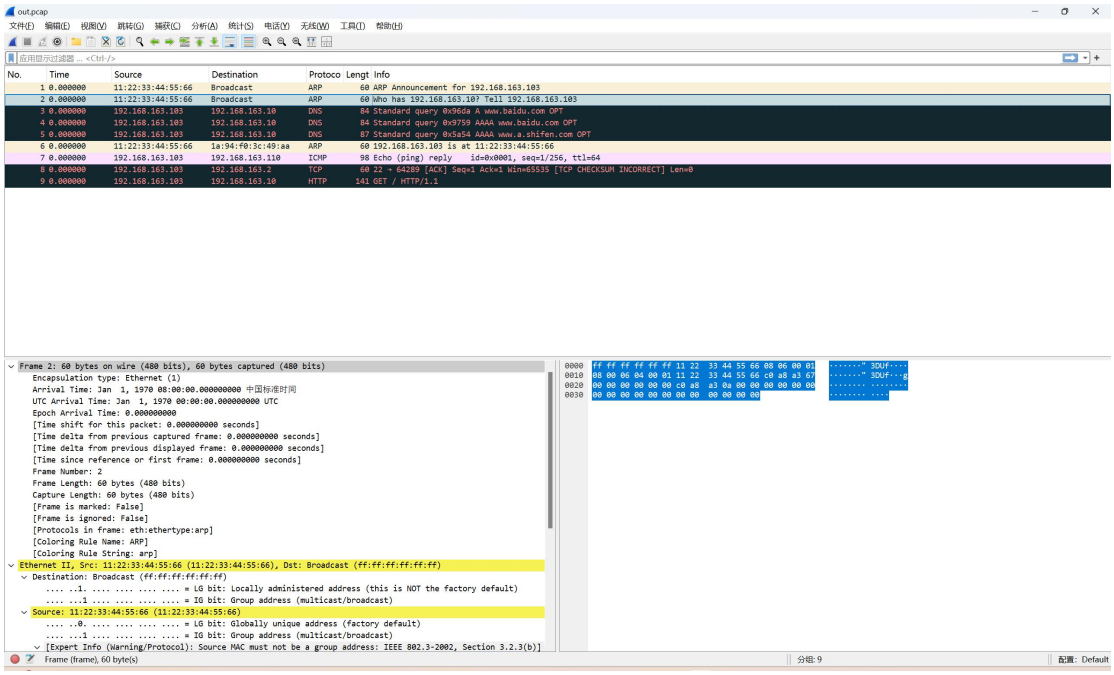
展示 IP 数据包（包括分片）的捕获截图，分析 IP 头部字段的正确性。检查版本号、首部长度、总长度、标识、标志位、片偏移、TTL、协议类型等字段，同时分析分片机制是否准确。

ip_test 的结果分析：

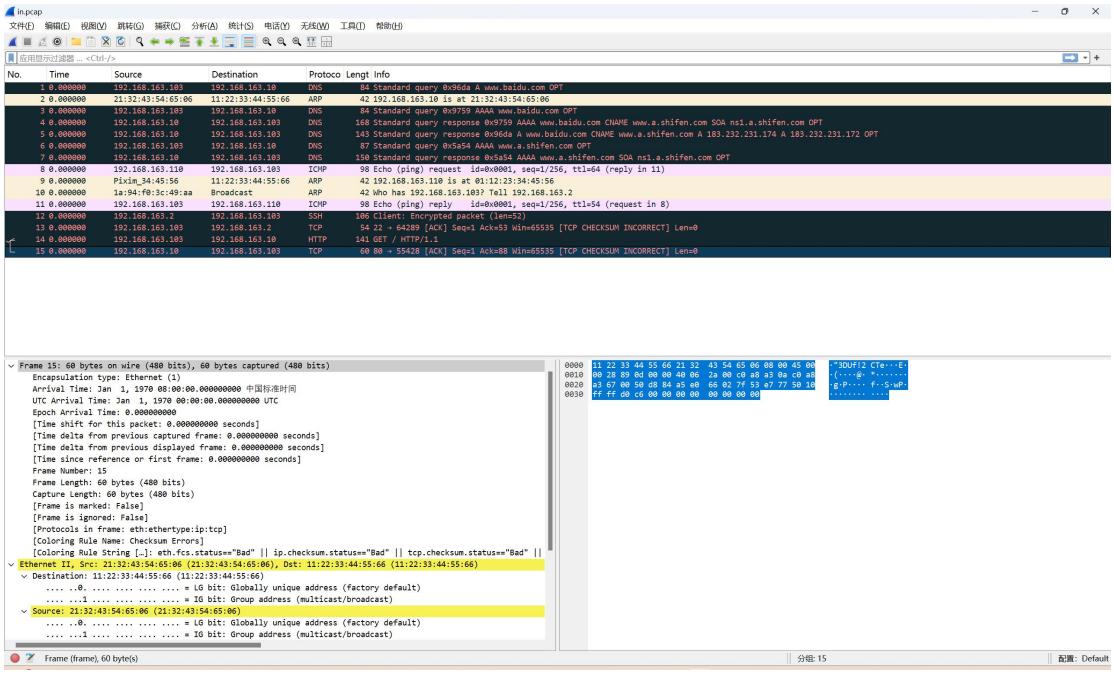
demo_out.pcap 文件如下图所示：



out.pcap 文件如下图所示：



in.pcap 文件如下图所示：

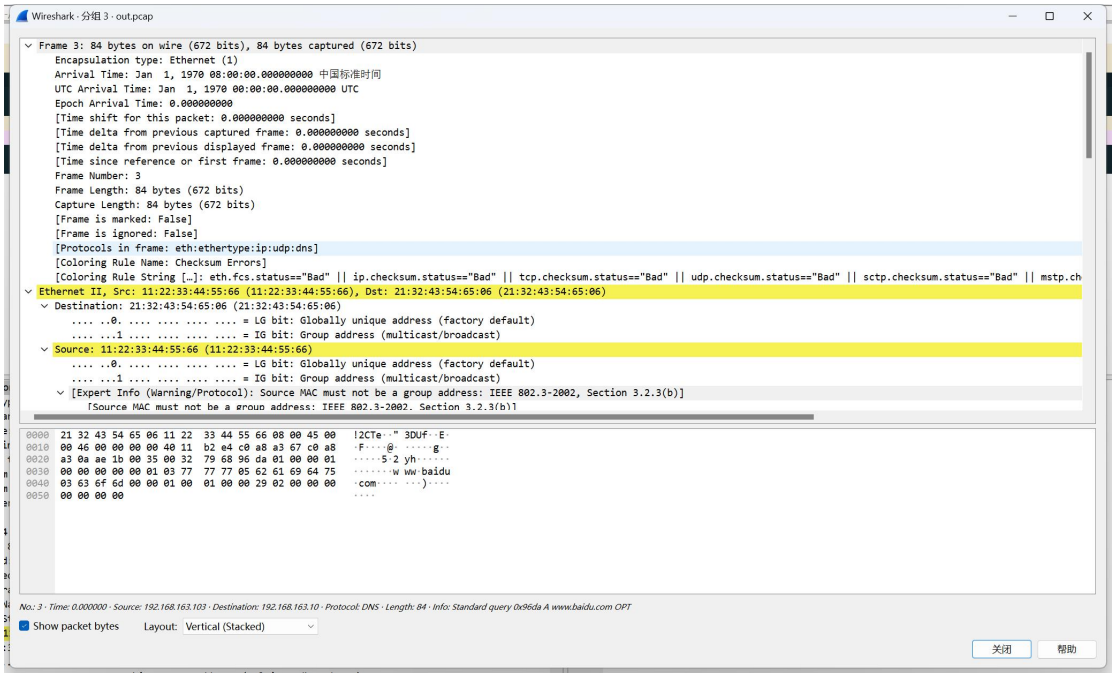


经对比 out.pcap 与 demo_out.pcap 文件，可以确认两者内容一致，测试因此顺利通过。进一步比较 out.pcap 与 in.pcap 可见：

- 在 out.pcap 的第 3、4、5 行中，所期望发送的 DNS 请求均已成功发出，并在 in.pcap 的第 4、5、7 行中收到了对应的 DNS 响应；
- 同样地，在接收到 in.pcap 第 8 行的 ICMP 请求后，程序也正确地返回了 out.pcap 第 7 行中的 ICMP 响应；
- 此外，后续涉及 SSH、TCP 和 HTTP 协议的报文也都被正确处理，无论是接收还是发送，均未出现异常。

综上，程序对网络报文的处理逻辑正确，能够实现预期的协议交互。

以下图为例进行分析：



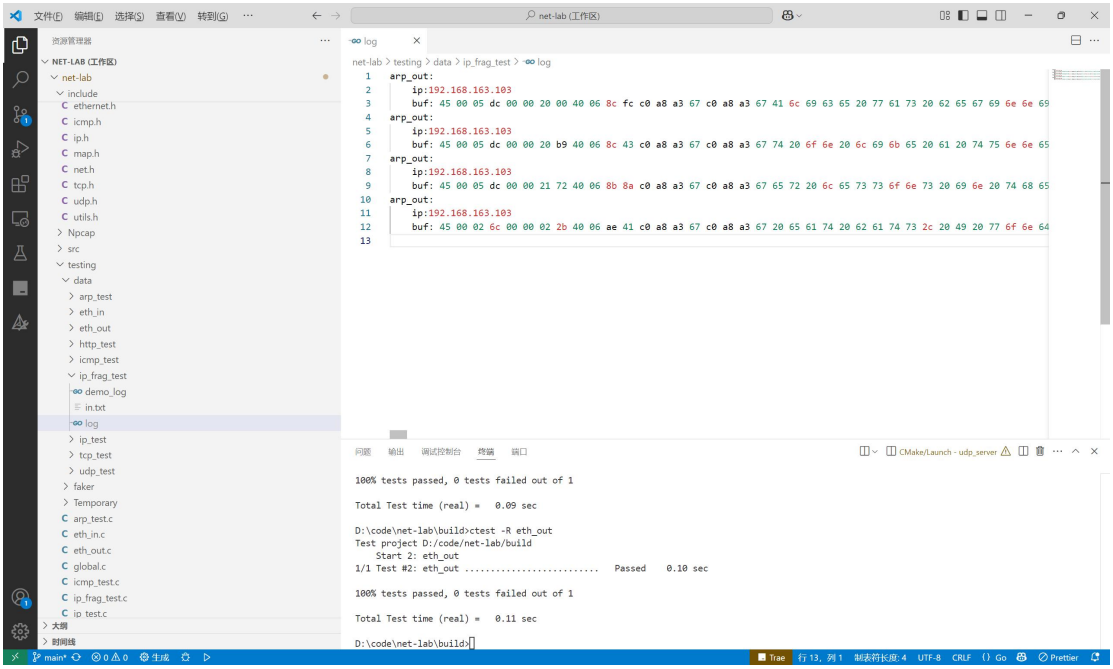
字段名	值	是否正常	说明
版本号	4	正常	IPv4 报文
首部长度	5	正常	IPv4 的最小头部长度，未使用额外选项字段。
总长度	70	正常	包括 IP 头部（20 字节）和 UDP 数据部分（50 字节），数值匹配。
标识	0x0000	正常	标识字段为 0，说明没有特别的分片需求，常见于未分片的简单报文。
标志位	0x0	正常	表示没有设置“Don't Fragment” 和 “More Fragments”，且未分片。
片偏移	0	正常	表示该报文为未分片报文的首片。
TTL	64	正常	是常见初始 TTL 值之一
协议类型	17(UDP)	正常	表示该 IP 报文承载的是 UDP 协议数据
头部校验和	0xb2e4	正常	校验结果与计算值一致，说明头部未损坏。
源地址	192.168.163.103	正常	与以太网帧源 MAC 地址匹配，符合逻辑。
目标地址	192.168.163.10	正常	与 DNS 服务器目标地址一致，合法。

该帧中 IP 头部各字段填写均正确，结构完整，没有逻辑错误或异常字段，符合 IPv4 协议规范。

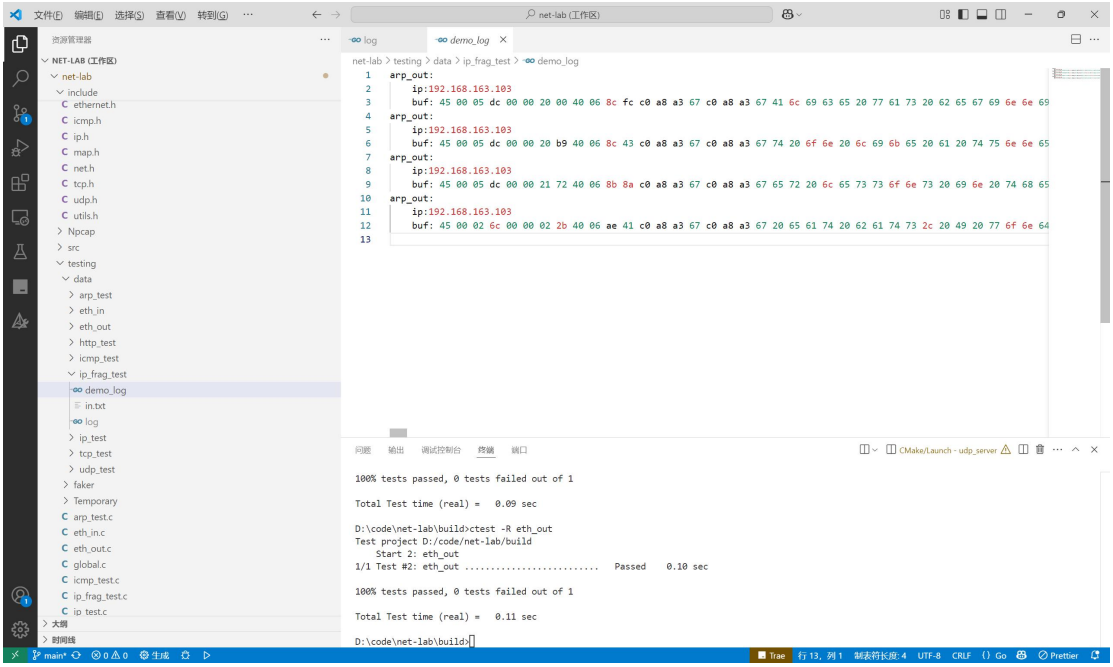
ip_frag_test 的结果分析：

通过查看 ip_frag_test.c 文件可以发现，该测试程序并未对 pcap 文件进行检查，而是读取一个名为 in.txt 的文本文件，其中包含较长的字符串内容。程序先将该文本内容读入到缓冲区 buf 中，再将其作为参数传入 ip_out 函数，从而验证该函数是否能够正确地对数据进行 IP 分片处理。

log 截图：



demo_log 截图：



前20字节是 IPv4 的头部。

Offset	Bytes	含义
0	45	版本4 + 头部长度5（表示20字节）
1	00	服务类型 TOS
2-3	05 dc	总长度（1500字节）
4-5	00 00	标识符
6-7	20 00	标志+偏移（不分片）
8	40	TTL（64）
9	06	协议号（TCP）
10-11	8c fc	Header checksum（校验和）
12-15	c0 a8 a3 67	源地址（192.168.163.103）
16-19	c0 a8 a3 67	目标地址（也是192.168.163.103）

从第 21 字节开始就是数据内容，即 TCP 数据（不含 TCP 头，因为只显示了 IP 包）

对比可发现 demo_log 和 log 文件的内容完全一致，因此可以通过测试。故分片功能实现正确。

综上，ip_frag_test 的功能实现正确。

4. ICMP 协议实验结果及分析

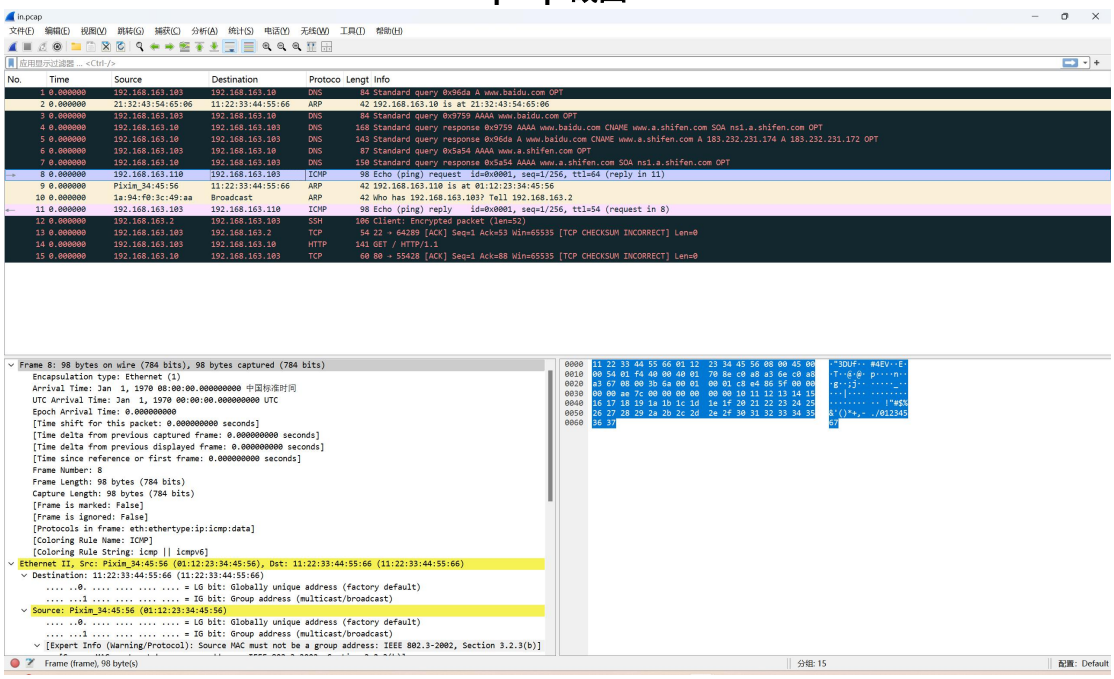
（展示 ICMP 报文的捕获截图，分析其报文内容（包括差错报文和查询报文）。检查 ICMP 类型、代码、校验和等字段，以及报文携带的信息。）

1. 实验结果：与 ICMP 协议相关的测试：icmp_test 的测试已通过，如下图所示：

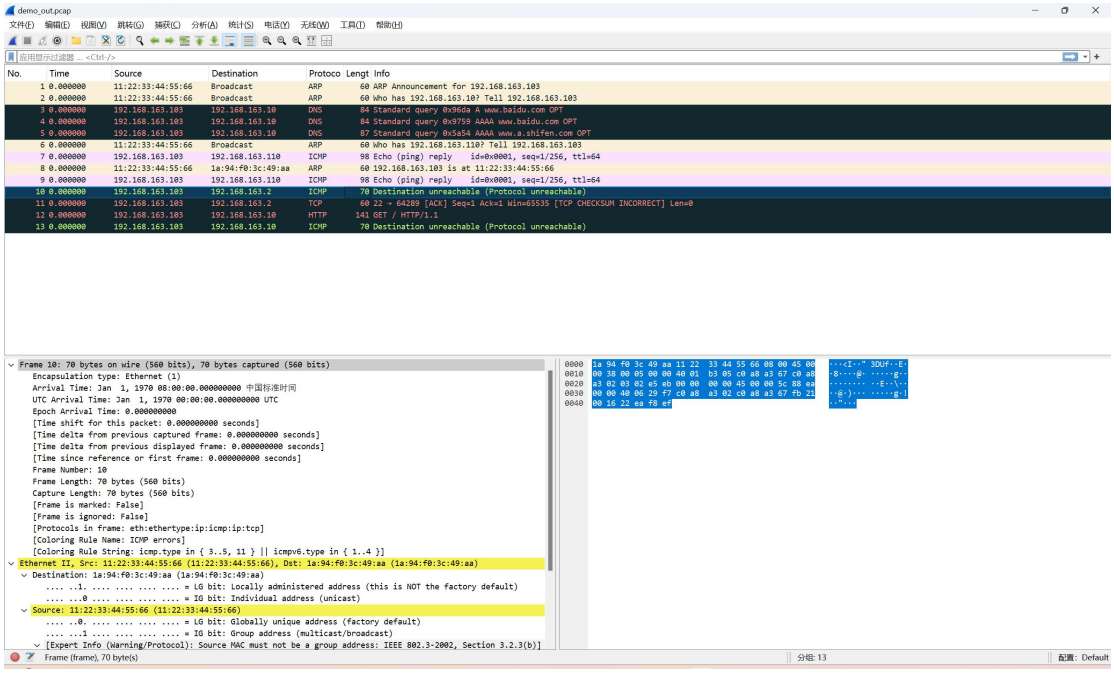
```
D:\code\net-lab\build>ctest -R icmp_test
Test project D:/code/net-lab/build
    Start 6: icmp_test
1/1 Test #6: icmp_test .........................   Passed    0.45 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.46 sec
```

2. 展示 ICMP 报文的捕获截图，分析其报文内容（包括差错报文和查询报文）。检查 ICMP 类型、代码、校验和等字段，以及报文携带的信息。

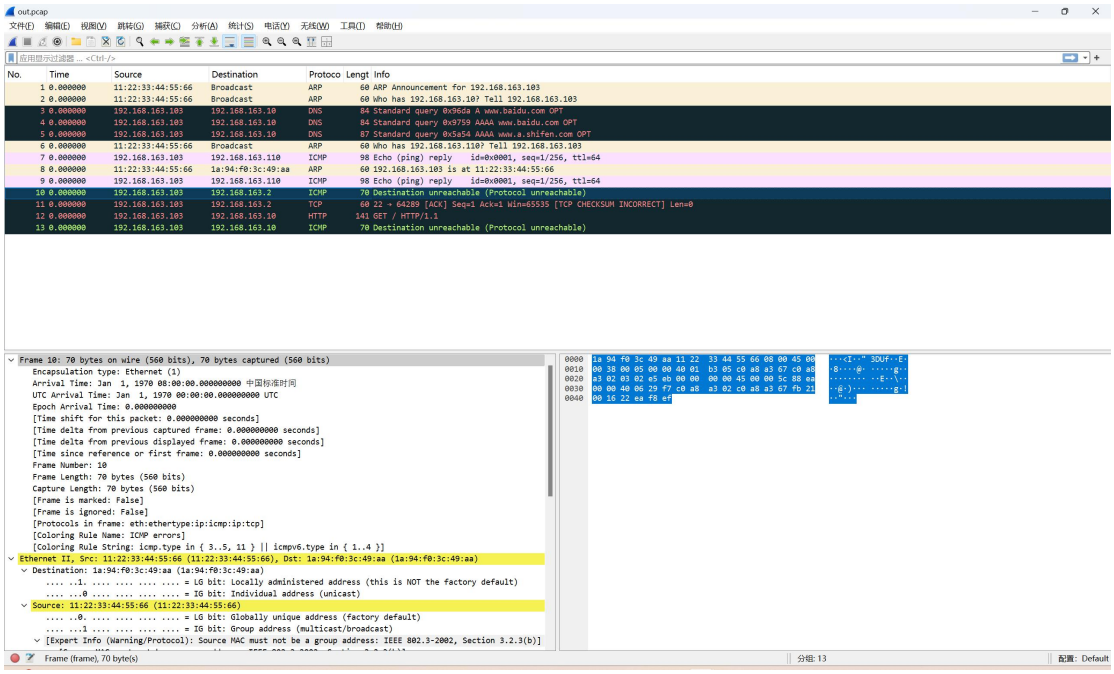
in.pcap 截图：



demo_out.pcap 截图：



out.pcap 截图：

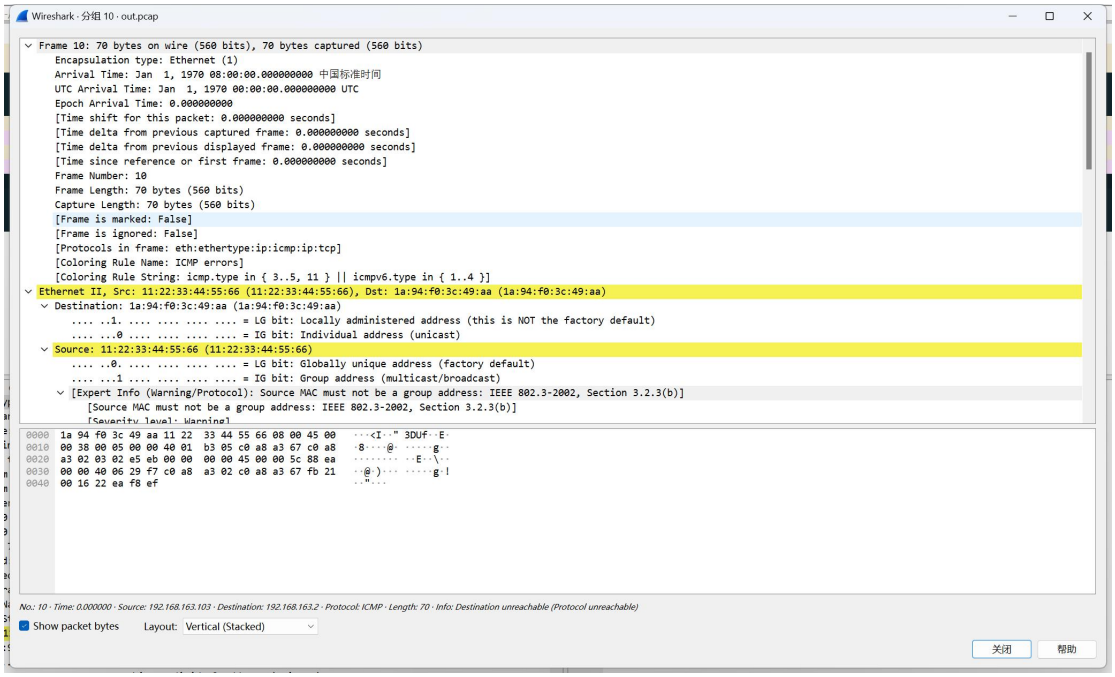


文件 out.pcap 与 demo_out.pcap 中的内容完全一致，因此测试结果符合预期。从分析可以看出，in.pcap 中第 8 行成功接收了一个 ICMP 回显请求，第 11 行成功接收了一个 ICMP 回显应答；同时，out.pcap 中的第 7 行与第 9 行分别成功发送了 ICMP 回显应答。这表明 ICMP 的查询与信息报文功能已正确实现。

另一方面，由于本实验的协议栈尚未实现全部协议，out.pcap 的第 10 行和第 13 行成功发送了 ICMP 差错报文（类型为协议不可达），反映出程序能够正确识别未知协议并进行差错处理，说明 ICMP 的差错报文功能也已正确实现。

综上所述，icmp_test 的相关功能实现是正确且符合预期的。

差错报文分析：



1. ICMP Type: 3 → Destination Unreachable（目的不可达）
2. Code: 2 → Protocol Unreachable（协议不可达）

结论：这是一个差错报文，表示接收端无法识别发送端发送的上层协议（如 TCP/UDP），即“协议不可达”。

属性	值
ICMP 类型	3 → Destination Unreachable
ICMP 代码	2 → Protocol Unreachable
校验和	正确 0xe5eb
是否为差错报文	是，表示协议不可达
引用的原始报文	一个 TCP 报文，目的端口为 22（SSH）

报文携带的信息：

这个差错报文还携带了原始导致错误的数据包的 IP 首部 + 前 8 个字节数据（这是 ICMP 差错报文的规范内容）：

携带的原始 IP 报文：

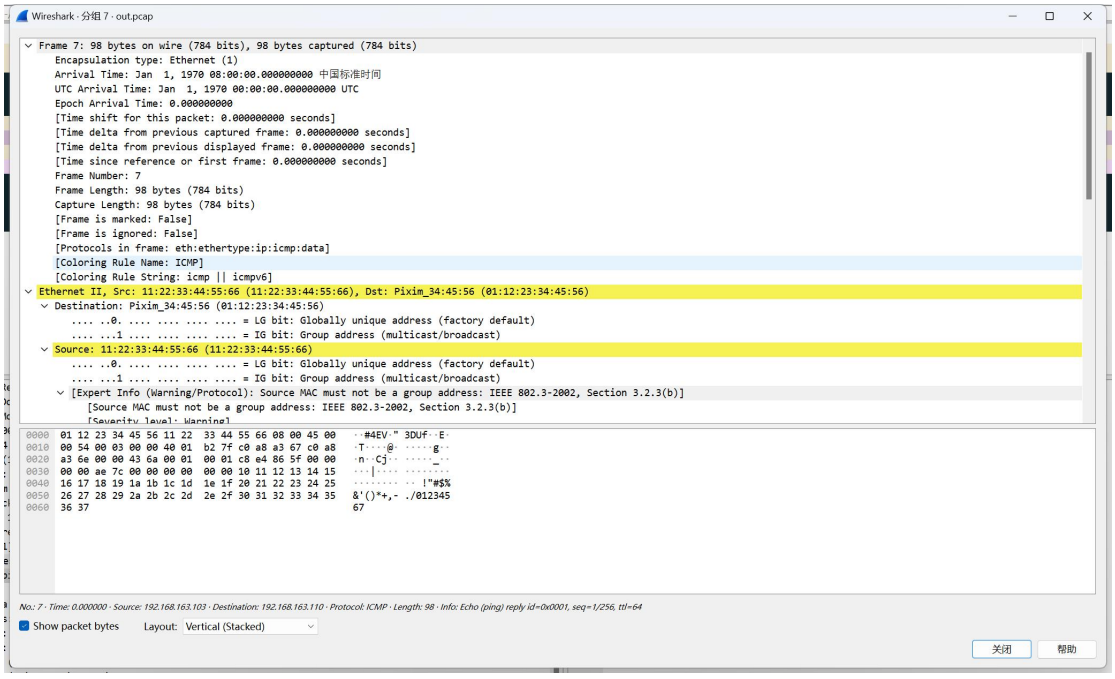
- 源 IP：192.168.163.2
- 目的 IP：192.168.163.103
- 上层协议：TCP (6)

携带的原始 TCP 报文：

- 源端口：64289
- 目的端口：22（SSH）

说明：这个 TCP 报文可能尝试连接 192.168.163.103:22，但对方没有 TCP 协议栈或未启用 TCP，因此回应了一个“协议不可达”错误。

查询报文分析：



报文类型字段：

Type = 0 → 表示 Echo Reply（回显应答）

Code = 0 → 表示无特定子类型，标准回显响应

所以，这是一个 ICMP 查询报文（Query Message）中的一种，属于 Echo Reply（回应 ping 请求）

属性	值
ICMP 类型	0 → Echo Reply
ICMP 代码	0 → 无特定子类型
校验和	正确，0x436a
是否为查询报文	是，表示回显应答
Sequence	1 Ping 顺序编号
数据区	56 字节，ping 的有效载荷

数据部分：

Data: c8e4865f00000000ae7c000000000000...

[Length: 56]

- 这是 ICMP 报文中携带的 payload，通常在 Echo Request 和 Echo Reply 中会保持一致。
- 长度 56 字节，是 ping 默认数据长度（一般为 56 bytes + ICMP header = 64 bytes）

5. UDP 协议实验结果及分析

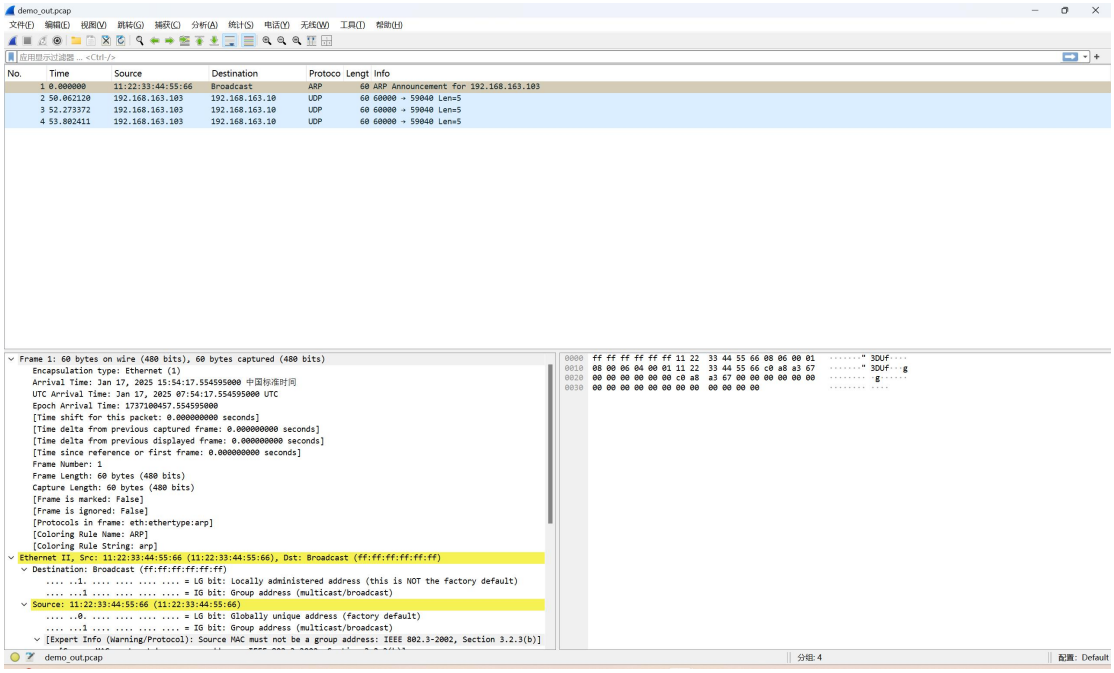
（展示 UDP 数据包的捕获截图，解析 UDP 头部和载荷内容，分析是否达到预期。检查源端口号、目的端口号、长度、校验和等字段，以及载荷数据。）

1. 实验结果：与 UDP 协议相关的测试：udp_test 的测试已通过，如下图所示：

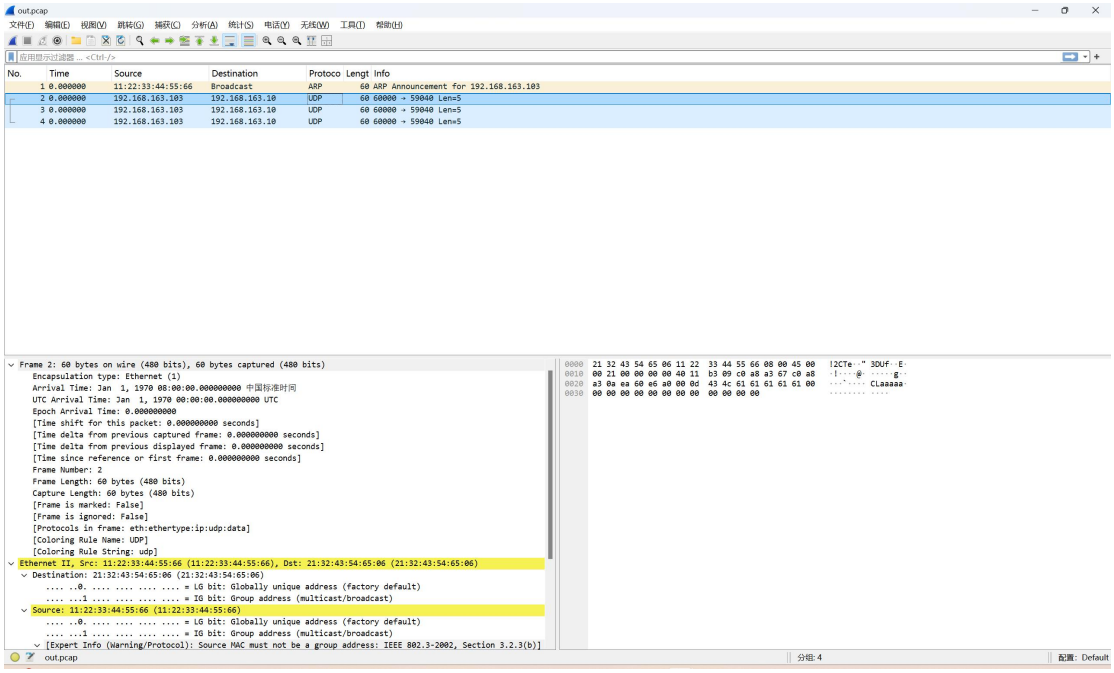
```
D:\code\net-lab\build>ctest -R udp_test
Test project D:/code/net-lab/build
    Start 7: udp_test
1/1 Test #7: udp_test .........................   Passed    0.04 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.05 sec
```

2. 展示 UDP 数据包的捕获截图，解析 UDP 头部和载荷内容，分析是否达到预期。检查源端口号、目的端口号、长度、校验和等字段，以及载荷数据。

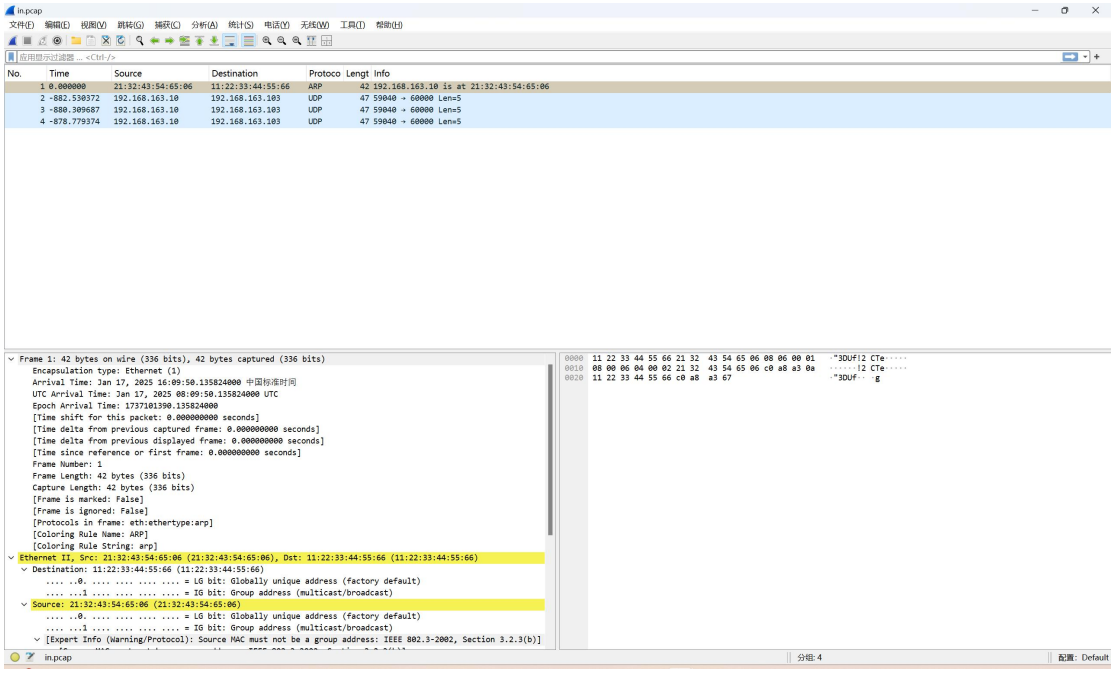
demo_out.pcap 截图：



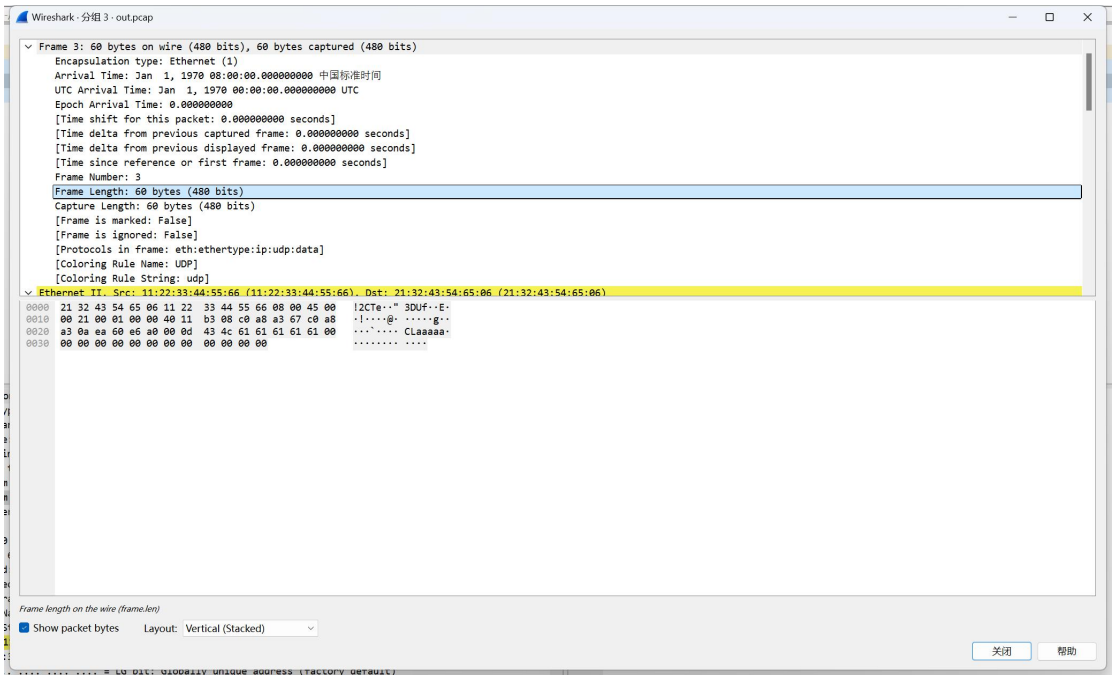
out.pcap 截图：



in.pcap 截图：



经过对比和分析，文件 out.pcap 与 demo_out.pcap 中的内容完全一致，因此测试结果符合预期。



UDP 头部内容：

字段	值	分析说明
源端口号	60000	客户端临时端口
目的端口号	59040	自定义端口
长度	13 字节	包括头部 8 字节+载荷 5 字节
校验和	0x434c	校验正确

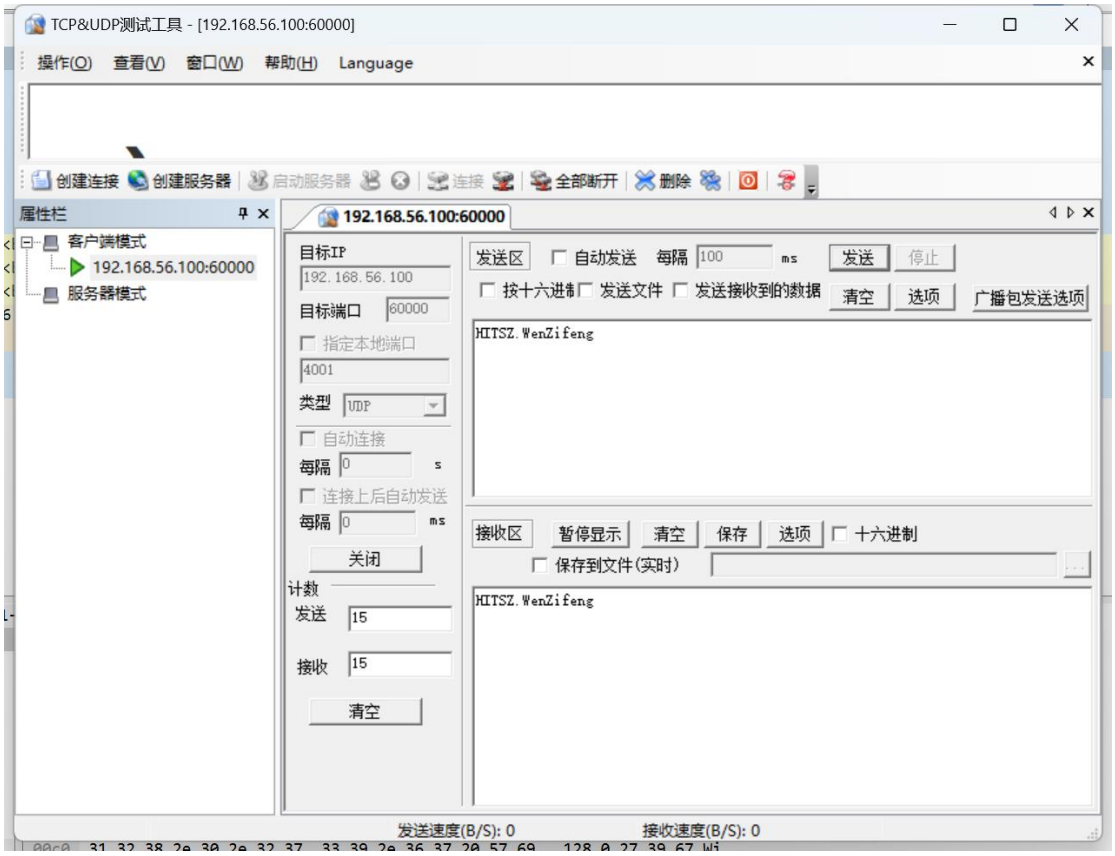
UDP 载荷内容：

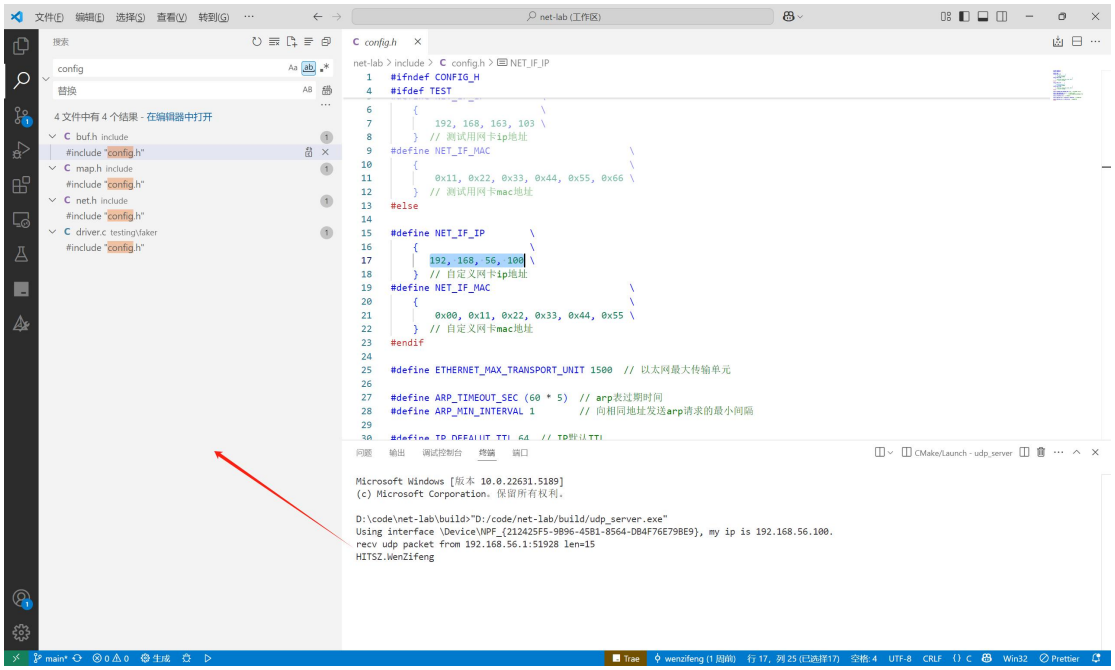
1. 载荷长度：5 字节
2. 原始数据（Hex）：61 61 61 61 61
3. ASCII 解码：aaaaa

UDP 数据包结构清晰，所有字段都符合规范。校验和也计算正确，载荷数据 aaaaa 和预期发送的一致。因此：该数据包完全达到了预期目标，是一个结构完整、内容准确的 UDP 包。

IP 总长度正确	20 + 13 = 33
UDP 长度正确	8 + 5 = 13
校验和有效	正确
源端口号	60000
目的端口号	59040
长度	13 字节
校验和	0x434c
载荷数据	61 61 61 61 61 (aaaaa)

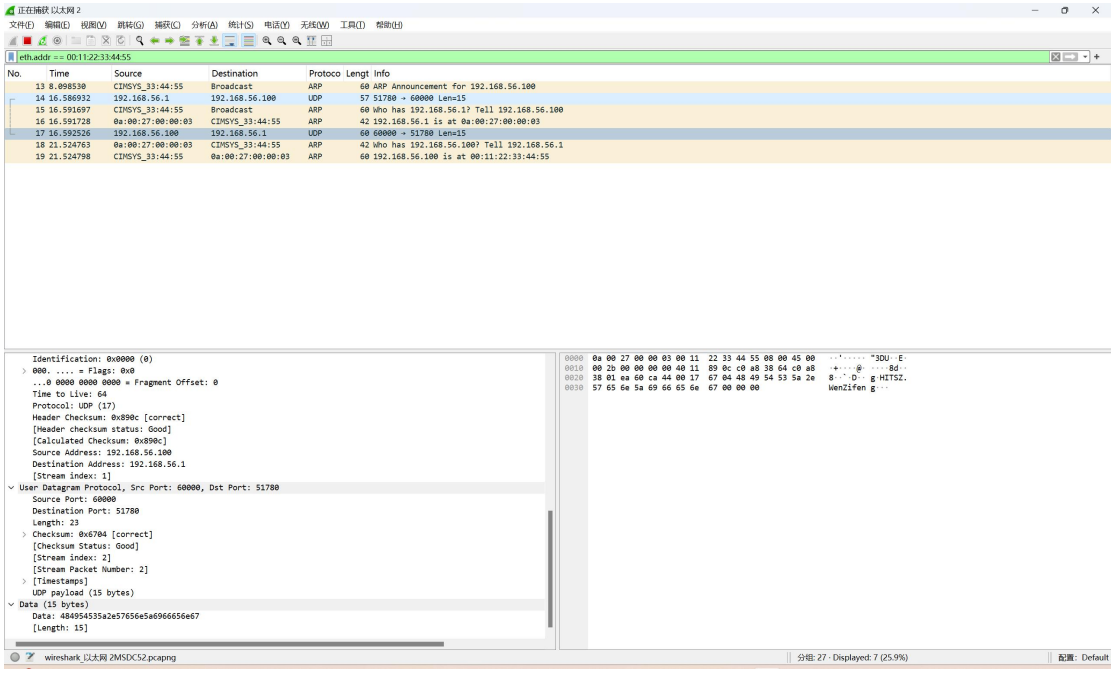
3. 实验结果：与 UDP 协议相关的测试：udp_server 的测试已通过，如下图所示：



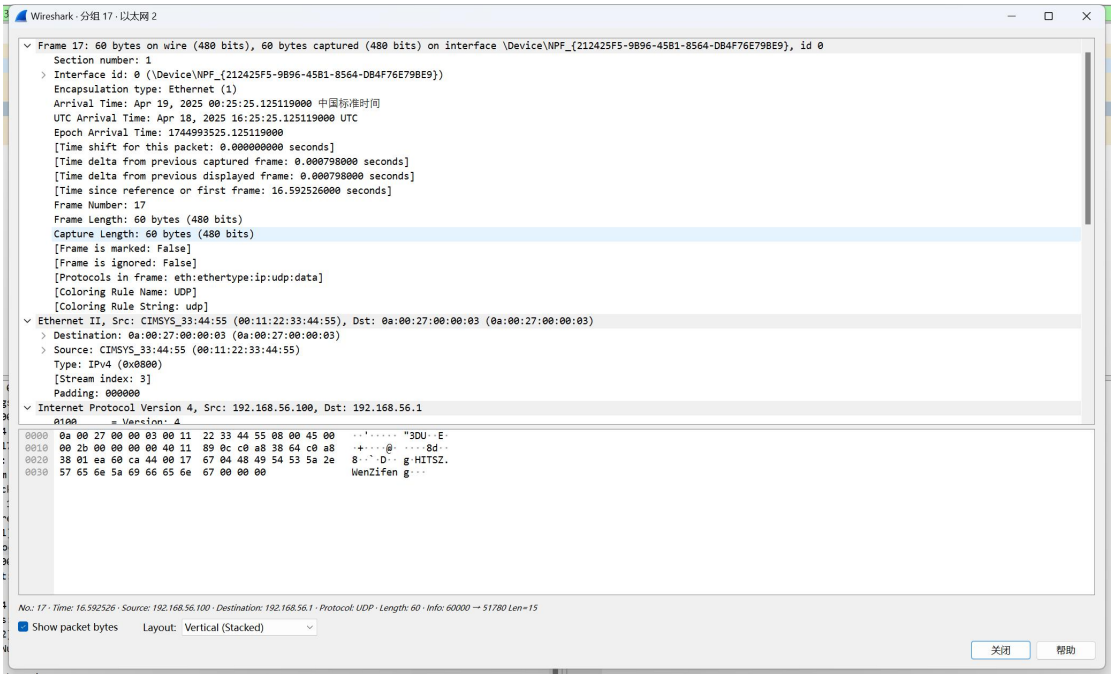


```
#ifdef TEST
#define NET_IF_IP          \
    {                      \
        192, 168, 163, 103 \
    } // 测试用网卡ip地址
#define NET_IF_MAC                          \
    {                                       \
        0x11, 0x22, 0x33, 0x44, 0x55, 0x66 \
    } // 测试用网卡mac地址
#else

#define NET_IF_IP        \
    {                    \
       192, 168, 56, 100 \
    } // 自定义网卡ip地址
#define NET_IF_MAC                          \
    {                                       \
        0x00, 0x11, 0x22, 0x33, 0x44, 0x55 \
    } // 自定义网卡mac地址
#endif
```

设定过滤条件为“eth.addr == 00:11:22:33:44:55”，其中“00:11:22:33:44:55”为自定义的虚拟网卡 MAC 地址，可以在 wireshark 中观察到相应的报文，如下图所示：



其中，“CIMSYS_33:44:55”为自定义网卡 MAC 地址，即“00:11:22:33:44:55”；“192.168.56.100”是自定义网卡 IP 地址，“192.168.56.1”则是本机网卡 IP 地址。



UDP 头部内容：

字段	值	分析说明
源端口号	60000	客户端临时端口
目的端口号	51780	自定义端口
长度	23 字节	包括头部 8 字节+载荷 5 字节
校验和	0x6704	校验正确

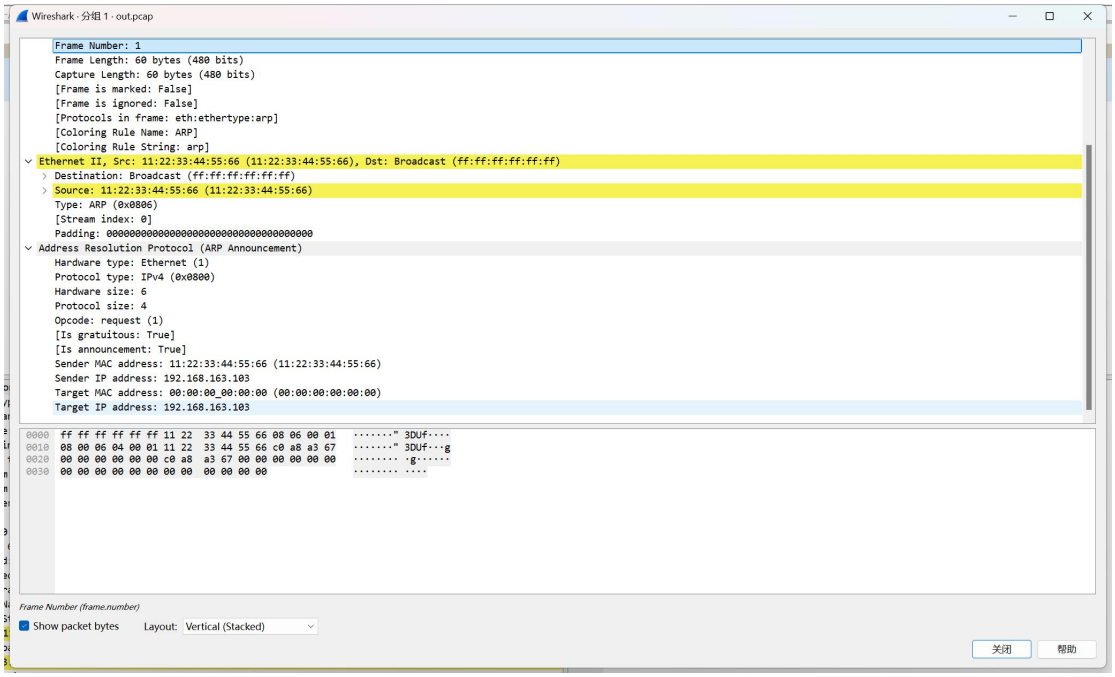
UDP 载荷内容：

1. 载荷长度：15 字节
2. 原始数据（Hex）：48 49 54 53 5a 2e 57 65 6e 5a 69 66 65 6e 67
3. ASCII 解码：HITSZ.WenZifeng

UDP 数据包结构清晰，所有字段都符合规范。校验和也计算正确，载荷数据 HITSZ.WenZifeng 和发送的一致。因此：该数据包完全达到了预期目标，是一个结构完整、内容准确的 UDP 包。

IP 总长度正确	43
UDP 长度正确	8 + 15 = 23
校验和有效	正确
源端口号	60000
目的端口号	51780
长度	15 字节
校验和	0x6704
载荷数据	4. 48 49 54 53 5a 2e 57 65 6e 5a 69 66 65 6e 67 HITSZ.WenZifeng

UDP 实验中，ARP 报文的分析：



1. 报文总长度为 60 字节（480 位），是一个标准的以太网帧

目的 MAC 地址：	ff:ff:ff:ff:ff:ff
源 MAC 地址：	11：22：33：44：55：66
协议类型：	ARP 协议（08 06）
硬件类型：	以太网（00 01）
上层协议类型：	IP 协议（08 00）
MAC 地址长度：	6 字节（06）
IP 地址长度：	4 字节（04）
操作类型：	ARP 请求（00 01）
源 IP 地址：	192.168.163.103
目的 IP 地址：	192.168.163.103
填充 padding：	000000000000000000000000000000000000

这是一台 MAC 地址为 11:22:33:44:55:66、IP 为 192.168.163.103 的设备，向全网广播一条 ARP 请求：“有没有人知道 192.168.163.103 的 MAC 地址？”但实际上是：“我就是 192.168.163.103，对应的 MAC 地址是 11:22:33:44:55:66，请大家更新你们的 ARP 缓存。”

TCP 协议实验结果及分析

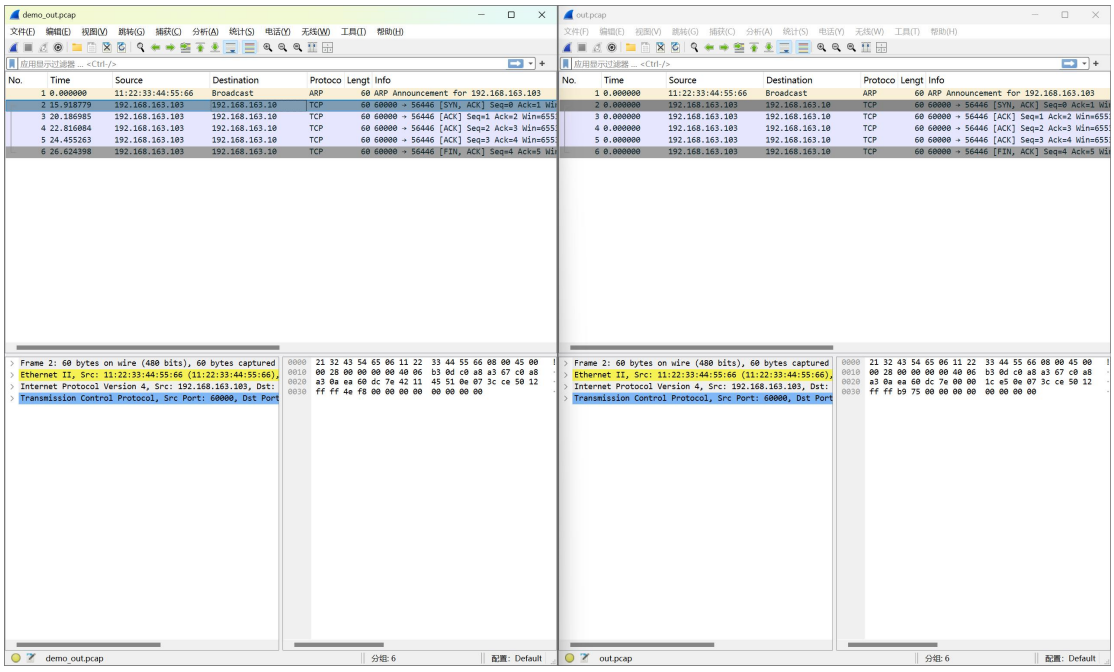
（展示 TCP 数据包的捕获截图，分析 TCP 连接的建立、数据传输和关闭过程。检查 TCP 头部的源端口号、目的端口号、序列号、确认号、标志位等字段，以及连接的状态转换。）

1. 实验结果：与 TCP 协议相关的测试：tcp_test 的测试已通过，如下图所示：

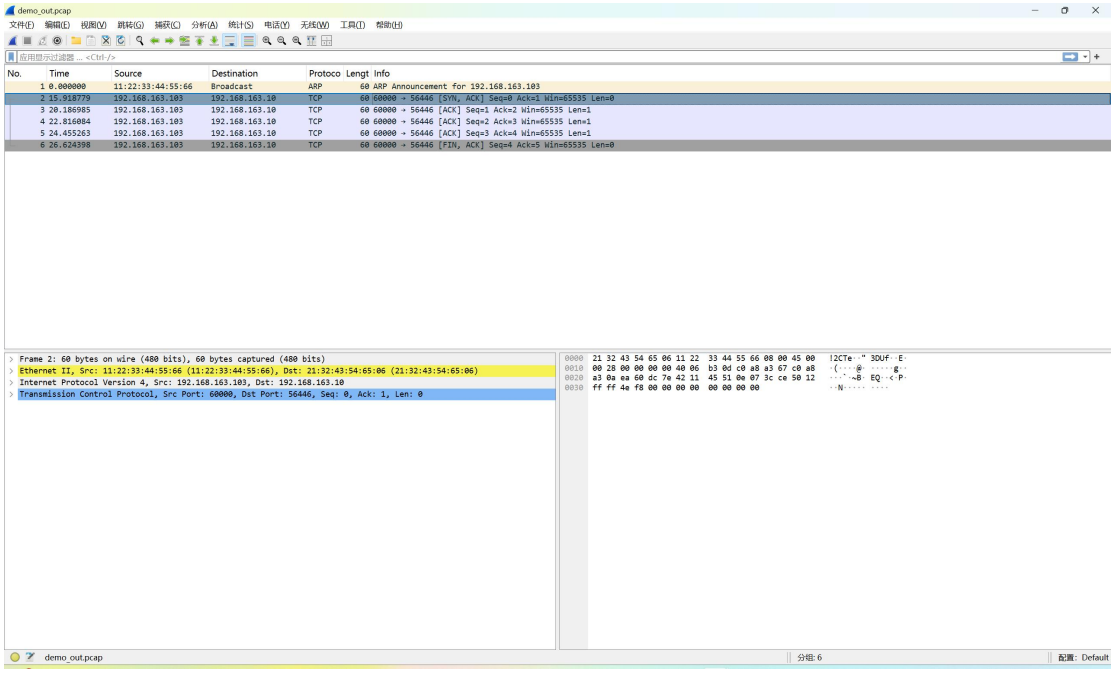
```
D:\code\net-lab\build>ctest -R tcp_test
Test project D:/code/net-lab/build
    Start 8: tcp_test
1/1 Test #8: tcp_test .........................   Passed    0.46 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =   0.47 sec
```

2. 展示 TCP 数据包的捕获截图，分析 TCP 连接的建立、数据传输和关闭过程。检查 TCP 头部的源端口号、目的端口号、序列号、确认号、标志位等字段，以及连接的状态转换。

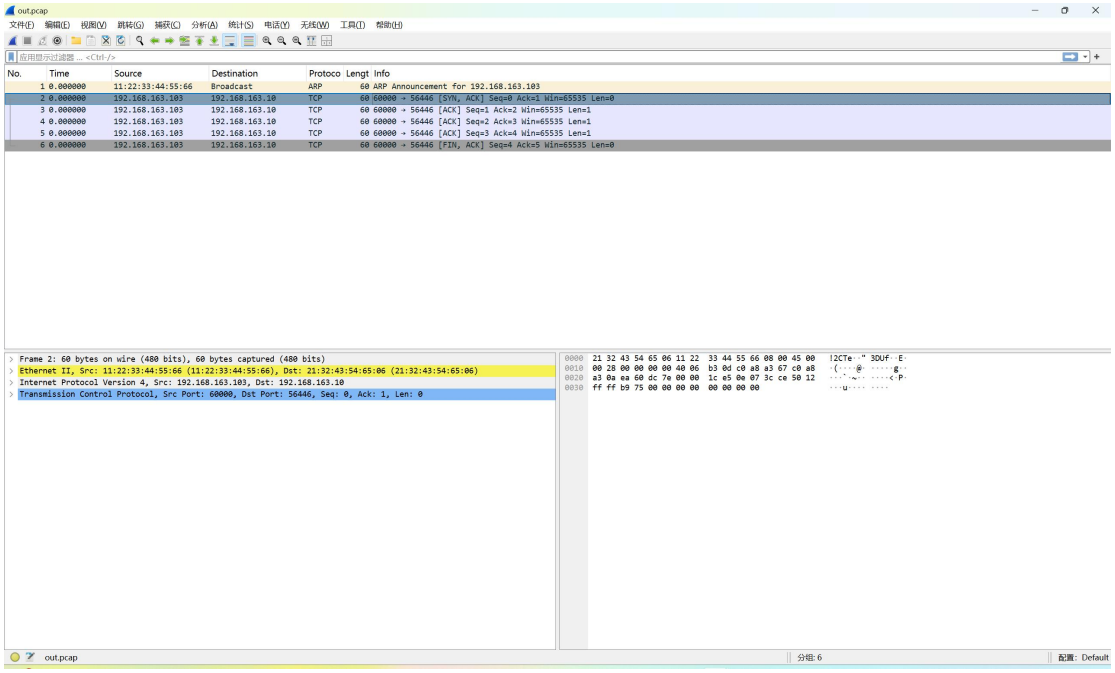
经过对比和分析，文件 out.pcap 与 demo_out.pcap 中的内容完全一致，因此测试结果符合预期。



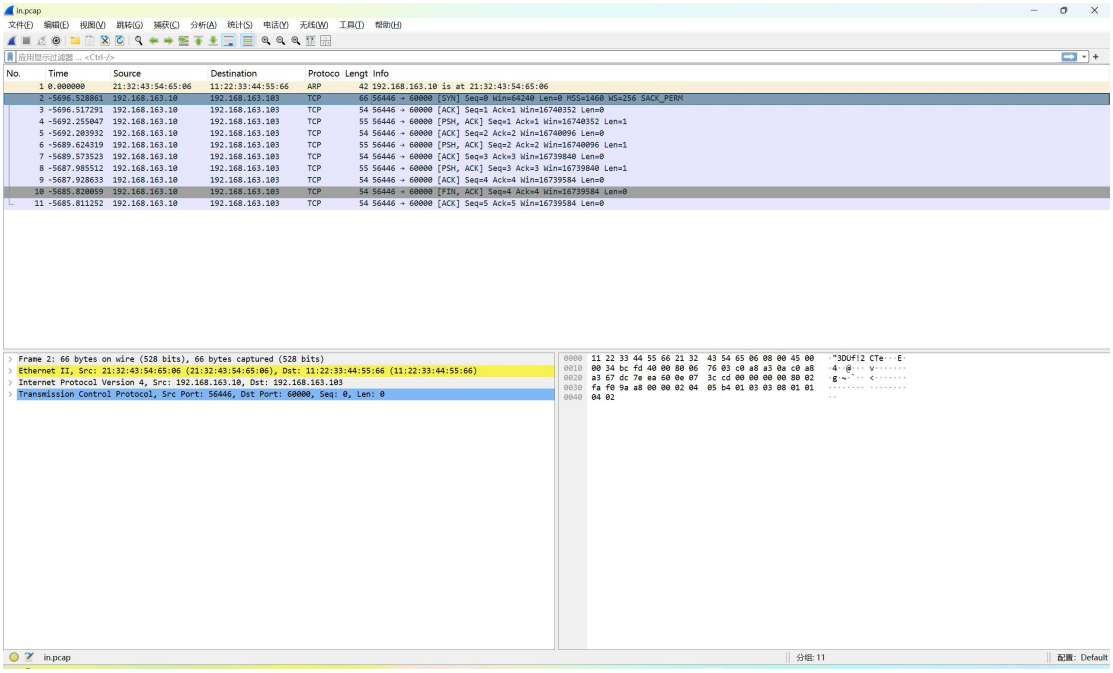
demo_out.pcap 截图：



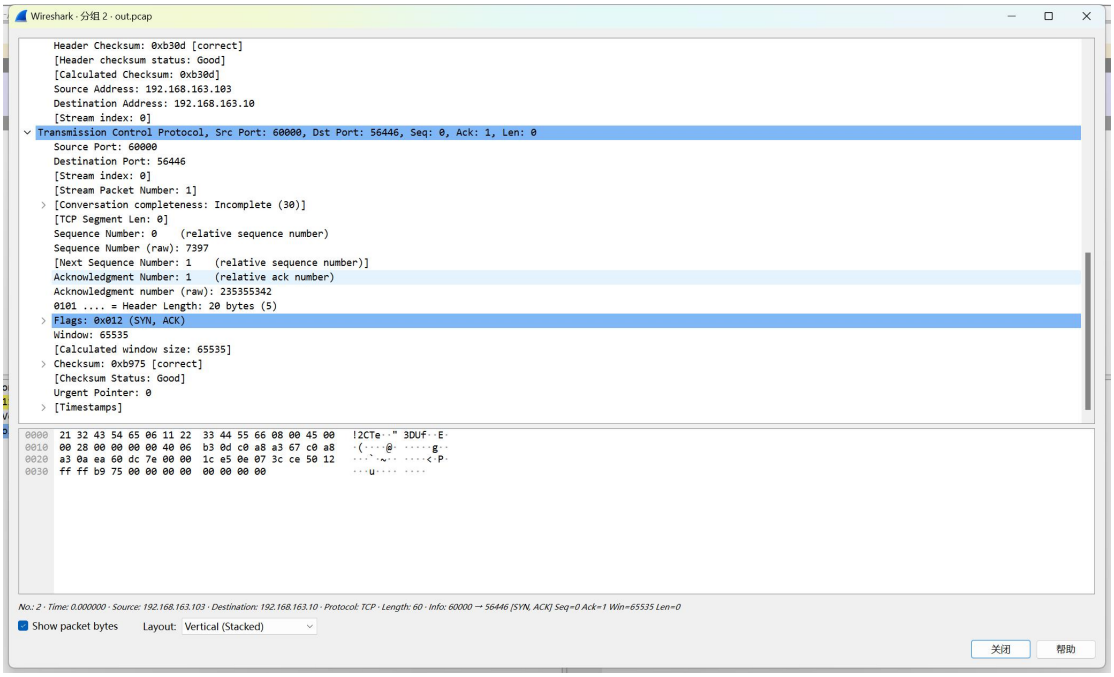
out.pcap 截图：



in.pcap 截图：



以握手第二步为例进行分析（其余握手过程分析类似从而省略）：



TCP 头部内容：

字段	值	分析说明
源端口号	60000	发起方主机的端口
目的端口号	59040	接收方主机的端口
序列号（Seq）	相对序列号：0 原始值：7397	发起方发送的第一个序列号
确认号（Ack）	相对确认号：1	确认接收到了对方的序列号

	原始值：235355342	为 0 的 SYN 包
标志位	0x012 → SYN + ACK	这是连接建立的第二步
窗口大小	65535	接受端的窗口大小（缓冲区）
数据长度	0	无负载，纯控制包
头部长度	20 bytes	无选项字段

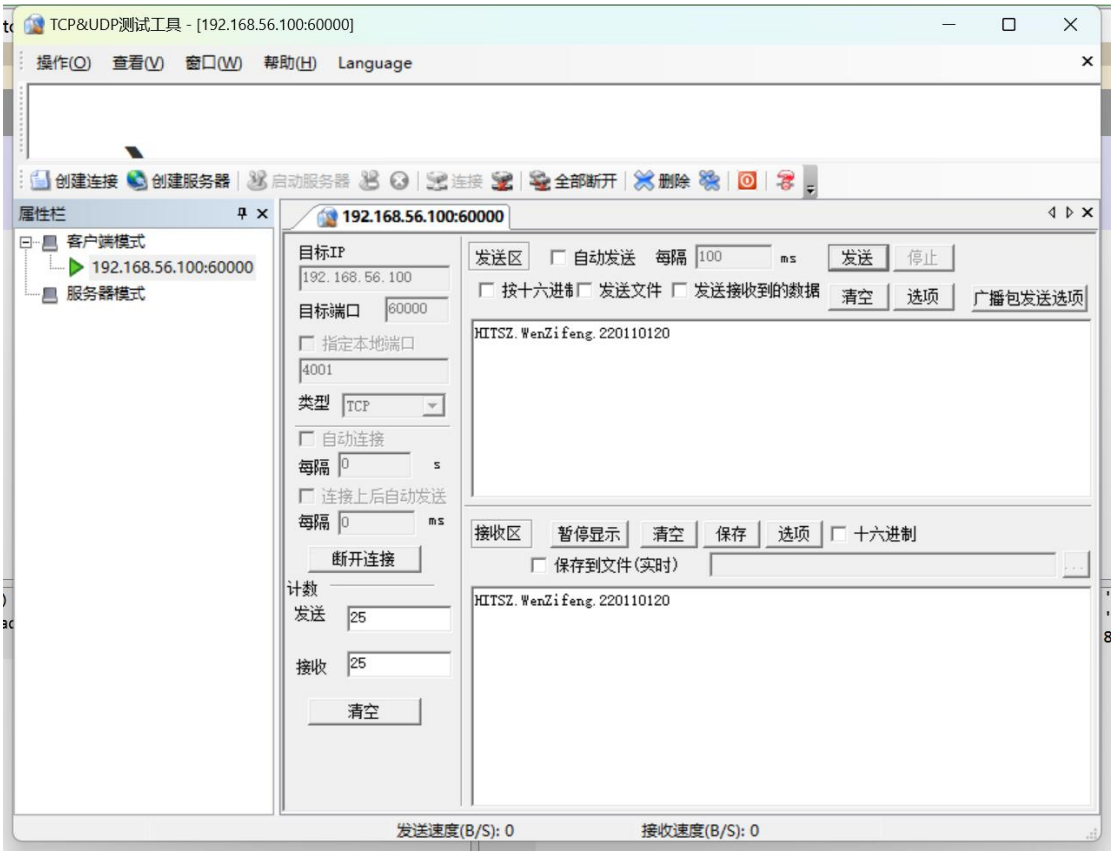
本帧的 Flags 为 `SYN + ACK`，是 **三次握手**的第 ② **步**：

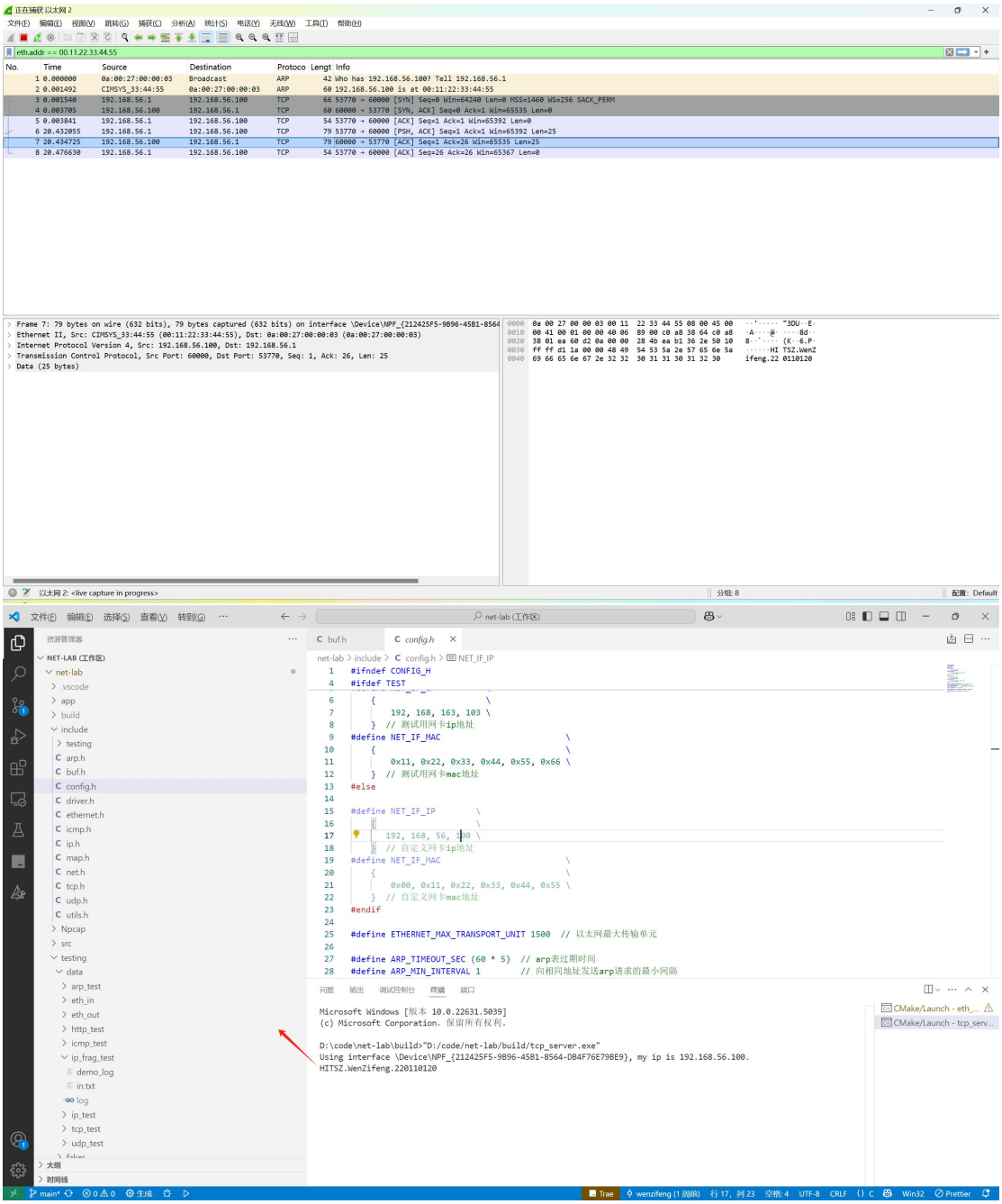
步骤	标志	源端口 → 目的端口	描述
①	SYN	A → B	A 请求建立连接，发送 SYN（例如：Seq=X）
②	SYN+ACK	B → A	B 同意连接，发送 SYN+ACK（Seq=Y, Ack=X+1） ← **当前帧**
③	ACK	A → B	A 再次确认，发送 ACK（Ack=Y+1），连接建立完成

该帧是一次 TCP 连接建立过程中的第二步（SYN+ACK）：

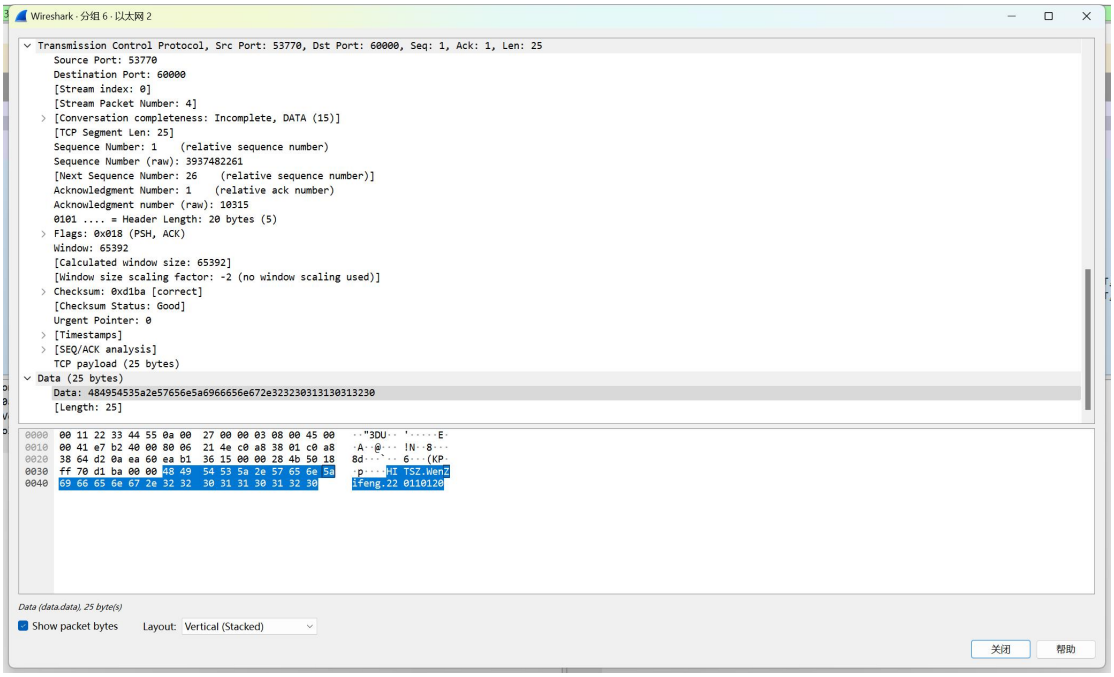
- 表明 192.168.163.103:60000 正在响应来自 192.168.163.10:56446 的连接请求。
- TCP 头部合法、校验和正确。
- 连接尚未完全建立（还差最后一个 ACK）。

3. 实验结果：与 TCP 协议相关的测试：tcp_server 的测试已通过，如下图所示：




正在捕获 以太网 2
eth.addr == 00:11:22:33:44:55
No. Time Source Destination Protocol Lenght Info
1 0.000000 0a:00:27:00:00:03 Broadcast ARP 42 Who has 192.168.56.100? Tell 192.168.56.1
2 0.001492 CIMSYS_33:44:55 0a:00:27:00:00:03 ARP 60 192.168.56.100 is at 00:11:22:33:44:55
3 0.001540 192.168.56.1 192.168.56.100 TCP 66 53770 → 60000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
4 0.003705 192.168.56.100 192.168.56.1 TCP 60 60000 → 53770 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
5 0.003841 192.168.56.1 192.168.56.100 TCP 54 53770 → 60000 [ACK] Seq=1 Ack=1 Win=65392 Len=0
6 20.432055 192.168.56.1 192.168.56.100 TCP 79 53770 → 60000 [PSH, ACK] Seq=1 Ack=1 Win=65392 Len=25
7 20.434725 192.168.56.100 192.168.56.1 TCP 79 60000 → 53770 [ACK] Seq=1 Ack=26 Win=65535 Len=25
8 20.476630 192.168.56.1 192.168.56.100 TCP 54 53770 → 60000 [ACK] Seq=26 Ack=26 Win=65367 Len=0
Frame 7: 79 bytes on wire (632 bits), 79 bytes captured (632 bits) on interface \Device\NPF_{212425F5-9B96-45B1-8564
Ethernet II, Src: CIMSYS_33:44:55 (00:11:22:33:44:55), Dst: 0a:00:27:00:00:03 (0a:00:27:00:00:03)
Internet Protocol Version 4, Src: 192.168.56.100, Dst: 192.168.56.1
Transmission Control Protocol, Src Port: 60000, Dst Port: 53770, Seq: 1, Ack: 26, Len: 25
Data (25 bytes)
0a 00 27 00 00 03 00 11 22 33 44 55 08 00 45 00
00 41 00 01 00 00 40 06 89 00 c0 a8 38 64 c0 a8
38 01 ea 60 d2 0a 00 00 28 4b ea b1 36 2e 50 10
ff ff d1 1a 00 00 48 49 54 53 5a 2e 57 65 6e 5a
69 66 65 6e 67 2e 32 32 30 31 31 30 31 32 30
以太网 2: <live capture in progress>
分组: 8
配置: Default
文件(F) 编辑(E) 选择(S) 查看(V) 转到(G)
net-lab (工作区)
资源管理器
NET-LAB (工作区)
net-lab
.vscode
app
build
include
testing
arp.h
buf.h
config.h
driver.h
ethernet.h
icmp.h
ip.h
map.h
net.h
tcp.h
udp.h
utils.h
Npcap
src
testing
data
arp_test
eth_in
eth_out
http_test
icmp_test
ip_frag_test
demo_log
in.txt
log
ip_test
tcp_test
udp_test
大纲
时间线
buf.h
config.h
net-lab > include > config.h > NET_IF_IP
#ifndef CONFIG_H
#ifdef TEST
{ \
192, 168, 163, 103 \
} // 测试用网卡ip地址
#define NET_IF_MAC \
{ \
0x11, 0x22, 0x33, 0x44, 0x55, 0x66 \
} // 测试用网卡mac地址
#else
#define NET_IF_IP \
{ \
192, 168, 56, 100 \
} // 自定义网卡ip地址
#define NET_IF_MAC \
{ \
0x00, 0x11, 0x22, 0x33, 0x44, 0x55 \
} // 自定义网卡mac地址
#endif
#define ETHERNET_MAX_TRANSPORT_UNIT 1500 // 以太网最大传输单元
#define ARP_TIMEOUT_SEC (60 * 5) // arp表过期时间
#define ARP_MIN_INTERVAL 1 // 向相同地址发送arp请求的最小间隔
问题 输出 调试控制台 终端 端口
Microsoft Windows [版本 10.0.22631.5039]
(c) Microsoft Corporation。保留所有权利。
D:\code\net-lab\build>"D:/code/net-lab/build/tcp_server.exe"
Using interface \Device\NPF_{212425F5-9B96-45B1-8564-DB4F76E79BE9}, my ip is 192.168.56.100.
HITSZ.WenZifeng.220110120
CMake/Launch - eth_...
CMake/Launch - tcp_serv...
main*
生成
行 17，列 23
空格: 4
UTF-8
CRLF
Win32
Prettier

分析 TCP 传输数据过程的报文：

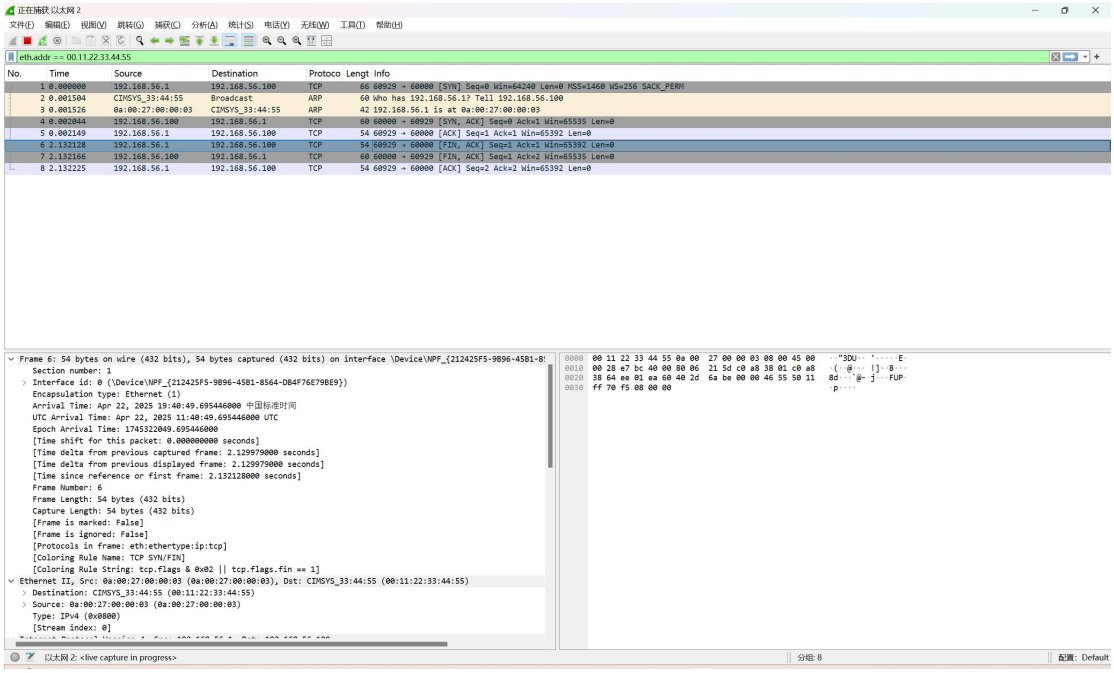


TCP 头部内容：

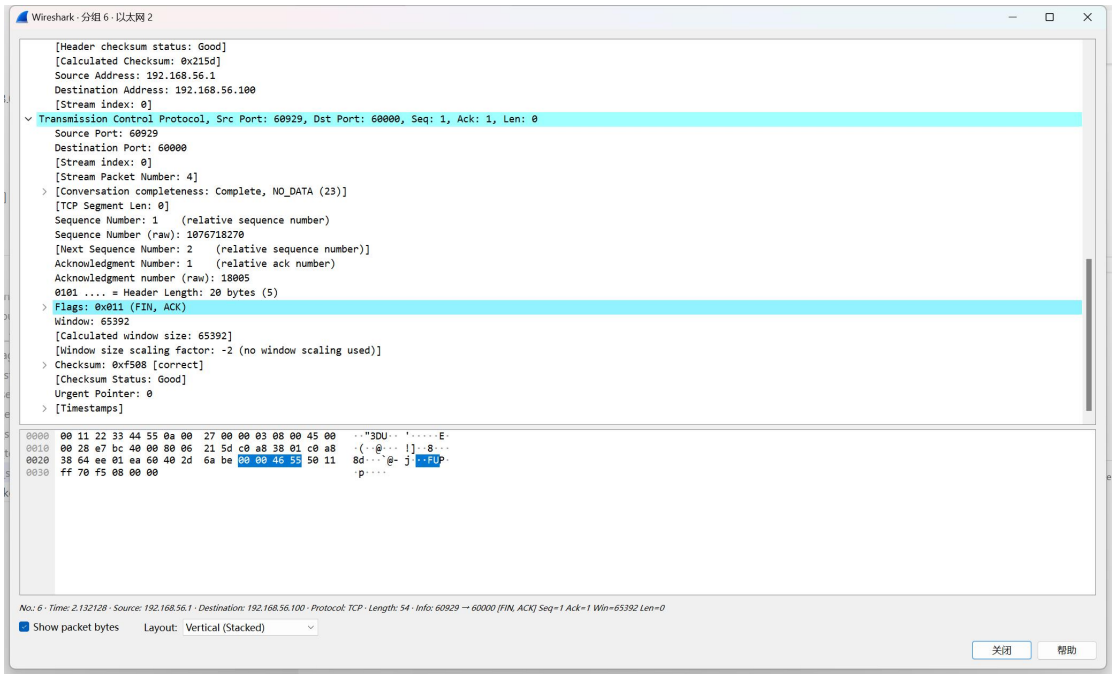
字段	值	分析说明
源端口号	53770	客户端的端口号
目的端口号	60000	服务器端口号
序列号（Seq）	相对序列号：1 原始值：3937482261	本报文中第一个字节的序列号，表示该连接在前一帧中已经完成了握手
确认号（Ack）	相对确认号：1 原始值：10315	确认号为 1，表示“我已经收到了你发送的序列号为 0 的数据”
标志位	0x018 （PSH, ACK）	该数据段是普通数据传输，同时带确认号
窗口大小	65392	用于流量控制
数据长度	25 bytes	有实际数据（正文部分）
头部长度	20 bytes (5)	没有额外 TCP 选项
Payload	484954535a2e57656e5a6966656e672e323230313130313230 16 进制	ASCII 解码为：HITSZ.WenZifeng.220110120

- 此时 TCP 状态已经进入 ESTABLISHED（已连接）
- 第 6 帧是连接建立后，客户端向服务端发送的首个带有数据的数据段，携带用户层负载 "HITSZ.WenZifeng.220110120"

断开连接后：



分析 TCP 关闭过程的其中一个报文；



TCP 头部内容：

字段	值	分析说明
源端口号	60929	客户端的端口号
目的端口号	60000	服务器端口号
序列号（Seq）	相对序列号：1 原始值：1076718270	发送方这次发送的数据起始字节序号。由于是 FIN 报文，没有数据，但这个序列

		号用于后续确认。
确认号（Ack）	相对确认号：1 原始值：18005	表示期望收到对方的下一个字节序号，说明之前的数据都已成功接收。
标志位	0x011 (FIN, ACK)	表示此报文既是确认，也请求断开连接（即第一次挥手）
窗口大小	65392	用于流量控制
数据长度	0	无数据负载，仅用于控制连接状态

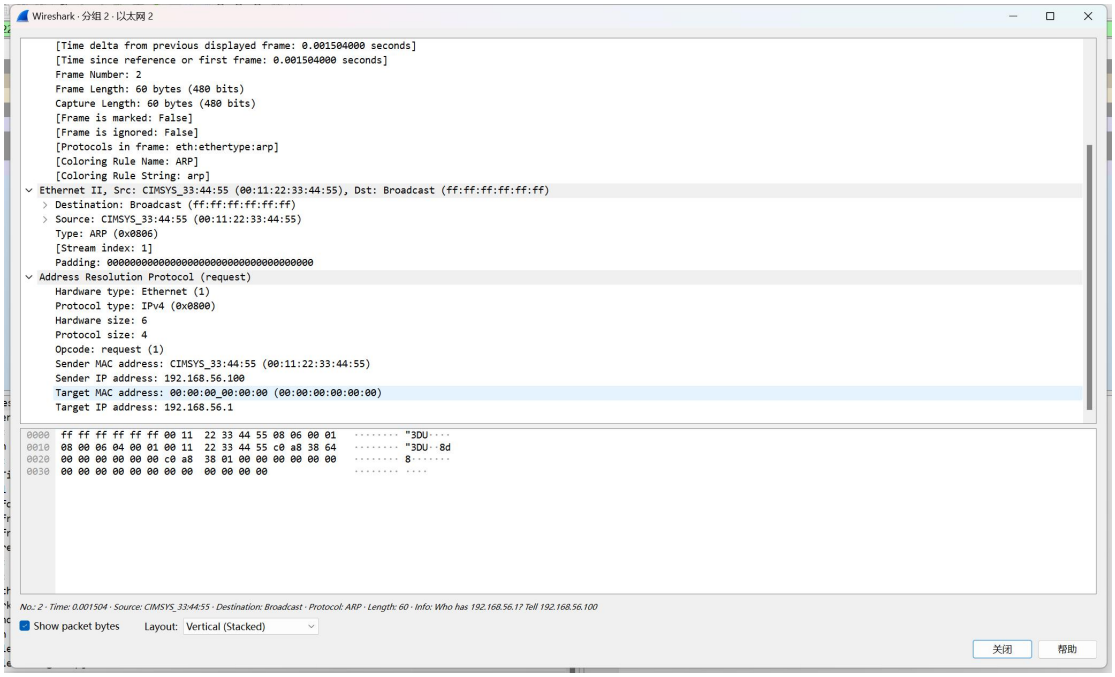
本帧是 TCP 连接断开过程中的第一次挥手：

源地址 192.168.56.1 的主机（端口 60929）向目标地址 192.168.56.100（端口 60000）发送一个 带有 FIN 和 ACK 标志 的 TCP 报文段，请求关闭当前连接。此时客户端从 ESTABLISHED（连接已建立）状态 进入 FIN_WAIT_1 状态，等待对方的确认（ACK）。

TCP 四次挥手：

1. 客户端 → 服务器：FIN, ACK（你这帧）
2. 服务器 → 客户端：ACK
3. 服务器 → 客户端：FIN
4. 客户端 → 服务器：ACK

TCP 实验中，ARP 报文的分析：



2. 报文总长度为 60 字节（480 位），是一个标准的以太网帧

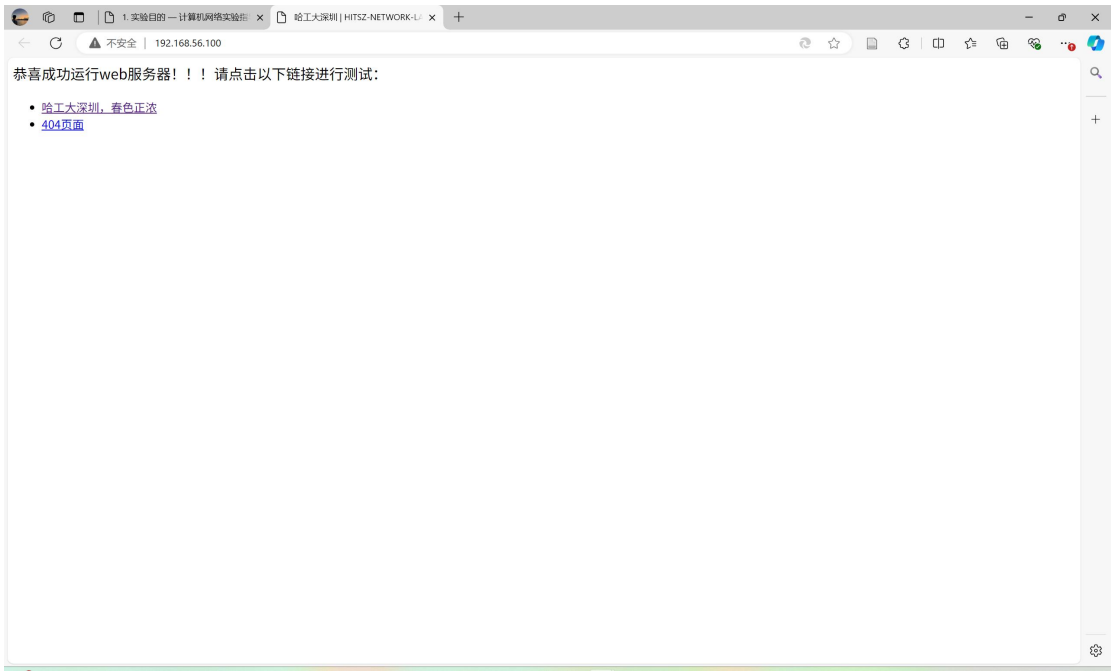
目的 MAC 地址：	ff:ff:ff:ff:ff:ff
源 MAC 地址：	11：22：33：44：55：66
协议类型：	ARP 协议（08 06）
硬件类型：	以太网（00 01）
上层协议类型：	IPv4 协议（08 00）
MAC 地址长度：	6 字节（06）
IP 地址长度：	4 字节（04）
操作类型：	ARP 请求（00 01）
源 IP 地址：	192.168.163.100
目的 IP 地址：	192.168.56.1
目标 MAC 地址	00：00：00：00：00：00
填充 padding：	000000000000000000000000000000000000

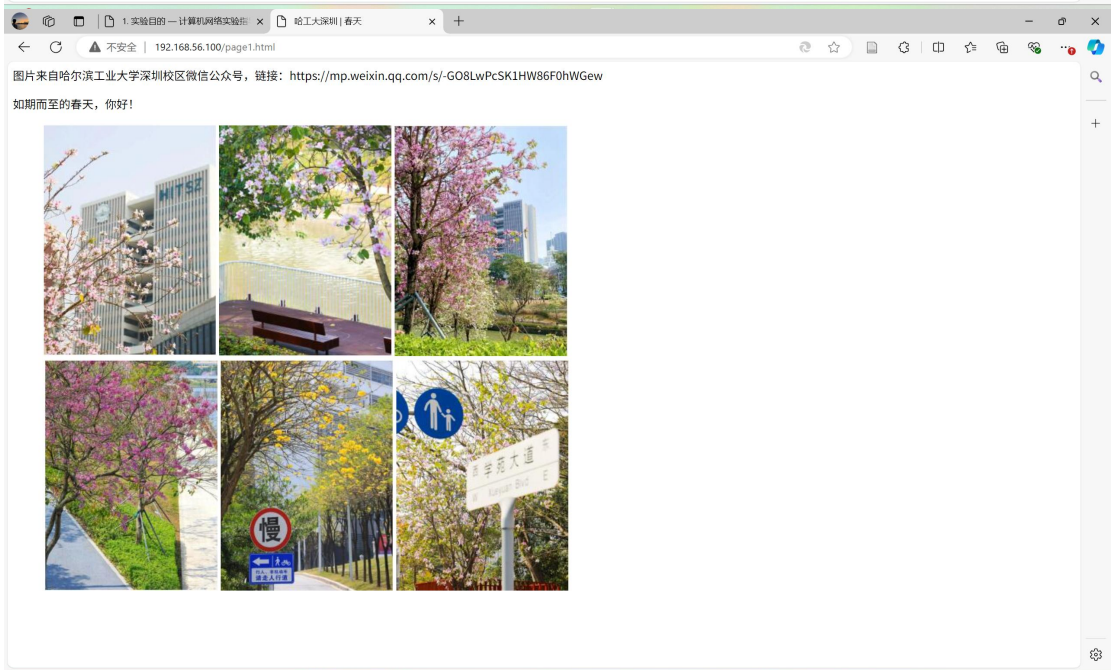
这是一条标准的 ARP 请求帧，用于问“谁是 192.168.56.1？请告诉我你的 MAC 地址”。

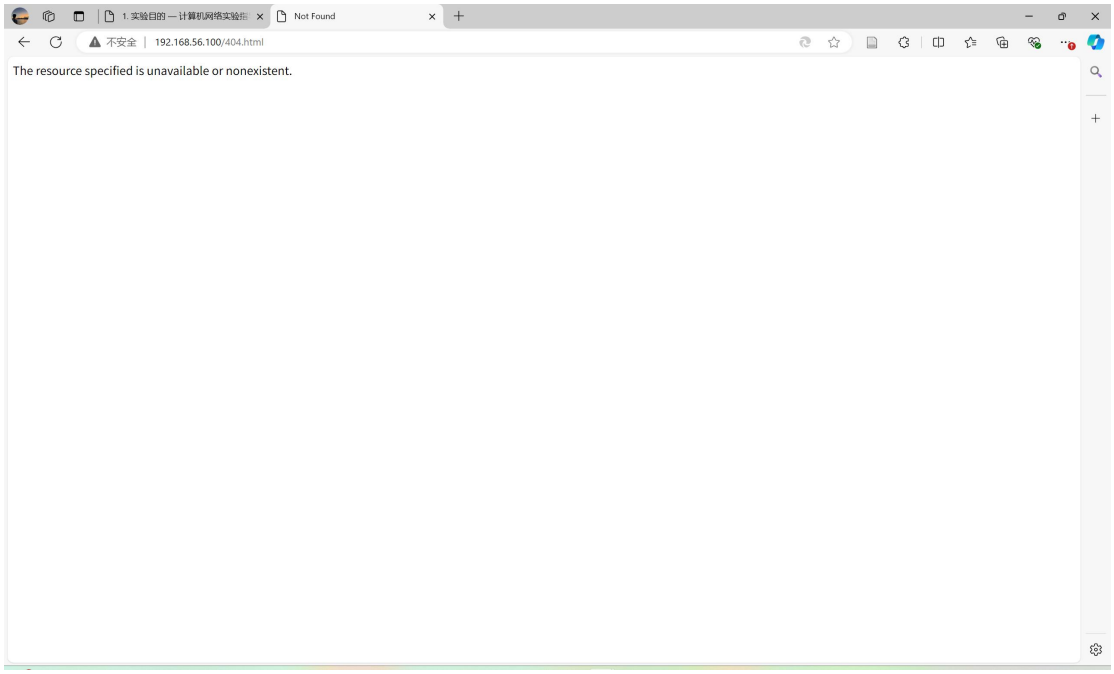
6. web 服务器实验结果及分析

（展示 web 服务器的请求和响应过程截图，分析 HTTP 请求和响应的格式、内容。检查请求方法、请求 URL、请求头、响应状态码、响应头、响应体等部分。）

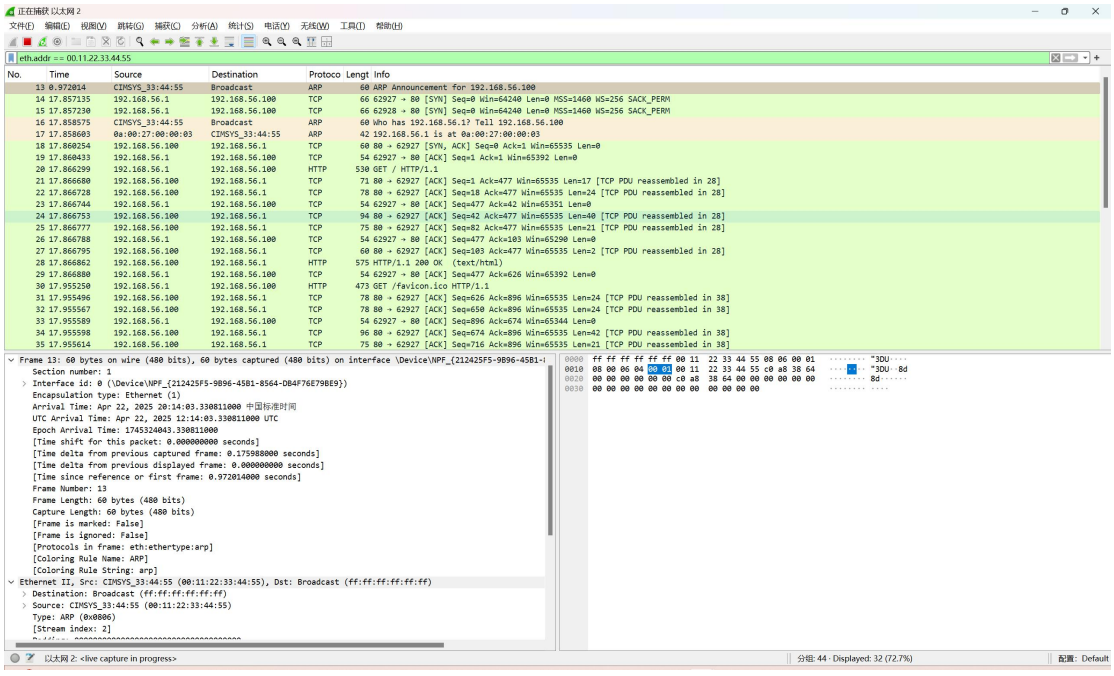
1. 展示 web 服务器的请求和响应过程截图：



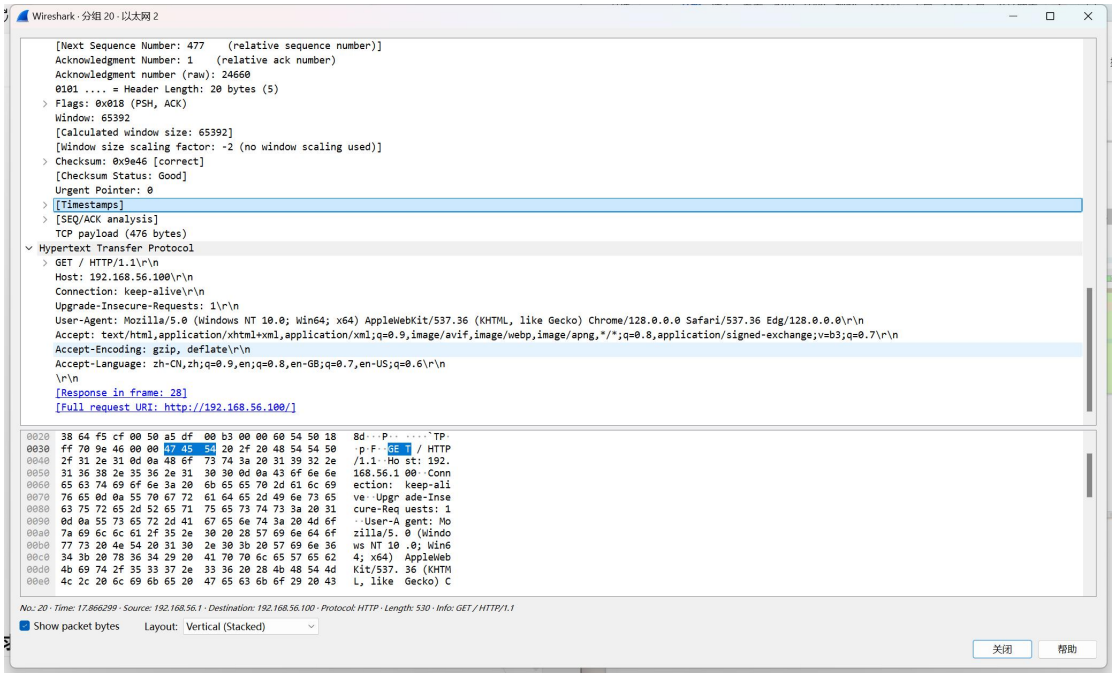




2. 展示 web 服务器的请求和响应过程截图，分析 HTTP 请求和响应的格式、内容。检查请求方法、请求 URL、请求头、响应状态码、响应头、响应体等部分。）



分析 HTTP 请求报文：



HTTP 请求分析

请求 (Frame 20)：

- 请求方法: GET
- 请求 URL: http://192.168.56.100/
- 请求协议: HTTP/1.1

请求头：

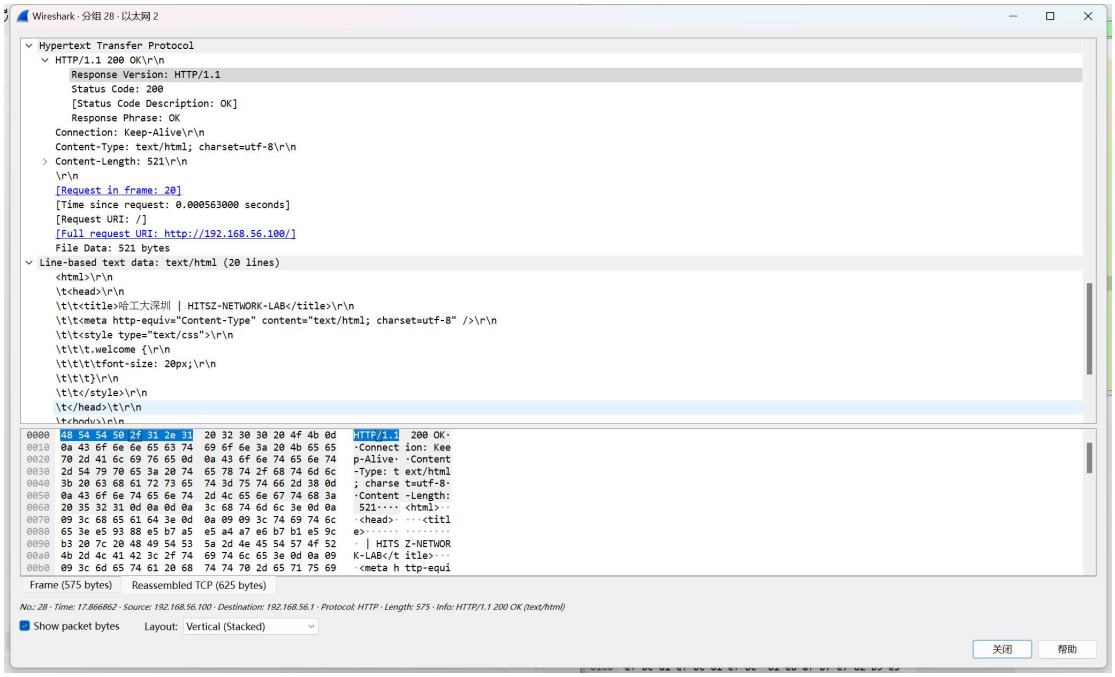
- Host: 192.168.56.100 — 目标服务器的 IP 地址。
- Connection: keep-alive — 保持连接，以便在后续请求中复用。
- Upgrade-Insecure-Requests: 1 — 请求升级不安全的内容。
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0 — 客户端浏览器信息（Chrome 浏览器在 Windows 10 系统上）。
- Accept: text/html, application/xhtml+xml, application/xml;q=0.9, image/avif, image/webp, image/apng,*/*;q=0.8, application/signed-exchange;v=b3;q=0.7 — 浏览器接受的内容类型（HTML, XML, 图片等）。
- Accept-Encoding: gzip, deflate — 浏览器支持的压缩方式（gzip、deflate）。
- Accept-Language: zh-CN, zh;q=0.9, en;q=0.8, en-GB;q=0.7, en-US;q=0.6 — 浏览器支持的语言（优先中文，接着英文）。

请求体：

- 请求体为空，因为这是一个简单的 GET 请求。

这个 HTTP 请求是一个标准的浏览器发出的 GET 请求，目的是从目标服务器 192.168.56.100 获取首页内容。请求头部携带了浏览器信息、支持的语言、接受的内容类型等，表明了客户端的需求。

分析 HTTP 响应报文：



HTTP 请求部分

- 请求方法：GET
- 请求 URL：http://192.168.56.100/
- 请求头：
  - Connection: Keep-Alive：要求保持连接，连接不会立即关闭。
  - Content-Type: text/html; charset=utf-8：说明响应的内容类型是 HTML 格式，字符编码为 UTF-8。

HTTP 响应部分

- 响应状态码：200 OK
  - 200 状态码表示请求已成功，并且服务器返回了请求的资源。
- 响应头：
  - Connection: Keep-Alive：与请求头类似，响应也表明连接将保持。
  - Content-Type: text/html; charset=utf-8：响应的内容类型为 HTML 格式，字符编码是 UTF-8。
  - Content-Length: 521：响应体的长度为 521 字节。

响应体部分

响应体包含的 HTML 页面展示了基本的网页结构，包括标题、样式和正文内容，表示服务器正在运行并成功响应请求。内容如下：

html

复制代码

```
<html>
  <head>
    <title>哈工大深圳 | HITSZ-NETWORK-LAB</title>
```

```
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
    <style type="text/css">
      .welcome {
        font-size: 20px;
      }
    </style>
  </head>
  <body>
    <p class="welcome">恭喜成功运行 web 服务器！！！请点击以下链接进行测试：</p>
    <div>
      <ul>
        <li><a href="page1.html">哈工大深圳，春色正浓</a></li>
        <li><a href="404.html">404 页面</a></li>
      </ul>
    </div>
  </body>
</html>
```

这个抓包数据展示了一个成功的 HTTP 请求和响应过程，服务器通过响应返回了一个简单的 HTML 页面。你可以进一步测试其他页面链接，或者查看错误页面如何响应。

三、 实验中遇到的问题及解决方法

（详细描述在设计或测试过程中遇到的问题，包括错误描述、排查过程以及最终的解决方案。）

```
解释代码 | 注释代码 | 生成单测 | ×
118  void tcp_out(tcp_conn_t *tcp_conn, buf_t *buf, uint16_t src_port, uint8_t *dst_ip, uint16_t dst_port, uint8_t flags) {
119      /* ============================== TODO 1 BEGIN ============================== */
120      // Step1 添加 TCP 报头
121      // 首先调用buf_add_header()函数添加TCP报头
122      buf_add_header(buf, sizeof(tcp_hdr_t));
123      // Step2 填充 TCP 首部字段：
124      // 将 win 字段设置为最大 TCP 窗口大小（TCP_MAX_WINDOW_SIZE）
125      // uptr 字段置为零。操作时需注意字节序问题。
126      tcp_hdr_t *hdr = (tcp_hdr_t *) buf->data;
127      hdr->src_port16 = swap16(src_port);
128      hdr->dst_port16 = swap16(dst_port);
129      hdr->seq = swap32(tcp_conn->seq);
130      hdr->ack = swap32(tcp_conn->ack);
131      hdr->doff = (sizeof(tcp_hdr_t) / 4) << 4;
132      hdr->flags = flags;
133      hdr->win = TCP_MAX_WINDOW_SIZE;
134      hdr->uptr = 0;
135      // Step3 计算并填充校验和：
136      // 先把校验和字段填充为 0，接着调用 transport_checksum() 函数计算校验和
137      // 最后将计算得到的校验和结果填入校验和字段
138      hdr->checksum16 = 0;
139      hdr->checksum16 = transport_checksum(
140          NET_PROTOCOL_TCP,
141          buf,
142          net_if_ip,
143          dst_ip
144      );
145      // Step4 发送 TCP 数据报：
146      // 调用 ip_out() 函数，将处理好的 TCP 数据报发送出去
147      ip_out(buf, dst_ip, NET_PROTOCOL_TCP);
148      /* ============================== TODO 1 END ============================== */
149  }
```

在设计或测试过程中遇到的问题:

在本次实验中，我遇到了一个与 TCP 报文构造相关的问题，具体表现为服务器端在接收 TCP 报文时，报文头部为空，导致连接无法正常建立或数据无法正确解析。这个问题初看似乎与网络环境或缓存机制有关，但最终我通过 GDB 调试结合 Wireshark 抓包分析，才找到了根本原因。

问题描述：

在调试 tcp_server 时，发现服务器在接收到 TCP 报文后，报文头部字段为空或不合理（例如头部长度为 0），从而引起后续处理逻辑失败，包括无法正确提取 seq、ack、flags 等字段。

排查过程：

基础检查：起初我检查了 TCP 的连接状态和接收逻辑是否正常，确认连接建立过程无误，客户端数据也成功发送。

使用 GDB 逐步调试：在 tcp_out() 函数中，我重点关注了 TCP 报头的构造部分。通过在 tcp_out 中下断点、单步执行，验证每个字段的赋值是否正确。结果发现大部分字段值都正常，唯独 hdr->doff 设置存在可疑之处。

怀疑 doff 字段设置问题： doff 字段（data offset）表示 TCP 头部长度，以 32-bit 为单位存储在高 4 位中。因此，它应当使用如下位移设置：

```
hdr->doff = (sizeof(tcp_hdr_t) / 4) << 4;
```

然而，早期代码中没有进行左移操作，仅写为：

```
hdr->doff = sizeof(tcp_hdr_t) / 4;
```

这样 doff 字段的值就落在了低位，协议栈解析 TCP 头时认为头部长度为 0，从而忽

略了整个头部，最终表现为“头部为空”。

使用 Wireshark 抓包验证：为进一步验证这一判断，我使用 Wireshark 抓取 TCP 报文。果然，分析发现发出的 TCP 报文中 “Header Length” 字段为 0（正确应为 20 字节，即 5 × 4 字节），印证了 doff 设置不当的问题。

解决方法：

我对 tcp_out() 函数中的 doff 字段赋值语句进行了修正：

```
hdr->doff = (sizeof(tcp_hdr_t) / 4) << 4;
```

这一修改确保了头部长度被正确编码在 TCP 报文中。之后重新编译并运行程序，通过 GDB 再次调试并用 Wireshark 抓包确认，发现 TCP 头部字段已经恢复正常，服务器能够成功解析 TCP 报文。

总结

此次问题暴露出 TCP 协议头构造时对字节级细节的要求非常严格。一个字段的位移错误就可能造成整个头部失效。此外，本次调试让我更加熟练地掌握了 Wireshark 抓包工具与 GDB 联合使用的技巧。通过这次排查，我深刻体会到协议实现中的位操作、字段填充与字节序等底层细节的重要性，稍有不慎就会导致整个网络通信链路的异常。

四、 实验收获和建议

（总结配置实验及协议栈实验过程中的实践收获，结合实操体验针对性提出实验流程优化及环境完善建议，为后续实验教学与研究的迭代改进提供参考依据。）

本次配置实验及协议栈实验深入涵盖了从数据链路层到应用层的协议实现，让我对计算机网络协议的工作机制有了系统且直观的理解。通过逐层构建并调试协议栈，我不仅掌握了各层的功能和实现流程，还在调试过程中体会到了协议之间如何协同配合完成网络通信的全过程。

层级	作用	主要函数
Ethernet 层	接收/发送帧	`ethernet_in`, `ethernet_out`
ARP 协议	地址解析	`arp_in`, `arp_req`, `arp_resp`, `arp_out`
IP 层	分发协议、分片、校验	`ip_in`, `ip_out`, `checksum16`, `ip_fragment_out`
ICMP 协议	Ping、不可达等	`icmp_in`, `icmp_resp`, `icmp_unreachable`
UDP 协议	轻量级通信	`udp_in`, `udp_out`, `transport_checksum`
TCP 协议	连接通信	`tcp_in`, `tcp_out`
Web服务器	应用层响应	`http_respond`

实践过程中使用的关键工具：

- cmake：编译环境管理与构建配置
- wireshark：抓包分析网络协议行为，精准定位错误（如 TCP doff 字段错误）
- gdb：内存级别调试工具，辅助追踪函数行为、变量变化
- TCP/UDP 调试工具：验证服务器端口响应情况，测试 UDP/TCP 收发流程

实践收获

1. 对协议实现细节有了更深刻认识

每一层协议都涉及精细的字段操作和格式规范，比如 TCP 报文中 doff 字段的计算（需左移四位），IP 层中的分片偏移计算，以及 checksum 校验的字节顺序问题。这些过去在课堂上只是“了解”的细节，在调试阶段变成了“必须理解、必须掌握”的关键点。

2. 调试能力大幅提升

在调试 tcp_out 函数时，我曾遇到 TCP 报头为空的问题。通过 GDB 查看内存布局、再结合 Wireshark 抓包分析，最终定位到是 doff 字段未左移四位，导致接收端无法正确识别 TCP 报文。这次经历不仅让我学会了分析抓包数据，还提升了对比规范文档与代码行为之间差异的能力。

3. 增强了“动手实现 + 调试验证”的思维方式

实现每个协议之后，我都通过真实数据包抓取验证协议行为是否正确。相较于只看输出结果，这种“用工具看过程”的方式，让我更能理解协议运行的动态过程，比如 TCP 三次握手、IP 分片重组、ARP 请求与响应的时序逻辑等。

4. 更理解协议栈中各层的职责与协作关系

在实现 Web 服务器模块时，我深刻体会到：HTTP 的实现高度依赖 TCP 的可靠性传输，而 TCP 又依赖 IP 层的寻址与分发，每一层协议都构成了上层正常运行的基础。只有底层稳定，上层应用才能正常响应。

实验流程优化建议

1. 建议在每一层协议调试前提供“预期输出”示例

比如 TCP 报文应包含哪些字段、字段值应该是什么样子。如果能结合抓包截图讲解一两个典型包的结构，将更有利于学生自检和验证。

2. 建议增加异常输入与错误注入实验

目前实验流程主要面向“正常路径”，建议引入如 IP 分片丢包、TCP 重传、伪造 ARP 包等“非正常路径”的处理，强化鲁棒性编程思维。

```
D:\code\net-lab\build>ctest
Test project D:/code/net-lab/build
    Start 1: eth_in
1/8 Test #1: eth_in ............................   Passed    0.05 sec
    Start 2: eth_out
2/8 Test #2: eth_out ...........................   Passed    0.03 sec
    Start 3: arp_test
3/8 Test #3: arp_test ..........................   Passed    0.05 sec
    Start 4: ip_test
4/8 Test #4: ip_test ...........................   Passed    0.06 sec
    Start 5: ip_frag_test
5/8 Test #5: ip_frag_test ......................   Passed    0.03 sec
    Start 6: icmp_test
6/8 Test #6: icmp_test .........................   Passed    0.05 sec
    Start 7: udp_test
7/8 Test #7: udp_test ..........................   Passed    0.04 sec
    Start 8: tcp_test
8/8 Test #8: tcp_test ..........................   Passed    0.05 sec

100% tests passed, 0 tests failed out of 8

Total Test time (real) =   0.40 sec
```